SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1737 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 3 (CO3 – Unit III: Knowledge and Reasoning)

Logical Agents, Model Checking versus Resolution, First-Order Knowledge Engineering, Unification, Chaining and a Hospital Medication-Safety Design

Item	Details
Student A	**Saai Vardhan D** – Register No. **192525205**
Student B (partner)	**[Saravanan T]** – Register No. **[1925260349]**
Course	CSA17 – Artificial Intelligence (L-T-P-C: 3-0-2-4)
Assignment	Assignment 3 – CO3 (Pair work, 50 marks, Tasks 1–7 and Contribution Record)
Language / tools	Python 3 (own implementations of every algorithm), Matplotlib; run in a Linux container with PYTHONHASHSEED=0 for reproducible counts
Date of submission	29 October 2026

Method followed in every task: Represent → Infer → Implement → Measure → Debug → Evaluate → Defend. Each result follows the evidence chain *source code → test input → expected behaviour → actual output → performance metrics over at least three sizes → interpretation*. Operation counts (models, clauses, inference steps, goals, unifications) are the primary evidence; time is secondary. Where a task states an expected answer (query suites, test cases), the expected answer was written from the specification before the program was run, and failures are reported and debugged openly.

Pair-work note. Both roles of every task are written out in this report. The parts led by Student A and Student B are labelled, the cross-understanding answers show that each student can explain the partner's component, and the Contribution Record at the end lists the primary contribution and the joint work for each task.

Contents and Summary of Findings

Task	Title	Marks
1	Logical agents: knowledge bases, entailment and reasoning patterns (Wumpus world)	6
2	Model checking versus resolution (smart-building policy)	7
3	Knowledge engineering in first-order logic (exam-eligibility advisor)	7
4	Unification and lifted inference	7
5	First-order resolution	6
6	Forward and backward chaining (organisation hierarchy)	7
7	Day-4 design challenge: hospital medication-safety advisor	10
	Total	**50**

Key findings (all measured by the programs in the appendices)

Task 1. A 4×4 Wumpus KB has 64 symbols and 155 sentences, so 2^64 models. Restricting to the 18 symbols relevant to a query cut the work to 295,561 models, and inference rules answered "is [2,2] safe?" in **8 steps** against 262,144 models (65,458×).

Task 2. Resolution on implication chains of length 80 generated 3,166 clauses; model checking needs 2^81. The reverse also holds: on the pigeonhole family PHP(4,3) model checking needed 8,192 models in 0.014 s while resolution generated 5,738 clauses (13.2 s), and PHP(5,4) was not finished by resolution in 60 s. The structural property is **symbol count versus clause interaction**.

Task 3. The propositional encoding of the exam regulations needs $s\cdot(2 + 4c + c(c-1)/2)$ rules (816,000 for 3,000 students × 20 courses); the first-order encoding needs **10 rules for any s and c** and 8× fewer sentences in total. Unification makes a single query slower per call, so the first-order form pays off when the KB is large and queries are fewer than a break-even of 1 to 3,376 queries.

Task 4. Ground instances grow as k^r (27,000 for 3 variables at k = 30); function symbols make ground terms grow doubly exponentially (2, 8, 74, 5,552, 30,830,258). Lifted backward chaining needed 180 goals for k = 60 where propositionalising produced 219,600 ground rules.

Task 5. The first-order prover proved that an authorised employee exists. Plain resolution needed 507 steps; unit preference 53; set of support 128. With a Skolem function the non-entailed query never saturates (step cap hit), while on the finite KB it saturates: first-order entailment is only semi-decidable.

Task 6. Forward chaining derived 154,671 facts (3.07 million unifications) for 11,167 base facts, **of which 100 % (to two decimals) were irrelevant to one approval query**, which backward chaining answered in 23 goals. A left-recursive rule never terminates; a loop check terminates but loses 462 of 466 answers, while tabling is complete.

Task 7. The final design (Data log definite clauses, facts indexed by patient, rules indexed by trigger drug, backward chaining with proof-tree explanations) passed 37/37 test cases after fixing 3 failures found by the suite. Latency stayed nearly flat with KB size (80 to 178 µs mean at 42 to 2,000 rules) while the unindexed baseline grew linearly (29,625 µs at 2,000 rules, 166× slower); 50,000 patients were served at a mean 158 µs per check.

Task 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 marks)

1.1 Shared group work: the Wumpus-world knowledge base

The world is an n × n grid. Square [x, y] has column x and row y; [1,1] is the start square. Each square has four proposition symbols: **P**xy (pit at [x,y]), **W**xy (wumpus at [x,y]), **B**xy (breeze perceived at [x,y]) and **S**xy (stench perceived at [x,y]). The percept rules relate a percept to the hazards in the neighbouring squares (the squares that share a side):

Rule	Sentence	How many
R1, R2	¬P11 and ¬W11 (the start square is safe)	2
Rb(x,y)	Bxy ⇔ (P of every neighbour of [x,y]) (disjunction over the neighbours)	n^2
Rs(x,y)	Sxy ⇔ (W of every neighbour of [x,y])	n^2
Rw1	At least one wumpus: W11 ∨ W21 ∨ … ∨ Wnn	1
Rw2	At most one wumpus: ¬Wa ∨ ¬Wb for every pair of squares a ≠ b	$n^2(n^2-1)/2$

Example for n = 4: `B21 ⇔ (P11 ∨ P31 ∨ P22)` because [2,1] has the neighbours [1,1], [3,1] and [2,2]. The program `wumpus.py` generates the whole KB for any n (155 sentences for n = 4, listed by the program and reproduced in Appendix B). The **percept sequence** I designed for n ≥ 3 is: at [1,1] no breeze and no stench; at [2,1] a breeze and no stench; at [1,2] no breeze and a stench. The queries are "is there a pit at [3,1]?", "is there a pit at [2,2]?", "is there a pit at [3,3]?" and "is [2,2] safe (no pit and no wumpus)?".

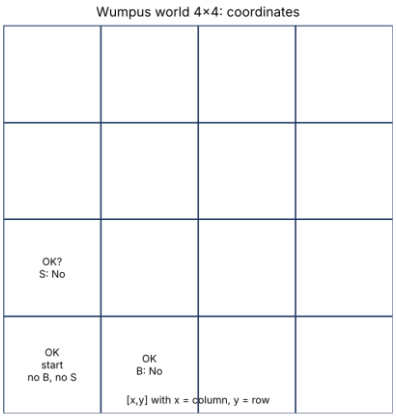


Figure 1.1 The Wumpus-world grid and the percepts used in the experiments.

Truth-table entailment checker. `tt_entails(KB, α)` enumerates every assignment to the symbols of the KB and the query. For each model it evaluates the KB; if the KB is true in the model it must make α true, otherwise the answer is "not entailed". The function is instrumented with three counters: models enumerated, sentence evaluations (each sentence of the KB tested in a model; the loop stops at the first false sentence) and elapsed time.

1.2 Student A – Design and mathematical analysis

(a) Counting symbols and sentences as a function of n

Four symbols per square and n^2 squares give **m = 4n²** symbols. The number of sentences is the sum of the rows of the table above:

$$N(n) = 2 + n^2 + n^2 + 1 + n^2(n^2-1)/2 = 3 + 2n^2 + n^2(n^2-1)/2$$

The program counts the generated sentences and compares them with the formula:

n	symbols m = 4n²	counted	sentences N(n)	counted	models 2^m
2	16	16	17	17	65,536
3	36	36	57	57	68,719,476,736
4	64	64	155	155	1.8×10^{19}
5	100	100	353	353	1.3×10^{30}
6	144	144	705	705	2.2×10^{43}

The sentence count is polynomial (dominated by the pairwise at-most-one-wumpus rules, $O(n^4)$), but the number of models is exponential in n^2.

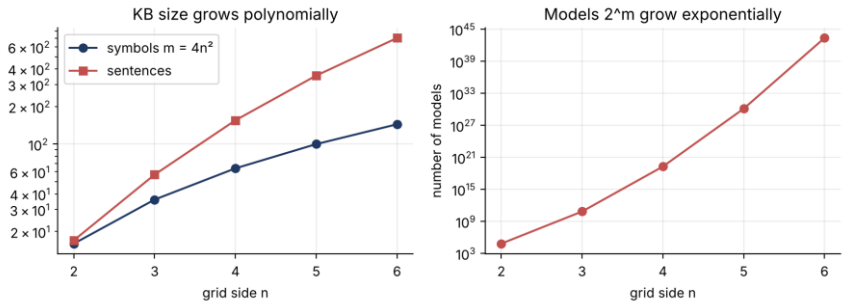


Figure 1.2 Left: symbols and sentences grow polynomially. Right: the number of models 2^m grows exponentially.

(b) The number of models and the relevant symbols

A model assigns true or false to each of the m symbols, so there are **2^m** models: 65,536 for n = 2 but 2^64 ≈ 1.8×10^{19} for n = 4. A checker that enumerates 10^6 models per second needs 5.8 × 10^5 years for n = 4.

Minimal relevant symbol set for one query. The percept sentences mention B11, S11, B21, S21, B12 and S12. Only the rules Rb and Rs of these six squares can link the percepts to the hazard symbols; they mention the pit symbols P11, P21, P12, P31, P22, P13 and the wumpus symbols W11, W21, W12, W31, W22, W13. Together with the six percept symbols that is **18 symbols**, whatever n is (for n ≥ 3). Every other square's rules only relate other symbols to each other and cannot affect whether a pit at [3,1] follows. Dropping them keeps the answer correct because entailment from a subset of the KB is sound: if the subset entails α, so does the full KB (more sentences only remove models). The restriction can lose completeness (a query could need dropped sentences), so a "not entailed" answer from the subset must be confirmed on the full KB; Section 1.3 does that with an independent solver.

(c) Soundness and completeness of model checking

- **Sound** – the checker answers "entailed" only if α is true in every model in which the KB is true. This holds because the loop really tests every model of the KB and returns "no" as soon as it finds one that violates α, so a "yes" survives all models.
- **Complete** – whenever KB ⊨ α the checker answers "yes". The set of models is finite (2^m), the loop visits all of them, and in each model of the KB α is true, so the checker never returns a false "no". It always terminates.

Model checking is therefore a *sound and complete* decision procedure for propositional entailment; its only weakness is the cost 2^m.

1.3 Student B – Experimental validation

Student B instrumented the checker and ran it for n = 2, 3, 4, with the full symbol set and with the relevant-symbol restriction. For each query the checker is called twice (KB ⊨ α and KB ⊨ ¬α); the table gives the **sum** of models enumerated and sentence evaluations over the two calls. This is how the three answer types are told apart: *entailed* (KB ⊨ α), *negation entailed* (KB ⊨ ¬α), *neither*.

n	Query (type)	Method	Symbols	Models enumerated	Sentence evals	Time (s)	Verdict
2	[1,2] has no pit (entailed)	full	16	65,634	131,688	0.064	ENTAILED
2		relevant	8	257	512	0.0002	ENTAILED
2	pit at [2,1] (negation)	full	16	65,634	131,688	0.062	NEG. ENTAILED
2		relevant	8	257	512	0.0002	NEG. ENTAILED
2	pit at [2,2] (neither)	full	16	25,028	47,591	0.022	NEITHER
2		relevant	9	18	77	0.00	NEITHER
3	pit at [3,1] (entailed)	full	36	2^36 – infeasible	–	~6.1 × 10^4	–
3		relevant	18	295,561	586,960	0.281	ENTAILED
3	pit at [2,2] (negation)	relevant	18	295,561	586,960	0.282	NEG. ENTAILED
3	pit at [3,3] (neither)	relevant	19	133,906	254,201	0.125	NEITHER
3	[2,2] safe (entailed)	relevant	18	295,561	586,960	0.279	ENTAILED
4	pit at [3,1] (entailed)	full	64	2^64 – infeasible	–	~1.6 × 10^{13}	–
4		relevant	18	295,561	586,960	0.329	ENTAILED
4	pit at [2,2] (negation)	relevant	18	295,561	586,960	0.298	NEG. ENTAILED
4	pit at [3,3] (neither)	relevant	19	133,906	254,201	0.128	NEITHER
4	[2,2] safe (entailed)	relevant	18	295,561	586,960	0.294	ENTAILED

Where the full symbol set becomes infeasible. For n = 2 (16 symbols) the full check takes 0.06 s. At n = 3 there are 36 symbols (6.9 × 10^{10} models); at the measured speed of about 0.9 µs per model that is roughly 6 × 10^4 s (17 hours), and for n = 4 roughly 10^{13} s. **n = 3 is the first infeasible size.** With the relevant-symbol restriction the work is the same for n = 3 and n = 4 (295,561 models), because only 18 symbols matter.

Cross-check of the restricted answers. A restricted KB is a subset, so I checked every verdict against the *full* KB with an independent DPLL satisfiability solver (KB ∧ ¬α unsatisfiable ⇔ KB ⊨ α). All 8 verdicts for n = 3 and n = 4 agreed (Appendix B, section C of the output).

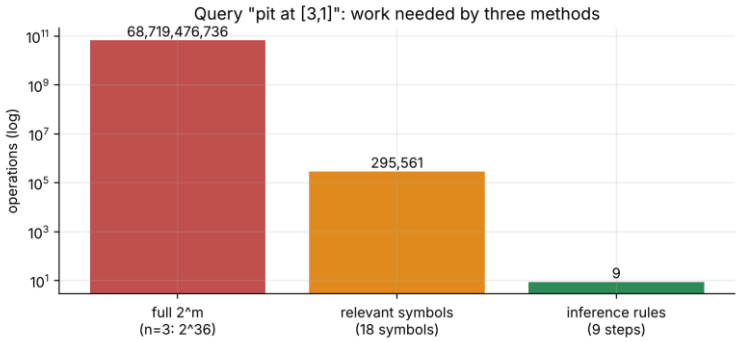


Figure 1.3 Work needed for "pit at [3,1]" (n = 3): full model checking, relevant-symbol model checking and 9 inference-rule steps (log scale).

1.4 Part B – Reasoning patterns (shared work, derivations)

Using the same KB (n = 4) and percepts, I derived the safety of squares by applying **Biconditional-Elimination**, **And-Elimination**, **Modus Ponens** and **Unit Resolution** instead of model checking. Every rule application is one step; steps that the goal does not depend on are pruned (so the number below is the length of *that* derivation).

Derivation 1: [2,2] is safe (no pit and no wumpus) – 8 steps

#	Rule	Result
1	Biconditional-Elimination on Rb12	(B12 ⇒ (P22 ∨ P11 ∨ P13)) ∧ ((P22 ∨ P11 ∨ P13) ⇒ B12)
2	And-Elimination on 1	(P22 ∨ P11 ∨ P13) ⇒ B12
3	And-Elimination (split into clauses)	{¬P22, B12}
4	Biconditional-Elimination on Rs21	(S21 ⇒ …) ∧ ((W11 ∨ W31 ∨ W22) ⇒ S21)
5	And-Elimination on 4	(W11 ∨ W31 ∨ W22) ⇒ S21
6	And-Elimination (split)	{¬W22, S21}
7	Unit Resolution: {¬P22, B12} with ¬B12 (percept)	**¬P22**
8	Unit Resolution: {¬W22, S21} with ¬S21 (percept)	**¬W22**

Both ¬P22 and ¬W22 hold, so [2,2] is safe. The line "from the percept ¬B12 and the clause {¬P22, B12} conclude ¬P22" is the whole idea: *no breeze at [1,2] means none of its neighbours has a pit*.

Derivation 2: [1,2] and [2,1] have no pit – 6 steps

Biconditional-Elimination on Rb11 (1), And-Elimination (2), splitting into {¬P21, B11} and {¬P12, B11} (3, 4), then Unit Resolution with the percept ¬B11 gives ¬P21 and ¬P12 (5, 6).

Derivation 3: there is a pit at [3,1] – 9 steps (uses Modus Ponens)

From the breeze at [2,1] the rule gives B21 ⇒ (P11 ∨ P31 ∨ P22). **Modus Ponens** with the percept B21 gives (P11 ∨ P31 ∨ P22) (step 7). Unit Resolution with ¬P11 (given), then with ¬P22 (derived from the no-breeze percept at [1,2], steps 1–3 and 6) leaves **P31**. The full list (9 steps) is printed in Appendix B.

Student A – comparison with the model-checking cost for the same query (experimental investigation)

Goal	Inference steps	Model checking (18 relevant symbols)	Ratio
[2,2] is safe	8	262,144 models, 523,661 sentence evaluations	65,458×
[1,2] and [2,1] have no pit	6	262,144 models, 523,661 sentence evaluations	87,277×
pit at [3,1]	9	262,144 models, 523,661 sentence evaluations	58,185×

Rule-based derivation is five orders of magnitude cheaper, because it follows only the sentences that matter, like a human would, instead of examining every world. The price: it needs a *search* for the right rules to apply (the program does this by a simple forward closure over the percept squares), and it is complete only if the rule set is complete; Unit Resolution plus the elimination rules alone cannot prove every entailed sentence.

1.5 Student B – Soundness reasoning

A rule is **sound** if its conclusion is true in every model in which all its premises are true. Truth tables check this exhaustively (T = true, F = false). Only the rows where *all premises* are true matter.

α	β	α ⇒ β	Premises α, α⇒β true?	Conclusion β
T	T	T	yes	T ✓
T	F	F	no	–
F	T	T	no (α false)	–
F	F	T	no (α false)	–

Modus Ponens: the only row with both premises true is α = β = T, and β is true there.

α	β	α ∧ β	Premise true?	Conclusions α / β
T	T	T	yes	T / T ✓
T	F	F	no	–
F	T	F	no	–
F	F	F	no	–

And-Elimination: α ∧ β is true only when both are true.

α	β	α ⇔ β	(α ⇒ β) ∧ (β ⇒ α)	Equal?
T	T	T	T	yes
T	F	F	F	yes
F	T	F	F	yes
F	F	T	T	yes

Biconditional-Elimination: both sides have identical columns, so the rule is sound (and also reversible).

l_1	l_2	$l_1 \vee l_2$	$\neg l_1$	Premises true?	Conclusion l_2
T	T	T	F	no	–
T	F	T	F	no	–
F	T	T	T	yes	T ✓
F	F	F	T	no	–

Unit Resolution: the only row where both premises hold is l_1 = F, l_2 = T.

Why a derived sentence must hold in every model of the KB. Each step applies a sound rule to sentences that already hold in every model of the KB (the KB sentences themselves, or previously derived sentences). By induction on the length of the derivation, every derived sentence is true in every model of the KB; so the derived sentence is *entailed*. In the Wumpus KB this is why ¬P22 can be trusted for every world consistent with the percepts.

Why "not derived" is not "false". A derivation is a *search* for a proof. If I fail to derive α with the rules at hand, either α is not entailed, *or* it is entailed but my rule set (Biconditional-Elimination, And-Elimination, Modus Ponens, Unit Resolution) is incomplete. Example in this KB: the query "pit at [3,3]" is not derived and neither is its negation – the model checker confirms that it is genuinely open (verdict NEITHER). But failing to derive a sentence tells us nothing by itself; only a *complete* procedure (model checking, or resolution refutation) can turn "no proof" into "not entailed". And "not entailed" is still not "false": it means only that α is false in *some* models and true in others.

Cross-understanding (Task 1)

Student A explains Student B's component (soundness reasoning): each inference rule is checked by a truth table; since each step preserves truth in all models of the KB, the final sentence is true in all of them. Failure to derive means only that the (incomplete) rule set found no proof.

Student B explains Student A's component (design analysis): m = $4n^2$ symbols, N(n) sentences, 2^m models; for one query only the 18 symbols of the perceived squares matter, which turns 2^36 (n = 3) into 2^18 and makes the cost independent of n.

1.6 Comparison of the two reasoning approaches

Aspect	Model checking	Inference rules
Cost for pit at [3,1]	262,144–295,561 models (18 symbols); 2^64 on the full KB	9 steps
Soundness	Sound	Sound if every rule is sound (tables above)
Completeness	Complete (finite models)	Only if the rule set is complete; "not derived" proves nothing
Needs from the designer	Nothing (but must choose symbol restriction for speed)	A search strategy and a good rule set
Explanation	None (only a yes/no)	A readable proof: premises → conclusion

Task 2 – The Quality of Reasoning Method: Model Checking versus Resolution (7 marks)

2.1 Shared group work: implementations and the smart-building policy

Two implementations were written from scratch in `logic.py`: (1) a **truth-table model checker** with counters for models, KB evaluations and time; (2) a **propositional resolution prover**: conversion to CNF (eliminate ⇔, eliminate ⇒, push ¬ inwards, distribute ∨ over ∧), then resolution refutation of KB ∧ ¬α with counters for clauses generated, resolvents attempted, tautologies discarded and subsumed clauses removed. The saturation loop ends when the empty clause is derived (entailed) or when no new clause appears (not entailed).

The access-control policy (12 rules) contains biconditionals (R1, R4, R9), implications (R3, R5–R8, R10–R12) and disjunctions (R2, R12):

Rule	Meaning	Formula
R1	A badge is valid exactly when it is not expired	BadgeValid ⇔ ¬BadgeExpired
R2	Every person is an employee, a visitor or a contractor	Employee ∨ Visitor ∨ Contractor
R3	A visitor is not an employee	Visitor ⇒ ¬Employee
R4	Work hours exactly when it is not after hours	WorkHours ⇔ ¬AfterHours
R5	Employee with a valid badge in work hours may enter	(Employee ∧ BadgeValid ∧ WorkHours) ⇒ MayEnter
R6	Escorted visitor with a valid badge in work hours may enter	(Visitor ∧ Escorted ∧ BadgeValid ∧ WorkHours) ⇒ MayEnter
R7	After hours, no emergency, not security staff: may not enter	(AfterHours ∧ ¬Emergency ∧ ¬Security) ⇒ ¬MayEnter
R8	Emergency override: everybody may enter	Emergency ⇒ MayEnter
R9	The door is unlocked exactly when entry is allowed	MayEnter ⇔ DoorUnlocked
R10	An emergency activates the alarm	Emergency ⇒ Alarm
R11	Server-room access needs permission and employee status	ServerRoom ⇒ (MayEnter ∧ Employee)
R12	A contractor needs an escort or may not enter	Contractor ⇒ (Escorted ∨ ¬MayEnter)

The KB has 14 symbols (2^14 = 16,384 models). Six designed cases were run with both methods (Table in 2.3); in every case the two methods agree. Case C2 (an unescorted visitor in work hours) is *not entailed in either direction* – the policy leaves it open, which is a policy gap discovered by the checker. Case C5 shows that the facts ServerRoom ∧ Visitor contradict R3 and R11, so the KB is inconsistent and entails everything.

2.2 Student A – Resolution investigation

(a) Manual CNF conversion of three sentences (checked against the program)

Step	R1: BadgeValid ⇔ ¬BadgeExpired	R5: (Employee ∧ BadgeValid ∧ WorkHours) ⇒ MayEnter	R7: (AfterHours ∧ ¬Emergency ∧ ¬Security) ⇒ ¬MayEnter
1 Eliminate ⇔	(V ⇒ ¬E) ∧ (¬E ⇒ V)	unchanged	unchanged
2 Eliminate ⇒ (α⇒β ≡ ¬α ∨ β)	(¬V ∨ ¬E) ∧ (¬¬E ∨ V)	¬(Emp ∧ V ∧ W) ∨ M	¬(A ∧ ¬Em ∧ ¬S) ∨ ¬M
3 Push ¬ inwards (De Morgan, ¬¬)	(¬V ∨ ¬E) ∧ (E ∨ V)	¬Emp ∨ ¬V ∨ ¬W ∨ M	¬A ∨ Em ∨ S ∨ ¬M
4 Distribute ∨ over ∧ (clauses)	{¬V, ¬E} and {E, V}	{¬V, ¬Emp, ¬W, M}	{¬A, ¬M, Em, S}

(V = BadgeValid, E = BadgeExpired, Emp = Employee, W = WorkHours, M = MayEnter, A = AfterHours, Em = Emergency, S = Security.) R1 gives two clauses, R5 and R7 one each.

(b) Resolution proof of an entailed query (case C1)

Query: an employee with a non-expired badge during work hours finds the door unlocked. Premises: Employee, ¬BadgeExpired, WorkHours. Resolution refutation of KB ∧ ¬DoorUnlocked:

#	Clause	Justification
1	{BadgeExpired, BadgeValid}	R1 (CNF)
2	{¬BadgeExpired}	fact
3	{BadgeValid}	resolve 1, 2 on BadgeExpired
4	{¬BadgeValid, ¬Employee, ¬WorkHours, MayEnter}	R5 (CNF)
5	{Employee}	fact
6	{¬BadgeValid, ¬WorkHours, MayEnter}	resolve 4, 5
7	{¬WorkHours, MayEnter}	resolve 3, 6 on BadgeValid
8	{¬MayEnter, DoorUnlocked}	R9 (CNF)
9	{¬DoorUnlocked}	negated query
10	{¬MayEnter}	resolve 8, 9
11	{¬WorkHours}	resolve 7, 10 on MayEnter
12	{WorkHours}	fact
13	{ } empty clause	resolve 11, 12 – contradiction, so KB ⊨ DoorUnlocked

Proof tree (read bottom-up): the empty clause (13) comes from {¬WorkHours} (11) and the fact {WorkHours} (12); clause 11 comes from {¬WorkHours, MayEnter} (7) and {¬MayEnter} (10); clause 7 comes from {BadgeValid} (3) and (6); clause 10 comes from R9 (8) and the negated query (9); clauses 3 and 6 come from R1, R5 and the facts. The prover found it after generating **94 clauses** (56 tautologies discarded, 196 subsumed) from 534 attempted resolvents; the proof itself has 13 lines.

(c) Growth with the KB: implication chains of length 10, 20, 40, 80

KB: A0, A0 ⇒ A1, …, A(n−1) ⇒ An; query An. Counters from `logic.pl_resolution`:

n	Clauses in KB	Generated	Attempted	Tautologies	Subsumed	Rounds	Time (s)
10	11	57	149	0	32	4	0.000
20	21	214	952	0	118	5	0.002
40	41	814	6,780	0	450	6	0.009
80	81	3,166	51,204	0	1,754	7	0.059

Generated clauses grow roughly quadratically with n (×3.8 per doubling), attempted resolvents grow faster (×6 to ×8 per doubling, roughly n^3) because every new clause is paired with all existing clauses. Subsumption removes about a fifth of the candidates: at n = 40 it cut the resolvents attempted from 8,537 to 6,780 (clauses generated 816 → 814); its effect on time was negligible at this size (0.008 s in both cases). Tautology deletion removed nothing in the chain, which has no clause containing both a literal and its negation.

(d) A query that is not entailed, and why the process terminates

The same chain **without** the fact A0, query An. Resolution saturates after generating 55 / 210 / 820 / 3,240 clauses for n = 10 / 20 / 40 / 80 and stops (all `terminates = True`). Termination argument: all clauses are built from the n + 1 symbols of the KB; a clause is a set of literals, so there are at most 3^{n+1} different clauses and every clause is generated at most once (duplicates are discarded). Each round either adds a new clause or the loop ends; hence the process ends. When it ends without the empty clause, the saturated set is

satisfiable, so the query is not entailed. *Termination is guaranteed for propositional resolution, but the bound 3^{m} is itself exponential.*

2.3 Student B – Model-checking comparative analysis

Case (12-rule policy, 14 symbols)	Models enumerated	Resolution: generated	attempted	tautologies	subsumed	MC / RES verdict
C1 employee ⊨ DoorUnlocked	16,384	94	534	56	196	True / True
C2 visitor unescorted ⊨ MayEnter?	1,028	20	188	30	70	False / False (open)
C3 after hours ⊨ ¬DoorUnlocked	16,384	77	476	129	156	True / True
C4 emergency ⊨ DoorUnlocked ∧ Alarm	16,384	64	704	215	207	True / True
C5 visitor in server room ⊨ false	16,384	26	129	44	36	True / True
C6 contractor unescorted ⊨ ¬MayEnter	16,384	41	301	61	106	True / True

On the small 14-symbol policy KB model checking costs 16,384 models (a few milliseconds) while resolution generates 20–94 clauses. Both are instantaneous; the question is how they behave when the KB changes.

Family 1 – where model checking is hopeless: Horn chains with many symbols

Symbols m	Model checking models	MC time (s)	Resolution clauses	Resolution time (s)
9	512	0.000	43	0.0002
13	8,192	0.006	76	0.0003
17	131,072	0.129	151	0.0009
19	524,288	0.490	181	0.0011
21 (n = 20 chain)	2,097,152	2.12	214	0.002
81 (n = 80 chain)	2^81 = 2.4×10^{24}	≈ 7×10^{10} years at the measured 0.93 µs per model	3,166	0.059

Cause: the chain is a Horn KB with a linear implication structure. Each resolution step is guided by the single complementary pair, so the number of clauses grows polynomially; model checking pays 2^m regardless of structure.

Family 2 – where model checking wins: the pigeonhole principle

PHP(k+1, k): k + 1 pigeons, k holes, “every pigeon sits in a hole”, “no two pigeons share a hole”: a small, dense, unsatisfiable KB. The query is the contradiction Z ∧ ¬Z, so the KB entails everything and each method must prove unsatisfiability.

k	Symbols m	Clauses	MC models	MC time (s)	Resolution generated	attempted	time (s)	Result
2	7	9	128	0.000	64	540	0.00	proved
3	13	22	8,192	0.014	5,738	7,479,819	13.24	proved
4	21	45	2,097,152	5.53	62,578	955,397	60.03	LIMIT (60 s)
5	31	81	2^31 infeasible	–	139,067	651,742	60.00	LIMIT (60 s)

For k = 3 model checking needs 8,192 models in 0.014 s; resolution needs 7.5 million resolvent attempts and 13 s. At k = 4 model checking still finishes (5.5 s) while resolution gives up at the 60 s cap. **Cause:** pigeonhole formulas have no short resolution proofs (Haken’s theorem: every resolution refutation needs

exponentially many clauses), while the symbol count stays small, so 2^m is cheap. At k = 5 (31 symbols) model checking also becomes infeasible: neither method works.

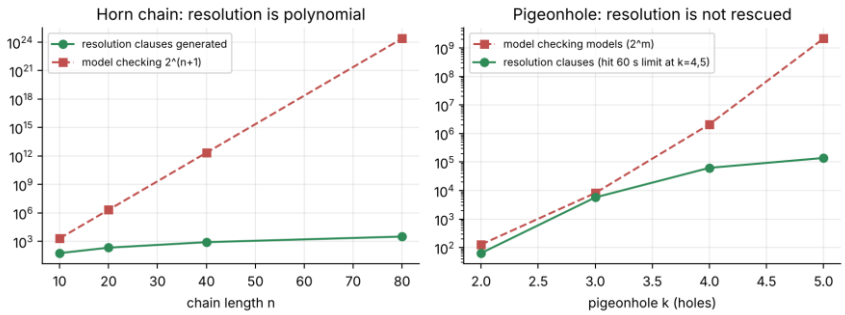


Figure 2.1 Left: resolution is polynomial on implication chains while model checking is exponential. Right: on pigeonhole formulas the order reverses at small k.

Structural property that decides the winner

Model checking cost = 2^(number of symbols) × (KB size) – independent of how the sentences are structured.

Resolution cost depends on the *interaction structure of the clauses*: unit propagation and Horn structure keep it polynomial; densely coupled clauses with no short refutation (pigeonhole) make it exponential even when there are few symbols.

Cross-understanding (Task 2)

Student B explains Student A's component: the not-entailed process terminates because the number of distinct clauses over the KB's symbols is finite and each is generated once; the saturated clause set without the empty clause is satisfiable.

Student A explains Student B's component: model checking is cheap exactly when m is small (the pigeonhole cases) and hopeless when m is large even if the KB is trivial (chains).

2.4 Joint design question

If the KB size and query types are not known in advance, what do I need to know before choosing a method? My measurements say the decision depends on four facts:

1. **The number of distinct symbols m.** If m ≤ about 20 (here: 2^20 ≈ 1 s) model checking is a safe choice; beyond about 25 symbols it is not runnable (the n = 80 chain would take 7×10^{10} years).
2. **The clause structure.** If most clauses are Horn (at most one positive literal), use chaining or resolution with unit preference: the chains needed only 3,166 clauses for 81 symbols. If the KB is dense and highly coupled with many negative interactions (pigeonhole-style cardinality constraints), neither method scales and a SAT solver with learning (DPLL/CDCL) is the right tool; my DPLL cross-check in Task 1 solved the 64-symbol Wumpus KB instantly.
3. **Whether the query is entailed.** Resolution on a *non*-entailed query must saturate the whole clause set (3,240 clauses at n = 80), while entailed queries often finish earlier; model checking costs the same either way.
4. **How many queries share one KB.** Resolution results (the saturated clause set) can be reused; model checking cannot. Also, resolution gives a proof that can be shown to a user; model checking gives only yes or no.

Task 3 – Knowledge Engineering in First-Order Logic: University Exam-Eligibility Advisor (7 marks)

3.1 Shared group work: the knowledge-engineering process

Step	What was done for the exam-eligibility advisor
1 Identify the task	Decide whether student s may write the examination of course c, and explain the decision. Query types: yes/no ("may alice write CS201?") and bindings ("who may write CS201?").
2 Assemble the knowledge	Regulations: (i) attendance at least the course minimum (75 %) or an approved leave; (ii) every prerequisite, direct or indirect, passed (grade ≥ 40); (iii) no fees due; (iv) not barred by an **active** disciplinary case; (v) the student must be enrolled.
3 Decide the vocabulary	Predicates: Student, Course, Enrolled(s,c), Attendance(s,c,percent), MinAtt(c,percent), ApprovedLeave(s,c), Grade(s,c,mark), Requires(c,p), FeeDue(s), DisciplinaryCase(s,k), Active(k); derived: AttendanceOK, Passed, Needs, MissingPrereq, PrereqsCleared, FeesCleared, Barred, Eligible. Constants: students (alice…ida), courses (CS101, CS201, CS301, MA101), cases (k1, k2), numbers. No function symbols. Built-in comparison Ge(a,b) for a ≥ b.
4 Encode general knowledge	The 14 axioms below; 10 of them are executed as definite clauses with negation as failure.
5 Encode the instance	9 students, 4 courses, a prerequisite chain CS101 → CS201 → CS301, 45 facts.
6 Pose queries	The 15-query competency suite of Section 3.5, with expected answers written from the regulations **before** running.
7 Debug the KB	Three genuine defects were found by failing queries and fixed (Section 3.5).

3.2 Student A – Design derivation

(a) Vocabulary and the 14 axioms (quantifiers)

Axioms marked * are executed as definite clauses (with negation as failure for ¬); the others complete the first-order theory but are not Horn clauses and are therefore not executed by the chaining engine. Variables s, c, p, m, g, k, q are universally quantified unless an existential quantifier is shown.

```
1*  Ax,c,p,m  Enrolled(s,c) & Attendance(s,c,p) & MinAtt(c,m) & Ge(p,m)  =>  AttendanceOK(s,c)
2*  Ax,c      Enrolled(s,c) & ApprovedLeave(s,c)  =>  AttendanceOK(s,c)
3*  Ax,c,g    Grade(s,c,g) & Ge(g,40)  =>  Passed(s,c)
4*  Ac,p      Requires(c,p)  =>  Needs(c,p)
5*  Ac,p,q    Requires(c,q) & Needs(q,p)  =>  Needs(c,p)              (transitive prerequisites)
6*  As,c      Enrolled(s,c) & Ep (Needs(c,p) & ~Passed(s,p))  =>  MissingPrereq(s,c)
7*  As,c      Enrolled(s,c) & ~MissingPrereq(s,c)  =>  PrereqsCleared(s,c)
8*  As        Student(s) & ~FeeDue(s)  =>  FeesCleared(s)
9*  As        Ek (DisciplinaryCase(s,k) & Active(k))  =>  Barred(s)
10  As        Barred(s)  =>  Ek (DisciplinaryCase(s,k) & Active(k))     [EXISTENTIAL: the "only if" half of 9]
11* As,c      AttendanceOK(s,c) & PrereqsCleared(s,c) & FeesCleared(s) & ~Barred(s)  =>  Eligible(s,c)
12  As,c      Eligible(s,c)  =>  Enrolled(s,c)                          [sanity axiom]
13  As,c      Passed(s,c)  =>  Ee Grade(s,c,e)                          [EXISTENTIAL: a pass has a recorded
grade]
14  Ac        Course(c)  =>  Ef Coordinator(c,f)                        [EXISTENTIAL: every course has a
coordinator]
```

The existential quantifier in 6 and 9 sits in the *body* of a Horn rule, where it becomes a universally quantified body variable ($\exists p\ (A \wedge B) \Rightarrow H \equiv \forall p\ (A \wedge B \Rightarrow H)$), so these rules run directly. Axioms 10, 13 and 14 have existentials in their *heads*; they would require Skolem functions (Task 5) and are kept as part of the specification. Negation as failure (¬ = "cannot be proved") assumes the **closed world**: a student with no FeeDue fact has no fees due. This is appropriate for a registry database and is one of the limits listed in 3.7.

(b) Growth of the propositional encoding and the constancy of the FOL axioms

In the propositional encoding every sentence mentions concrete individuals, so a separate copy of each rule is needed for each student and course:

Group of rules	Copies	Count
FeesCleared_s ⇐ Student_s, ¬FeeDue_s; Barred_s ⇐ Case_s, ActiveCase_s	per student	2 s
$\text{AttendanceOK}_{s,c}$ ⇐ Enrolled, AttGEmin and ⇐ Enrolled, Leave	per student and course	2 s c
$\text{MissingPrereq}_{s,c}$ ⇐ Enrolled, $\neg\text{Passed}_{s,p}$ for every earlier course p (a chain: course j has j prerequisites)	per student, course j	s · c(c−1)/2
$\text{PrereqsCleared}_{s,c}$; $\text{Eligible}_{s,c}$	per student and course	2 s c

$$N_prop(s, c) = s \cdot [\, 2 + 4c + c(c-1)/2 \,] \rightarrow \textit{linear in s, quadratic in c}$$

The first-order encoding has the same **10 executable rules (14 axioms)** for every s and c; only the *facts* (the data that any registry holds anyway) grow, linearly in s·c. The program built both encodings for random registries and counted the sentences; the rule counts agree with the formula in every row:

s	c	Propositional rules	Formula N_prop	FOL axioms	FOL facts	Propositional sentences (rules + facts)	FOL sentences
10	5	320	320	10	108	400	118
100	5	3,200	3,200	10	941	3,996	951
1,000	5	32,000	32,000	10	9,662	40,056	9,672
10	20	2,720	2,720	10	430	3,027	440
100	20	27,200	27,200	10	3,802	30,282	3,812
1,000	20	272,000	272,000	10	37,131	302,387	37,141
3,000	20	816,000	816,000	10	111,967	907,450	111,977

Maintenance consequence. If the minimum attendance of one course changes, the first-order KB changes **one fact** (MinAtt); the propositional KB must recompute the proposition AttGEmin for every student enrolled in that course (up to s facts) and, if the threshold were written into the rules, up to s·c sentences.

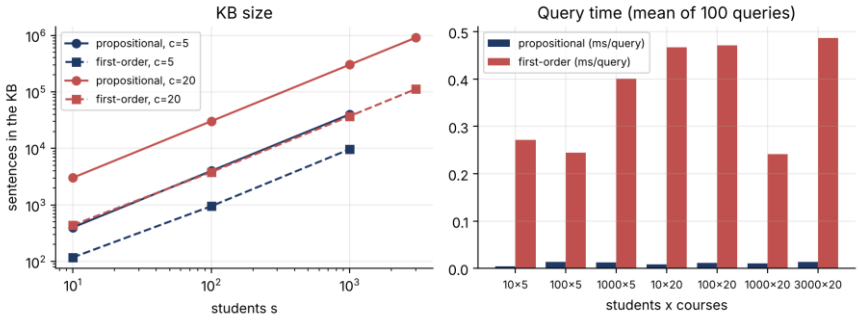


Figure 3.1 Left: sentences in the two encodings (log–log). Right: mean time of one eligibility query for both encodings.

3.3 Student B – Crossover investigation

(a) The competency queries (15) – expected answers written before the first run

Instance: alice (grade 70 in CS101, attendance 82 % in CS201), bob (attendance 60 %), carol (attendance 60 % but approved leave), dave (failed CS101 and CS201, enrolled in CS301), erin (fees due), frank (active disciplinary case), gina (closed case), hari (CS101), ida (not enrolled). Expected results come from reading the regulations, not from the program.

Query	Expected	Draft 1	Final
S1 Eligible(alice, CS201)	true	pass	pass
S2 Eligible(carol, CS201) – approved leave	true	pass	pass
S3 Eligible(hari, CS101)	true	pass	pass
S4 Eligible(gina, MA101) – closed case	true	**FAIL (false)**	pass
S5 Passed(alice, CS101)	true	pass	pass
F1 Eligible(bob, CS201) – attendance	false	pass	pass
F2 Eligible(dave, CS301) – indirect prerequisite	false	**FAIL (true)**	pass
F3 Eligible(erin, MA101) – fees due	false	pass	pass
F4 Eligible(frank, MA101) – active case	false	pass	pass
F5 Eligible(ida, CS101) – not enrolled	false	pass	pass
B1 Eligible(?s, CS201)	{alice, carol}	pass	pass
B2 Needs(CS301, ?p)	{CS201, CS101}	**FAIL (none)**	pass
B3 Barred(?s)	{frank}	**FAIL ({frank, gina})**	pass
B4 FeesCleared(?s)	all but erin	**FAIL (error)**	pass
B5 Eligible(?s, MA101)	{gina}	**FAIL (none)**	pass

Draft 1 passed 9 of 15 queries; after the three fixes below the final KB passes **15 of 15** (the table is produced by `t3_run.py`; Appendix B lists goals expanded per query).

(b) Defect log – three genuine defects found and fixed

#	Symptom (failing query)	Diagnosis	Fix
D1	S4 fails (gina is wrongly refused) and B3 returns gina as well as frank	The rule Barred(s) ⇐ DisciplinaryCase(s,k) ignored whether the case is still open: a *closed* case barred the student.	Add Active(k) to the body (axiom 9).
D2	F2 succeeds although dave failed CS101; B2 has no answer	Only *direct* prerequisites were checked; CS301 requires CS201 which requires CS101, and no rule computed the transitive closure.	New predicate Needs with a base rule and a recursive rule (axioms 4, 5); MissingPrereq uses Needs.
D3	B4 raises a floundering error	FeesCleared(s) ⇐ ¬FeeDue(s), Student(s) evaluated the negative literal while s was still unbound; negation as failure is only meaningful on ground literals.	Reorder the body: Student(s) first so that s is bound (safe negation).

Defect D3 is a characteristic danger of negation as failure: a **logically equivalent** body in a different order behaves differently in the engine.

(c) An explanation (proof) for Eligible(alice, CS201) – 13 nodes

```
rule Eligible(?s,?c) :- AttendanceOK(?s,?c), PrereqsCleared(?s,?c), FeesCleared(?s), not Barred(?s)
  rule AttendanceOK(?s,?c) :- Enrolled(?s,?c), Attendance(?s,?c,?p), MinAtt(?c,?m), Ge(?p,?m)
    fact Enrolled(alice, CS201)   fact Attendance(alice, CS201, 82)   fact MinAtt(CS201, 75)   fact Ge(82, 75)
  rule PrereqsCleared(?s,?c) :- Enrolled(?s,?c), not MissingPrereq(?s,?c)
    fact Enrolled(alice, CS201)   not MissingPrereq(alice, CS201)  (no proof found: negation as failure)
  rule FeesCleared(?s) :- Student(?s), not FeeDue(?s)
    fact Student(alice)           not FeeDue(alice)  (no proof found)
  not Barred(alice)  (no proof found)
```

(d) Measurements: KB size and inference time as s and c grow

For each (s, c) a random registry was generated (seed fixed), both encodings were built, and **100 random Eligible(S_i, C_j) queries** were answered by each. The propositional engine is an indexed ground backward chainer with a memo table (the favourable case for propositional logic: arithmetic comparisons and the prerequisite closure were precomputed). Both gave identical answers in all 700 queries. The build time is the best of three builds.

s × c	Prop. build (s)	Prop. memory (MB)	Prop. goals (100 q)	FOL build (s)	FOL memory (MB)	FOL goals	FOL unifications	Answers agree	Break-even queries
10 × 5	0.000	0.1	579	0.000	0.0	962	1,341	yes	1
100 × 5	0.002	1.1	679	0.000	0.1	1,174	1,609	yes	6
1,000 × 5	0.066	11.6	692	0.008	1.3	1,147	1,604	yes	151
10 × 20	0.003	0.8	587	0.000	0.1	1,225	4,586	yes	5
100 × 20	0.041	7.3	620	0.001	0.3	1,191	4,666	yes	87
1,000 × 20	0.496	74.6	697	0.032	3.5	1,149	4,184	yes	2,005
3,000 × 20	1.685	224.6	578	0.091	9.3	963	3,835	yes	3,376

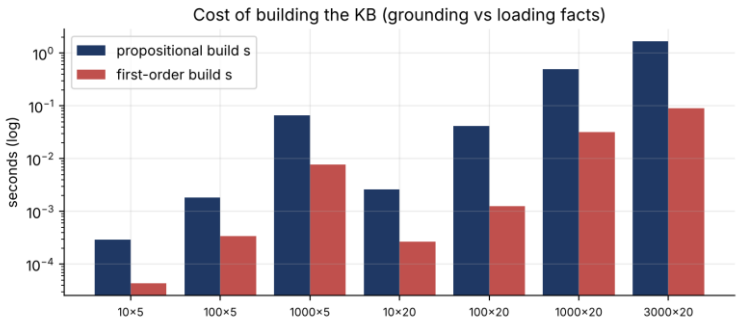


Figure 3.2 Cost of building the two knowledge bases (log scale): grounding every rule per student and course versus loading facts.

Reading the table. (1) *Size and memory*: the propositional KB grows 8× faster; at 3,000 × 20 it needs 225 MB against 9 MB. (2) *Per-query work*: both are flat in s and c: the number of goals does not grow because each query touches one student and one course. The first-order engine expands 1.7–2.1 times more goals per batch of 100 queries (and, with the course chain, several times more unifications) because every goal is unified with rule heads and renamed (0.07–0.5 ms per query against 0.005–0.014 ms, a 10–50× slower query in my measurements, which carry about ±50 % timing noise). (3) *Total cost* = build + queries. The first-order encoding is cheaper in total as long as the number of queries stays below the **break-even**, (build_prop − build_FOL)/(query_FOL − query_prop). This break-even grows with the size of the registry: 1 query for 10 × 5, about 6 for 100 × 5 or 10 × 20, about 90–150 for 100 × 20 and 1,000 × 5, and 2,000–3,400 for registries of 1,000 or more students and 20 courses.

Where does the first-order encoding become preferable?

Always for the size of the KB, its memory and its maintainability (one fact per course instead of one recomputed proposition per student-course).

In total time when the registry has at least about 100 students × 20 courses (or 1,000 × 5) *and* the KB is queried fewer than about 100 to 3,000 times between changes – a typical exam-eligibility scenario in which the rules or the data change every term, and the number of queries per term is bounded by the number of students × courses.

A propositional encoding (or a pre-computed table of answers) is better for a **small, static KB asked very many questions**, because it avoids the unification overhead.

3.4 Joint conclusion: why expressiveness does not automatically give performance

First-order logic lets us state each regulation once, quantified over all students and courses; that is why the axioms stayed at 10 rules when the propositional KB grew to 816,000. But expressiveness is paid for at inference time: every step needs **unification** (matching with substitution, renaming rule variables, an

index lookup), the engine must handle quantifier structure, negation as failure, floundering and recursion (D3, D2), and each query is 10–50× slower than a lookup in a ground table. For a small or static KB the propositional encoding wins in total time (break-even of 1–6 queries at 10 × 5), and for a registry of thousands of students the first-order encoding wins until the number of queries exceeds a few thousand. So the right question is not “which logic is more expressive” but “how large is the domain, how often does the KB change, and how many queries are asked?”. A third consideration is **correctness**: expressive rules hide defects (a missing Active condition, a missing transitive closure) which the 15-query suite had to find.

3.5 Limits of the encoding

Negation as failure and the closed-world assumption mean that a student whose fee record is missing is treated as having no fees due; a real registry would also need an *unknown* status. The existential axioms 10, 13, 14 are not executed (they would need Skolem functions and resolution). Attendance is compared numerically by a built-in rather than defined logically.

> **Cross-understanding (Task 3)**
>
> **Student B explains Student A’s derivation:** a propositional rule mentions fixed individuals, so each of the s students and c courses needs its own copy: $s \cdot (2 + 4c + c(c-1)/2)$ rules (the c^2 term is the prerequisite chain). The FOL axioms are quantified and do not mention individuals, so they stay at 10.
>
> **Student A explains Student B’s defect log:** D1 is a missing condition (open vs closed case), D2 a missing recursion (transitive prerequisites), D3 an unsafe order of literals under negation as failure; each was exposed by a query whose expected answer was written first.

Task 4 – The Reality of Unification and Lifted Inference (7 marks)

In this task the responsibilities are reversed compared with Task 3: **Student B** is responsible for design and correctness, **Student A** for experiments and engineering analysis.

4.1 Shared group work: implementations

- `unify(x, y, θ, occurs_check)` – unification of variables (names starting with ?), constants and compound terms $f(t_1,\ldots,t_n)$, with substitution walking, an optional **occurs check** and a counter of calls and bindings. A substitution that makes a variable refer to a term containing itself is reported as `CYCLIC`.
- `compose(s1, s2)` – composition of substitutions so that apply(apply(t, s1), s2) = apply(t, compose(s1, s2)).
- **Lifted Modus Ponens** – to apply $p_1 \wedge \ldots \wedge p_n \Rightarrow q$, unify the premises one by one with known facts, threading one substitution θ, and return SUBST(θ, q).
- `propositionalise(rules, terms)` – instantiate every universal rule with every tuple of ground terms.

4.2 Student B – Logical design and correctness

(a) Most general unifiers derived by hand (five designed pairs, plus a constant clash)

Algorithm: scan the two terms left to right; unify the first pair of arguments; apply the substitution found so far to the rest before continuing; a variable meets a term → bind it (after the occurs check); two different constants or function symbols → failure. The program's result is printed beside each hand derivation.

Pair	Hand derivation	Result (program)
P1 Knows(John, x) and Knows(y, Mother(y))	John vs y → {y/John}. x vs Mother(y): apply θ → x vs Mother(John) → {x/Mother(John)}.	{x/Mother(John), y/John}
P2 Likes(x, x) and Likes(Anna, Bob)	x vs Anna → {x/Anna}. Second argument: x → Anna vs Bob: two different constants, clash.	FAILURE
P3 P(x, f(x)) and P(y, y) (occurs check matters)	x vs y → {x/y}. f(x) → f(y) vs y: y occurs inside f(y); binding y/f(y) would make an infinite term.	FAILURE with the occurs check; without it a CYCLIC binding x = f(f(f(…)))
P4 Knows(John, x) and Knows(x, Elizabeth) (standardising apart is essential)	Both sentences use the name x. John vs x → {x/John}; then x → John vs Elizabeth: clash. But the two x's are different variables! Rename the second to x_1: John vs x_1 → {x_1/John}; x vs Elizabeth → {x/Elizabeth}.	FAILURE with shared name; **{x/Elizabeth, x_1/John}** after renaming (common instance Knows(John, Elizabeth))
P5 R(x, g(y, z), z) and R(h(w), v, k(w))	x vs h(w) → {x/h(w)}. g(y,z) vs v → {v/g(y,z)}. z vs k(w) → {z/k(w)}. Apply z/k(w) to earlier bindings: v/g(y, k(w)).	{x/h(w), v/g(y, k(w)), z/k(w)}
P6 Q(a, x) and Q(b, x)	a vs b: different constants.	FAILURE

Composition check: for σ_1 = {x/f(y), z/w} and σ_2 = {y/a, w/b}, compose(σ_1, σ_2) = {x/f(a), y/a, z/b, w/b}; the program verified apply(apply(t, σ_1), σ_2) = apply(t, composition) on test terms (True).

(b) Why lifted inference is sound, and why it avoids generating ground instances

Soundness. Lifted Modus Ponens uses two sound steps: (1) *Universal Instantiation* – if ∀x ϕ(x) holds then ϕ(t) holds for every term t, in particular for the instance obtained with the unifier θ: SUBST(θ, $p_1 \wedge \ldots \wedge p_n \Rightarrow q$); (2) ordinary Modus Ponens with the facts SUBST(θ, p_i). Since θ makes every premise *identical* to a known fact, the conclusion SUBST(θ, q) follows in every model. The *most general* unifier yields the weakest

instantiation, so the conclusion is the most general conclusion – any other sound conclusion is an instance of it. The occurs check is needed for soundness: without it P(x, f(x)) would unify with P(y, y), an instance that no model satisfies.

Why it avoids ground instances. Propositionalisation instantiates a rule with *every* tuple of ground terms (k^v instances) and then looks for matching facts; lifted inference instantiates only the variables that unification binds, by matching premises against the facts that exist. Example: Parent(x,y) ∧ Parent(y,z) ⇒ Grandparent(x,z) with facts Parent(Ann,Bob) and Parent(Bob,Cy): unify Parent(x,y) with the first fact → {x/Ann, y/Bob}; unify Parent(y,z) → Parent(Bob,z) with the second → {z/Cy}; conclusion Grandparent(Ann, Cy). Exactly one instance of the rule was ever considered; the other $k^3 - 1$ were never generated.

4.3 Student A – Experimental and engineering analysis

(a) Ground clauses as the number of constants and the predicate arity grow

A rule with r different variables has k^r ground instances over k constants. The program generated and counted them for k ≤ 20 and used the formula at k = 30:

k	3 variables (R1)	counted	2 variables	r = 1	r = 2	r = 3	r = 4	r = 5
3	27	27	9	3	9	27	81	243
5	125	125	25	5	25	125	625	3,125
10	1,000	1,000	100	10	100	1,000	10,000	100,000
20	8,000	8,000	400	20	400	8,000	160,000	3,200,000
30	27,000	(formula)	900	30	900	27,000	810,000	24,300,000

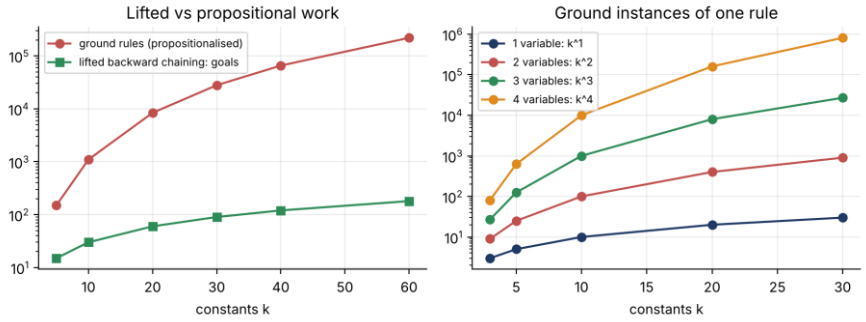


Figure 4.1 Left: ground rules produced by propositionalisation versus goals of lifted backward chaining. Right: ground instances of a rule with 1 to 4 variables.

(b) A function symbol: ground terms grow with the nesting-depth bound

With k = 2 constants and one unary function f, the ground terms of depth ≤ d are k(d + 1) = 2, 4, 6, 8, 10 – linear. With the unary f and a binary g the recurrence is $N_d = k + N_{d-1} + N_{d-1}^2$; the program counts agree with it:

Depth d	only f	formula k(d+1)	f and g: counted	recurrence
0	2	2	2	2
1	4	4	8	8
2	6	6	74	74
3	8	8	5,552	5,552
4	10	10	–	30,830,258

With even a single function symbol the Herbrand universe is infinite, so propositionalisation can be done only up to a depth bound; with a binary function symbol the number of ground terms grows *doubly exponentially* in the depth (the square in the recurrence).

(c) Lifted versus propositional inference on the same query

KB: Edge(c_0,c_1), …, Edge(c_{k-2}, c_{k-1}) and the rules Path(x,y) ⇐ Edge(x,y) and Path(x,z) ⇐ Edge(x,y) ∧ Path(y,z). Query Path(c_0, ?z). *Propositional*: instantiate the rules with all k constants and forward-chain. *Lifted*: backward chaining with unification.

k	Ground rules	Propositional FC firings	Prop. time (s)	Lifted BC goals	Unifications	Lifted time (s)	Answers
5	150	600	0.0005	15	18	0.0002	4 (both)
10	1,100	9,900	0.005	30	38	0.0003	9
20	8,400	159,600	0.054	60	78	0.0006	19
30	27,900	809,100	0.238	90	118	0.001	29
40	65,600	2,558,400	0.680	120	158	0.0015	39
60	219,600	12,956,400	3.39	180	238	0.0026	59

Lifted backward chaining does O(k) work (goals = 3k), propositional forward chaining O(k^3) (the 3-variable rule creates k^3 ground rules). At k = 60: 180 goals versus 12.96 million firings, and 0.0026 s versus 3.39 s, with identical answers. For a query about one constant (Path(c58, ?z), k = 60) lifted chaining expanded only **6 goals**; propositionalisation still created 219,600 ground rules.

4.4 Joint engineering decision: when to propositionalise

Propositionalise despite the availability of lifted inference **only when all** of the following hold:

1. **The domain is finite and small** – few constants (k ≤ ~5–10 in my table: 150 to 1,100 ground rules, solved in under 6 ms) and rules with at most 2–3 variables, so that k^r stays below roughly 10^4.
2. **There are no function symbols** (or the term depth is bounded). With f and g the term count after depth 3 is already 5,552 and after depth 4, 30 million.
3. **We want propositional tools**: a SAT solver (DPLL in Task 1 solved a 64-symbol KB instantly) or a model checker, or the same ground KB will be queried very many times, so that the one-off grounding cost is amortised.
4. **Most of the KB is relevant to the queries.** If a query concerns one constant (6 goals in my test) lifting is better because grounding generates 219,600 irrelevant rules.

Otherwise – a large or unbounded domain, higher-arity relations, function symbols, rarely asked queries – lifted inference with unification is the only practical choice.

> **Cross-understanding (Task 4)**
>
> **Student A explains Student B's component:** the occurs check is needed because P(x,f(x)) and P(y,y) would unify with the circular term x = f(f(…)); standardising apart is needed because two sentences using the same variable name would otherwise be forced to share a binding (P4).
>
> **Student B explains Student A's measurements:** ground instances grow as k^r (27,000 at k = 30 for r = 3); a function symbol makes the term universe infinite, doubly exponential in depth; lifted inference cost depends on the relevant facts and not on k^r.

Task 5 – Upgrading Resolution from Propositional to First-Order Logic (6 marks)

5.1 Shared group work: statements, FOL and CNF

The four English statements were translated into first-order logic with the predicates Emp(x), Doc(d), ContainsPD(d) (contains patient data), Restricted(d), Audited(d), Handled(x,d), Auth(x), and the constant R1:

#	English	First-order logic
S1	Every employee who handled a restricted document is authorised.	∀x (Emp(x) ∧ ∃d (Doc(d) ∧ Restricted(d) ∧ Handled(x,d)) ⇒ Auth(x))
S2	Every document containing patient data is restricted.	∀d (Doc(d) ∧ ContainsPD(d) ⇒ Restricted(d))
S3	Some employee handled every document that was audited.	∃e (Emp(e) ∧ ∀d (Doc(d) ∧ Audited(d) ⇒ Handled(e,d)))
S4	Report R1 contains patient data and R1 was audited.	Doc(R1) ∧ ContainsPD(R1) ∧ Audited(R1)
Query	An authorised employee exists.	∃x (Emp(x) ∧ Auth(x)) (negated for refutation: ∀x ¬Emp(x) ∨ ¬Auth(x))

Note on S1: "handled a restricted document" puts ∃d in the *antecedent*, which by quantifier duality becomes ∀d in the clause. Note on S3: the existential is outermost, so there is no universal variable in whose scope it lies.

5.2 Student A – Conversion and proof design

(a) Every conversion step (program output, checked by hand)

Step	Operation	S1	S3 (the existential)
1	Eliminate ⇒	∀x (¬(Emp(x) ∧ ∃d(Doc(d)∧Restr(d)∧Handled(x,d))) ∨ Auth(x))	∃e (Emp(e) ∧ ∀d (¬(Doc(d)∧Audited(d)) ∨ Handled(e,d)))
2	Move ¬ inwards (quantifier duality ¬∃ ≡ ∀¬)	∀x (¬Emp(x) ∨ ∀d(¬Doc(d) ∨ ¬Restr(d) ∨ ¬Handled(x,d)) ∨ Auth(x))	∃e (Emp(e) ∧ ∀d (¬Doc(d) ∨ ¬Audited(d) ∨ Handled(e,d)))
3	Standardise variables apart	x_1, d_2	e_4, d_5
4	Skolemise	no existential left	e_4 ↦ **Sk1** (a constant): Emp(Sk1) ∧ ∀d_5(¬Doc ∨ ¬Audited ∨ Handled(Sk1, d_5))
5	Drop ∀	¬Emp ∨ ¬Doc ∨ ¬Restr ∨ ¬Handled ∨ Auth	Emp(Sk1) ∧ (¬Doc(d_5) ∨ ¬Audited(d_5) ∨ Handled(Sk1,d_5))
6	Distribute ∨ over ∧ → clauses	[¬Emp(x_1), ¬Doc(d_2), ¬Restricted(d_2), ¬Handled(x_1,d_2), Auth(x_1)]	[Emp(Sk1)] and [¬Doc(d_5), ¬Audited(d_5), Handled(Sk1,d_5)]

S2 becomes [¬Doc(d), ¬ContainsPD(d), Restricted(d)], S4 becomes three unit clauses [Doc(R1)], [ContainsPD(R1)], [Audited(R1)], and the negated query becomes [¬Emp(x), ¬Auth(x)]. In total **7 axiom clauses and 1 negated-goal clause**.

(b) Justification of the Skolem constant

Skolemisation replaces an existentially quantified variable by a term naming "the witness". The witness may depend on the universally quantified variables whose scope contains the ∃. In S3, ∃e is *outside* ∀d: one employee must work for every document, so the witness cannot depend on d and is a **constant Sk1**. (If the sentence had meant "for every audited document some employee handled it", ∀d ∃e would give a Skolem *function* Sk(d); the two readings are different statements and the English sentence "some

employee handled every document" is the first.) The constant is a fresh name, so it carries no meaning beyond the clauses it appears in.

(c) Proof that an authorised employee exists, and the proof tree

Refutation by resolution of the 8 clauses. The prover with **set of support** (every resolvent must have a parent in the support set that starts with the negated goal) found this 15-step refutation; the complete step list is printed in Appendix B.

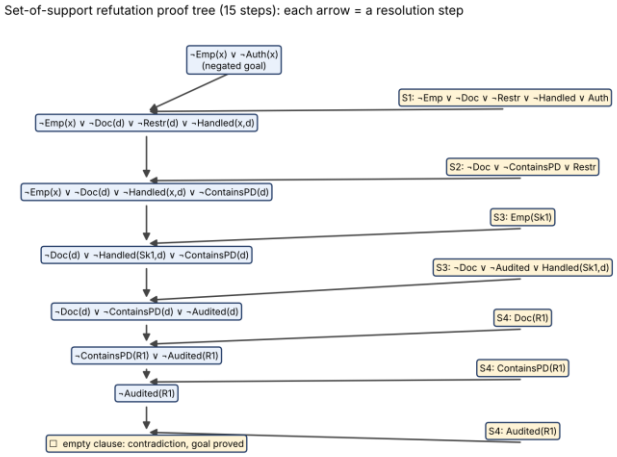


Figure 5.1 Proof tree of the set-of-support refutation. Blue clauses are resolvents, yellow clauses are given clauses (axioms). Each arrow is one resolution step; unifiers: x/Sk1, d/R1.

Reading the proof in words: the negated goal and S1 give "nobody handled a restricted document" (¬Emp ∨ ¬Doc ∨ ¬Restricted ∨ ¬Handled); S2 replaces Restricted by ContainsPD; Emp(Sk1) fixes x = Sk1; the Skolem clause from S3 replaces Handled(Sk1,d) by ¬Audited(d); the facts Doc(R1), ContainsPD(R1), Audited(R1) are then resolved one at a time with unifier d/R1, leaving the empty clause. The *witness* is the Skolem constant: some employee Sk1 handled R1, R1 is restricted, so Sk1 is authorised.

(d) The strategy: implementation

`fol_res.py` is a given-clause prover: pick a clause from the agenda, resolve it with every clause already processed (after standardising the pair apart, with the occurs check on), discard tautologies and duplicates (variable-renamed copies), and add the resolvents. The **agenda order** is what the strategies change: *plain* = first in first out (breadth-first); *unit preference* = shortest clause first; *unit + fair* = unit preference but every 5th pick is the oldest clause; *set of support* = only resolvents involving the support set. A clause may also be resolved with a *renamed copy of itself* (needed for recursive rules). Resolution is refutation-complete for first-order logic; unit preference and set of support are refutation-complete for the full language only when combined with fairness conditions, which is why I added the fair variant.

5.3 Student B – Comparative evidence

(a) With and without the strategy

Strategy	Result	Clauses generated	Resolution steps	Unifications	Given clauses	Proof length	Time (ms)
plain (FIFO)	PROVED	97	507	12,544	96	18	15.3
unit preference	PROVED	42	53	358	19	18	1.0
unit pref. + every 5th oldest	PROVED	42	53	359	19	18	1.1
set of support	PROVED	50	128	896	47	15	2.6



Figure 5.2 Left: cost of the entailed query under three strategies. Right: first-order set-of-support resolution against propositional resolution on instantiated clauses.

Unit preference reduced the resolution steps from 507 to 53 (9.6×) and the unifications from 12,544 to 358 (35×) with the same proof length (18); set of support gave the shortest proof (15 steps) with 4× fewer steps than plain resolution. Unit preference works here because three of the four axioms become unit clauses (the facts and Emp(Sk1)): resolving with a unit clause always shrinks the other clause.

(b) A second query that is not entailed, with a step cap

Query 2: *"some authorised employee handled a document R2"* – ∃x (Emp(x) ∧ Auth(x) ∧ Handled(x, R2)). Nothing says that Sk1 handled R2 (R2 is not audited, and nothing mentions it), so the query does not follow.

KB	Strategy	Outcome	Clauses generated	Steps
Finite Herbrand universe {Sk1, R1}	plain	**SATURATED** – no proof exists (clause set satisfiable)	205	1,195
	set of support	SATURATED	136	413

To expose the semi-decidability, S5 was added: *every employee has an employee manager*, ∀x (Emp(x) ⇒ ∃y (Emp(y) ∧ ReportsTo(x,y))). Skolemisation of S5 produces a Skolem **function** f(x): [¬Emp(x), Emp(f(x))] and [¬Emp(x), ReportsTo(x, f(x))]. Now the Herbrand universe {Sk1, f(Sk1), f(f(Sk1)), …} is infinite, and each new term can be resolved again.

Query (with S5)	Strategy	Outcome (cap: 20,000 steps / clauses)	Clauses generated	Steps
entailed: authorised employee	plain	PROVED	1,374	5,895
	unit preference	DIVERGED: term nesting exceeded the Python recursion limit (Sk1, f(Sk1), f(f(Sk1)), …)	–	–
	unit + fair	CAP reached (no proof within 20,000 steps)	11,304	20,025
	set of support	PROVED	186	413
not entailed: handled R2	plain	**CAP** – never terminates on its own	4,289	20,028
	unit preference	DIVERGED	–	–
	unit + fair	CAP	9,780	20,030
	set of support	CAP	6,058	20,002

Observations (several are not what one might expect). (1) Plain breadth-first resolution still proves the entailed query with S5, at 14× the clauses of the finite KB (1,374 against 97). (2) Pure unit preference is *incomplete in practice*: it keeps preferring the short Skolem-term unit clauses and diverges, so **a strategy that is fast on one KB can fail on another**. The fair variant did not rescue the proof within the cap (the oldest-clause rule is too rare). (3) Set of support is the most robust here: it found the proof with 186 clauses, and on the non-entailed query it behaves like every other strategy: it runs until the cap.

(c) Comparison with the propositional prover of Task 2 on an equivalent small domain

The FOL clauses were instantiated with every constant of a domain (Sk1, R1 and extra constants) and the ground clauses given to `logic.pl_resolution`:

Domain size	Ground clauses	Propositional: generated	attempted	time (ms)	FOL set of support: generated	steps	time (ms)
2	14	104	309	1.4	50	128	4.1
4	32	180	495	2.0	50	128	3.5
8	92	404	1,011	5.3	50	128	2.8
14	242	920	2,145	14.6	50	128	2.6

Both prove the query (result True), but the propositional prover must work through ground clauses that grow with the domain (14 → 242 ground clauses, 104 → 920 generated), while first-order resolution does the *same* 50 clauses and 128 steps for every domain size because unification finds the one useful instance (x/Sk1, d/R1) directly.

5.4 Cross-understanding requirements

Student B explains: why Skolemisation preserves satisfiability but not logical equivalence

Skolemisation turns ∃y ϕ(x,y) into ϕ(x, f(x)) with a **new** function symbol f. *Satisfiability is preserved*: if the original sentence has a model, define f in that model by picking, for every x, a witness y; the new sentence is true there. Conversely, a model of the Skolemised sentence gives, for every x, the witness f(x), so the original sentence holds in the same model. Hence the original and Skolemised sentences are satisfiable together (equisatisfiable), which is all that refutation needs.

Equivalence is not preserved: the Skolemised sentence uses a symbol f that does not occur in the original, so the two sentences have different vocabularies and cannot be equivalent. Example: ∃x P(x) is satisfied in a model where P holds of some element; P(Sk1) additionally asserts that the *name* Sk1 denotes such an element. An interpretation in which P holds of the element a but the new constant Sk1 denotes another element b where P is false satisfies ∃x P(x) and does not satisfy P(Sk1). So the Skolemised sentence is *stronger*: it entails the original sentence, but the original does not entail it, and they are not equivalent. The original sentence has a model exactly when the Skolemised one has (extend the model by letting Sk1 denote a witness).

Student A explains: what the experiments show about the cost of lifting resolution, and what the non-entailed query shows about decidability

Cost of lifting. Each resolution step now needs a unifier, a renaming of variables and an occurs check: the plain prover performed 12,544 unifications for 507 steps, 25 per step, because every candidate pair of literals must be unified to see whether it can be resolved. In return the search is independent of the domain size: the first-order prover needed 50 clauses and 128 steps for every domain size while the propositional prover grew from 104 to 920 clauses. Lifting thus moves the cost from the *number of ground instances* to the *cost per step* and to the *size of the search space of clause combinations*, which the strategies (unit preference: 507 → 53 steps) control.

Decidability. For the non-entailed query on the finite KB (no function symbols) the clause set saturates and the prover stops: the finite Herbrand universe makes entailment decidable (like the propositional case). With a Skolem function the Herbrand universe is infinite and, in my experiment, *all four strategies failed to terminate* on the non-entailed query (CAP or DIVERGED), while the entailed query was still proved (plain, set of support). That is the behaviour of a **semi-decidable** problem: a procedure exists that eventually proves every entailed query (resolution is refutation-complete), but no procedure can in general answer “no” to every non-entailed query. A step cap is therefore a *give-up* signal and not a proof of non-entailment.

Task 6 – Advanced Inference Mini-Investigations (7 marks)

6.1 The shared knowledge base: an organisation hierarchy

The facts are ReportsTo(e, m) (employee e reports to manager m), DeptMember(d, e), Lead(e) and Senior(e) (every employee who manages somebody). The hierarchy is a 4-ary tree rooted at e0. There are 10 definite clauses with approval rules (r5 is the approval rule):

```
r1  Above(x,y)       :- ReportsTo(y,x)
r2  Above(x,y)       :- ReportsTo(y,z), Above(x,z)              (right-recursive: recursive call last)
r3  Peer(x,y)        :- ReportsTo(x,m), ReportsTo(y,m)
r4  SameDept(x,y)    :- DeptMember(d,x), DeptMember(d,y)
r5  CanApprove(x,y)  :- Above(x,y), Senior(x)                   (approval rule)
r6  Escalation(x,m)  :- ReportsTo(x,m)
r7  SkipLevel(x,g)   :- Escalation(x,m), Escalation(m,g)
r8  TeamMember(x,m)  :- Above(m,x), Lead(m)
r9  Collab(x,y)      :- SameDept(x,y), Peer(x,y)
r10 Reviewer(x,y)    :- Peer(x,y), Senior(x)
```

Three fact sizes were generated by adding employees until the target was reached: **112, 1,117 and 11,167 base facts** (47, 467 and 4,667 employees; tree depth 3, 5, 6 for the leaf). Both engines use predicate indexes and a first/second-argument index on facts. Queries: **Q1** CanApprove(e0, leaf) (ground, true); **Q2** CanApprove(leaf, e0) (ground, false); **Q3** Above(e0, ?y) (every subordinate of the root); **Q4** CanApprove(?x, leaf) (who may approve the leaf); **Q5** CanApprove(e1, ?y) (subordinates of a mid-level manager).

6.2 Student A – Forward chaining investigation

`forward_chain` applies all rules until no new fact appears (the fixed point). It counts **facts inferred**, **rule firings** (every successful match of a rule body, whether or not the head is new), **redundant derivations** (firings whose head already existed or was derived in the same iteration), **iterations** and unifications. *Naive* evaluation re-matches every rule against all facts in every iteration; *semi-naive* evaluation lets a firing count only if its body uses a fact derived in the previous iteration.

Base facts	Employees	Mode	Iterations	Facts inferred	Rule firings	Redundant	Redundant %	Unifications	Time (s)
112	47	naive	5	1,195	5,313	4,118	77.5 %	15,720	0.04
		semi-naive	5	1,195	1,195	0	0 %	15,720	0.03
1,117	467	naive	7	13,841	84,417	70,576	83.6 %	254,300	0.84
		semi-naive	7	13,841	13,841	0	0 %	254,300	0.69
11,167	4,667	naive	8	154,671	1,030,838	876,167	85.0 %	3,070,851	39.4
		semi-naive	8	154,671	154,671	0	0 %	3,070,851	39.2

Iterations equal the depth of the hierarchy plus the two rule layers that sit on top of Above (CanApprove, TeamMember). **Semi-naive evaluation eliminates all redundant firings** (0 against up to 876,167): naive evaluation re-derives every old fact in every iteration. *Honest note on time:* my semi-naive filter is applied after matching, so the number of unifications and the run time are the same as for naive evaluation (3,070,851 and 39 s). A production semi-naive engine would join the *new* facts first through an index to avoid the matching work as well; the saving I measured is therefore in redundant derivations, not in unifications.

Derived facts and their relevance to the query

At 11,167 base facts forward chaining derives 154,671 facts: Above 26,184, CanApprove 26,184, Collab 15,864, Escalation 4,666, Peer 18,660, Reviewer 4,664, SameDept 46,649, SkipLevel 4,662 and

TeamMember 7,138. A derived fact is **relevant** to a query if it can take part in a derivation of the answer (computed by walking the derivations backwards through the fixed point).

Base facts	Query	Answers	Relevant derived facts	Total derived	Irrelevant %
112	Q1 CanApprove(e0, leaf)	1	4	1,195	99.67 %
	Q4 CanApprove(?x, leaf)	3	9	1,195	99.25 %
	Q3 Above(e0, ?y)	46	46	1,195	96.15 %
1,117	Q1	1	6	13,841	99.96 %
	Q4	5	20	13,841	99.86 %
	Q3	466	466	13,841	96.63 %
11,167	Q1	1	7	154,671	100.00 % (99.995 %)
	Q4	6	27	154,671	99.98 %
	Q3	4,666	4,666	154,671	96.98 %

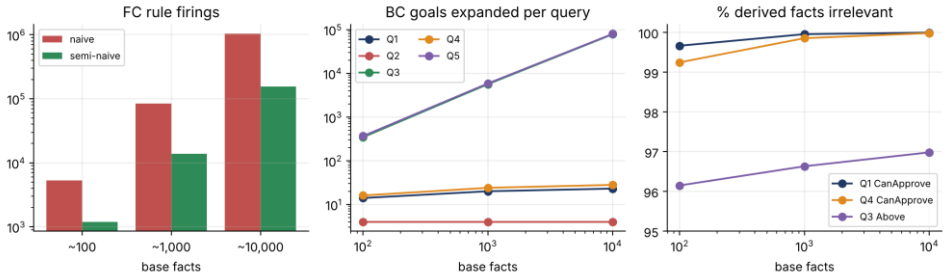


Figure 6.1 Left: rule firings of naive and semi-naive forward chaining. Middle: backward-chaining goals per query against KB size. Right: percentage of derived facts that are irrelevant to three queries.

What happens to the work when the query concerns a very small part of the KB. Forward chaining is *query-blind*: it derives everything. For Q1 (one ground question) it derived 154,671 facts of which 7 were relevant, so **99.995 % of the work is wasted** and grows linearly with the KB (4,118 → 70,576 → 876,167 redundant firings for naive evaluation). Even for Q3, which asks for the whole subtree of the root, 97 % of the facts are irrelevant (SameDept, Peer, Collab, Reviewer and the like). Forward chaining is attractive only when the answers to *many different* queries are needed, or when new facts arrive and all conclusions must be kept up to date.

6.3 Student B – Backward chaining investigation

`Prover` is SLD backward chaining with substitutions: the goal is unified with a fact or with the head of a renamed rule, the body goals are proved left to right under the accumulated substitution, and negation (not used in this KB) would be evaluated as failure on ground literals. Counters: **goals expanded**, **unifications**, **maximum depth** and **repeated subgoals** (a goal that is a *variant*, the same up to variable renaming, of a goal already expanded). Every answer set was compared with the forward-chaining fixed point.

Base facts	Query	Solutions	Goals expanded	Unifications	Max depth	Repeated subgoals	Time (ms)	= FC answer
112	Q1 CanApprove(e0,leaf)	1	14	16	5	0	0.14	yes
	Q2 CanApprove(leaf,e0)	0	4	3	2	0	0.04	yes
	Q3 Above(e0,?y)	46	345	416	4	306	2.8	yes

Base facts	Query	Solutions	Goals expanded	Unifications	Max depth	Repeated subgoals	Time (ms)	= FC answer
	Q4 CanApprove(?x,leaf)	3	16	18	5	4	0.12	yes
	Q5 CanApprove(e1,?y)	20	366	437	5	325	2.6	yes
1,117	Q1	1	20	24	7	0	0.15	yes
	Q2	0	4	3	2	0	0.04	yes
	Q3	466	5,649	7,068	6	5,295	43	yes
	Q4	5	24	28	7	6	0.20	yes
	Q5	210	5,860	7,279	7	5,504	69	yes
11,167	Q1	1	23	28	8	0	0.20	yes
	Q2	0	4	3	2	0	0.05	yes
	Q3	4,666	78,555	100,076	7	75,051	695	yes
	Q4	6	28	33	8	7	0.31	yes
	Q5	1,364	79,920	101,441	8	76,414	635	yes

Ground queries (Q1, Q2) and the "who may approve" query (Q4) are answered with 4–28 goals regardless of the KB size: Q1 grows from 14 to 23 goals while the KB grows 100× (the goals follow the *chain of managers*, whose length is the tree depth: 3, 5, 6). The *enumeration* queries (Q3, Q5) are linear in the number of answers (and a constant factor higher, 17 goals per answer at 11,167 facts) because the rule ReportsTo(y, z) is entered with y unbound and scans every employee; the repeated subgoals are the same intermediate managers reached through different employees.

Direction of inference: forward chaining for the whole closure against backward chaining for one query

Base facts	FC firings	FC unifications	FC time (s)	BC goals (Q1)	BC unifications	BC time (ms)	Unification ratio FC/BC	Queries before FC pays off (time)
112	1,195	15,720	0.03	14	16	0.145	982	214
1,117	13,841	254,300	0.69	20	24	0.146	10,596	4,731
11,167	154,671	3,070,851	39.2	23	28	0.203	109,673	192,734

Forward chaining performs **110,000× more unifications** than backward chaining at 11,167 facts, and one would have to ask nearly 200,000 *different* approval questions before the closure was amortised. Direction of inference is thus a *performance* choice and not a logical one: the two engines return identical answers (all 15 comparisons above).

6.4 The left-recursive rule: non-termination and three remedies (Student B)

Replace r2 by the equivalent **left-recursive** rule (the recursive call is first): `AboveL(x,y) :- AboveL(x,z), ReportsTo(y,z)` with the base rule `AboveL(x,y) :- ReportsTo(y,x)`. Both define "x is above y". Query AboveL(e0, ?y) on the 1,000-fact KB (466 correct answers).

Non-termination (demonstration). Plain backward chaining proves the first goal by calling AboveL(e0, $?z_1$) – a *variant of the goal itself* – which calls AboveL(e0, $?z_2$), and so on. The answers do appear (the base clause is tried at every level, 466 answers were produced), but the search never finishes: with a budget of 200,000 goals it was stopped at depth 432, and without the budget it would run until the stack overflows. The program cannot know that it has found all answers because no branch ever fails.

Remedy	Terminates?	Answers (of 466)	Goals expanded	Unifications	Comment
none (plain SLD)	**no**	466 found	200,000 (budget stop)	–	depth 432 reached
1 loop check (cut variants of an ancestor goal)	yes	**4 – incomplete**	3	6	cuts the recursive branch: only the base clause (direct reports) answers
2 depth bound L (iterative deepening)	yes	complete from L = 6	2,958 cumulative (L = 1…8)	4,796	must guess L; repeats the work of all smaller bounds
3 tabling (memoisation)	yes	466 – complete	929	3,676	5 passes, 468 tables, 5 table hits; 15 ms

Depth-bound detail (answers found, goals): L = 1: 0, 3; L = 2: 4, 5; L = 3: 20, 11; L = 4: 84, 33; L = 5: 340, 119; **L = 6: 466, 461**; L = 7: 466, 929; L = 8: 466, 1,397. The answers become complete at L = 6, equal to the depth of the hierarchy, but the program has no way to *know* that: the search for L = 7 and 8 still finds nothing new and still never fails. Each additional bound costs about as much as the whole search up to that depth (goals grow 2× to 4× per level while the tree widens).

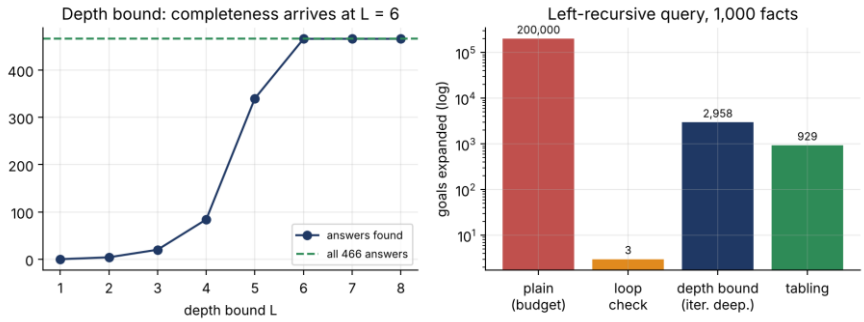


Figure 6.2 Left: answers found by a depth bound L. Right: goals expanded for each remedy on the left-recursive query (log scale).

Tabling at three sizes

Base facts	Answers	Complete	Passes	Tables	Goals	Unifications	Table hits	Time (ms)
112	46	yes	3	48	79	300	3	1.6
1,117	466	yes	5	468	929	3,676	5	15.5
11,167	4,666	yes	6	4,668	6,496	28,770	6	147

The tabling cost is linear in the KB (6,496 goals for 4,666 answers) and its price is *memory* (one table per call variant) and repeated passes until no table changes (passes = depth).

Repeated subgoals and cycles (right-recursive rule)

Cyclic data. With ReportsTo(a,b), ReportsTo(b,a), ReportsTo(c,b) the right-recursive Above loops as well: plain backward chaining expanded 3,000 goals (budget) to depth 1,000 on the failing query Above(z, c). The loop check terminates (10 goals, 1 cut) and is correct here; forward chaining always terminates (6 Above facts in 3 iterations) because facts are never repeated.

Matrix organisation (each employee has a second, dotted-line manager, so the reports-to graph is a DAG). Query Above(e2, leaf) that fails, so every path is explored:

Facts	Mode	Goals	Unifications	Repeated subgoals	Repeated %
110	plain	21	26	6	28.6 %
	tabled	34	56	4	11.8 %
1,080	plain	60	78	33	55.0 %

Facts	Mode	Goals	Unifications	Repeated subgoals	Repeated %
	tabled	64	128	10	15.6 %
10,797	plain	171	226	132	77.2 %
	tabled	98	210	20	20.4 %

The share of repeated subgoals of plain search grows (29 % → 55 % → 77 %) as paths merge; tabling expands each distinct goal once (98 against 171 goals at 10,797 facts). For small KBs tabling costs *more* goals (34 against 21) because of its repeated passes; it pays off when many paths share sub-hierarchies.

6.5 Verification questions

Student B explains: why is the choice of inference direction central to performance on this KB?

Because the KB has **many derived relations that are irrelevant to any one question** and a **hierarchy whose useful part is a thin path**. Forward chaining must compute all 154,671 consequences (SameDept, Peer, Collab… over 4,667 employees) before it can answer; backward chaining starts from the goal and follows only the chain of managers, 23 goals for Q1. The direction also decides whether a rule is cheap: r2 is efficient *backward* when the employee is known (follow ReportsTo upwards, O(depth)), but enumerating all subordinates backward scans every employee (78,555 goals), where forward chaining with the same rule would compute all pairs once. So the answer depends on the *query pattern*: one question → backward; all conclusions needed for many questions → forward.

Transfer question ("if the KB grows to 100,000 facts and queries are rare, would you change?"): the forward-chaining cost grew 11.6× per 10× facts (154,671 inferred for 11,167 facts) and would be about 1.8 million derived facts at 100,000 facts, while Q1 grew from 14 to 23 goals over a 100× increase. With rare queries I would use **backward chaining** (with tabling if the hierarchy has cycles or shared sub-hierarchies); I would switch to forward chaining only if every employee's approvals were queried continuously.

Student A explains: why can backward chaining remain efficient when the KB contains many irrelevant facts?

Backward chaining is **goal-directed**: it expands only goals that can contribute to proving the query, and it reaches facts through an index on predicate and first argument, so facts about other employees, departments and peers are never touched. The measured evidence: Q1 needed 14, 20 and 23 goals for 112, 1,117 and 11,167 facts – the work depends on the *depth of the relevant chain*, not on the number of facts. The relevant derived facts are 4–7 while more than 99.9 % of the forward-chaining closure is irrelevant to Q1 and Q4. The efficiency disappears for queries whose answer set is large (Q3: 78,555 goals), for left-recursive rules without a remedy, and when many goals repeat (DAG hierarchies), where tabling helps.

Task 7 – Day-4 Design Challenge: Hospital Medication-Safety Advisor (10 marks)

Problem. Given a patient record (age, allergies, current medicines, conditions) and a newly prescribed drug, decide whether the prescription is safe and explain why. The initial KB has about 40 rules and must grow to 2,000 rules and 50,000 patients. The work was done jointly in six stages; the independent proposals of Stage 1 are kept unchanged.

Stage 1 – Independent design proposals (1 mark)

Student A's initial proposal (written alone, before discussion)

Representation: propositional logic. One proposition per fact, e.g. `Allergy_penicillin`, `Takes_warfarin`, `Rx_aspirin`, `Pregnant`. One Horn rule per hazard, e.g. `Rx_amoxicillin ∧ Allergy_penicillin ⇒ Danger_allergy_amox`.

Vocabulary: one symbol per drug, allergen, condition and age band; about 100 symbols for 40 rules.

Inference: *forward chaining* over the patient's facts until no new danger symbol is added (linear time for propositional Horn clauses), because it is simple and terminates.

Explanation: record for each derived `Danger_*` symbol the rule that fired and its premises (a derivation list).

Weakness I did not see yet: one KB per patient (or the symbols must be tagged with the patient).

Student B's initial proposal (written alone, before discussion)

Representation: first-order logic as definite clauses over a patient variable: `Danger(p, why) ⇐ Rx(p, warfarin) ∧ Takes(p, aspirin)`; patient data as facts `Takes(P17, aspirin)`.

Vocabulary: predicates Age, Allergy, Takes, Condition, Rx, Danger, Child, Elderly; constants for drugs, allergens, conditions and patient ids.

Inference: *backward chaining* from the query Danger(patient, ?why), because only the rules of the prescribed drug matter and the KB will have 2,000 rules and 50,000 patients; **resolution** for the harder, non-Horn knowledge ("allergic to A or B").

Explanation: the proof tree of the query, printed as "because Rx(P17, warfarin) and Takes(P17, aspirin)".

Weakness I did not see yet: the cost of scanning 2,000 rules per query; negation as failure for "safe".

Stage 2 – Comparison of the two designs (1 mark)

Aspect	Student A (propositional, forward)	Student B (FOL, backward)	Decision
Where they agree	Rule-based, definite (Horn) clauses; explanations from the derivation; the check is deterministic	same	Keep: **definite clauses + derivation-based explanations**
Representation	No variables: a copy of every rule per patient or tagged atoms (50,000 patients × 2,000 rules = 10^8 ground rules)	One variable ?p: 2,000 rules for all patients; patient data are facts	**Use FOL (Datalog-style definite clauses)** – B
Inference	Forward chaining computes *all* dangers of a patient, linear time, terminates	Backward chaining touches only the rules relevant to the drug	**Backward chaining per prescription; forward chaining for batch audits** – both
Explanation	A list of fired rules	A proof tree with the facts that matched	Proof tree, rendered as text – B (A's rule list is its flattening)
Strength	Simplicity, guaranteed termination, no unification	Scales in the KB and the patient count, natural for explanation	–

Aspect	Student A (propositional, forward)	Student B (FOL, backward)	Decision
Weakness	Does not scale with patients; query-blind	Needs an index (2,000 rules to scan); NAF assumes complete records	Add a **rule index by trigger drug** – a new idea from the discussion
Left out / open	Resolution for non-Horn knowledge	Same	Out of scope of the executable KB; flagged as a limit (Stage 6)

Stage 3 – Joint final design (3 marks)

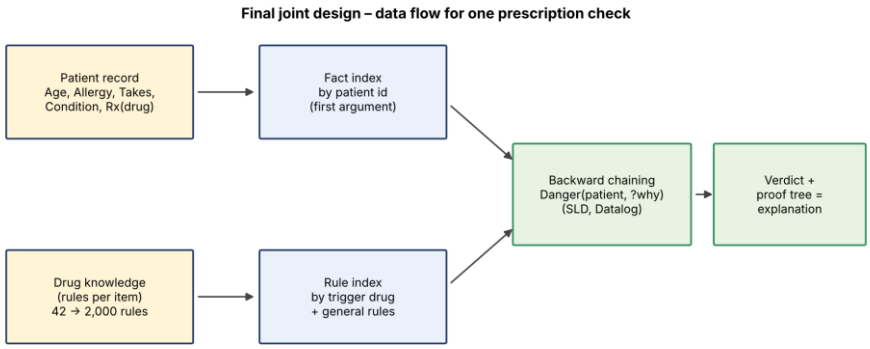


Figure 7.1 Data flow of the final design for one prescription check.

Representation and vocabulary

First-order **definite clauses** with a single universally quantified patient variable ?p (a Datalog-style KB: no function symbols, so the set of ground atoms is finite). The knowledge items are *rules*, one per hazard, because the hospital maintains pharmacological knowledge as rules that grow from about 40 to 2,000:

Family	Rule pattern	Example	In the 42-rule KB
Allergy	Danger(?p, allergy:S:D) ⇐ Rx(?p, D), Allergy(?p, S)	penicillin allergy, amoxicillin	9
Interaction	Danger(?p, interaction:D+E) ⇐ Rx(?p, D), Takes(?p, E)	warfarin + aspirin (both directions)	16
Contra-indication	Danger(?p, contra:D:C) ⇐ Rx(?p, D), Condition(?p, C)	ibuprofen with kidney disease	8
Age	Danger(?p, age:child:D) ⇐ Rx(?p, D), Child(?p) (and Elderly)	aspirin for a child	5
General	Danger(?p, duplicate:nsaid) ⇐ Rx(?p, D), NSAID(D), Takes(?p, E), NSAID(E)	two NSAIDs	1
Helpers	Child(?p) ⇐ Age(?p,a), Ge(11,a); Elderly(?p) ⇐ Age(?p,a), Ge(a,75); Unsafe(?p) ⇐ Danger(?p,w)		3

Predicates: Age(p, years), Allergy(p, allergen), Takes(p, drug), Condition(p, condition), Rx(p, drug), NSAID(drug); derived: Child (age ≤ 11), Elderly (age ≥ 75), Danger(p, reason), Unsafe(p). A prescription is **unsafe** if at least one Danger(p, ?why) is provable and **safe by the KB** if none is (negation as failure over a complete record).

Inference method, indexing and explanation

- **Inference:** SLD backward chaining on the goal Danger(patient, ?why) – all answers are collected, so *every* reason is reported, not only the first. A forward-chaining batch job (semi-naive) is used to audit all patients overnight.
- **Indexing for scale:** (1) *Facts are indexed by predicate and by patient id* (first argument), so a query touches only the 6 facts of the patient. (2) *Rules are indexed by the trigger drug* (the constant in Rx(?p, D)): the rules examined for a prescription are those of that drug plus the few general rules.

- **Explanation:** the proof tree of each provable Danger, printed as a rule and the facts that satisfied it. The length of the explanation is the number of nodes (1 rule + its fact leaves; 3 for a drug-triggered rule).

Pseudocode

```
function BUILD(rules, facts):
    for each fact f:  facts_by_patient[f.patient].add(f)          # index 1
    for each rule r:                                              # index 2
        if r has the form  Danger(p, why) <- Rx(p, D), ...  with constant D:
            rules_by_drug[D].add(r)
        else:
            general_rules.add(r)

function CHECK(patient, drug):                                    # one prescription
    R <- general_rules + rules_by_drug[drug]
    reasons <- {};  proofs <- {}
    for each rule  (Danger(p, why) <- b1, ..., bn)  in R:
        rename the variables of the rule (standardise apart)
        theta <- UNIFY(goal Danger(patient, ?why), head)           # binds p := patient, why := constant
        for each theta2 in PROVE-ALL([b1..bn], theta):             # left-to-right, depth-first
            reasons.add(SUBST(theta2, why));  proofs.add(proof tree)
    return (reasons, proofs)                                      # reasons = {} means: safe

function PROVE(goal, theta):                                      # SLD resolution
    if goal is Ge(a,b): yield theta if both are numbers and a >= b
    for each fact f in facts_by_patient[goal.patient or predicate index]:
        if (theta2 <- UNIFY(goal, f, theta)) succeeds:  yield theta2
    for each helper rule (Child, Elderly, Unsafe) with head matching goal:
        for each theta2 in PROVE-ALL(body, theta): yield theta2

function AUDIT(all_patients):                                     # batch forward chaining
    repeat until no new fact: apply every rule to the facts that are new since the last pass
```

Why this design: definite clauses give efficient, terminating, explainable inference (Stage 4A); one patient variable keeps the KB at 2,000 rules for 50,000 patients; the two indexes make the cost of a check independent of the number of rules and patients (Stage 4B).

Stage 4A – Student A: theoretical validation (1.5 marks)

Soundness and completeness of the chosen inference

- **Soundness.** Every step of SLD resolution on a definite clause is an instance of lifted Modus Ponens (Task 4): the substitution θ makes the body literals equal to proven facts, so the head SUBST(θ, head) is entailed. By induction on the proof, every reported Danger is entailed by the KB and the patient's facts. Therefore the advisor never invents a danger (no false alarms *relative to the KB*).
- **Completeness.** For definite clauses, SLD resolution with a fair, exhaustive search finds every entailed atom (Horn clauses have a unique least model, and the least model is the set of atoms derivable by chaining). My KB is *Datalog* – no function symbols, so the Herbrand base is finite and the rules are non-recursive – hence the search always terminates and returns *all* Danger reasons. The index does not lose completeness because a rule not triggered by the prescribed drug has a body literal Rx(?p, D') with D' ≠ D that can never match the patient's single Rx fact.
- **Negation as failure** is sound only under the *closed-world assumption*: "no danger is provable" means "no danger is entailed by what is recorded". If a record is incomplete (an allergy not entered), the advisor reports *safe*. This limit is discussed in Stage 6.

Complexity of a query as a function of the KB size

Let R be the number of rules, F the number of facts, $f_p \approx 6$ the facts of one patient, and k_d the number of rules triggered by drug d.

Method	Cost of one prescription check	Dependence on R and on the number of patients
Backward chaining, rules indexed by drug	O(k_d · f_p) unifications; k_d = (R_drug-rules / number of drugs) + g (g = general rules)	constant in R on average (k_d ≈ 6 at R = 2,000), independent of the patients
Backward chaining, no index	O(R · f_p): every Danger rule is renamed and its first body literal tested	linear in R, independent of the patients
Forward chaining (batch)	O(R · F_new) per pass, number of passes = depth of derived predicates (4)	linear in R and in the number of patients; amortised ≈ cost of one check per patient
Propositional model checking	O(2^m) for m symbols	infeasible: 50,000 patients × 2,000 rules = 10^8 ground rules
Propositional Horn entailment (forward chaining on ground rules)	O(total size of the ground KB) – linear (Dowling–Gallier)	linear, but the ground KB is 10^8 rules
General propositional entailment (non-Horn)	co-NP-complete	avoided by the Horn restriction

Effect of restricting the KB to definite clauses

Gains: (1) entailment is decidable *in polynomial time* for Datalog (data complexity polynomial; with the indices, constant per query); (2) one *minimal model* – the answers are unique and every conclusion has a proof tree for the explanation; (3) chaining is enough, no CNF conversion or resolution search is needed; (4) the KB is easy to maintain: one rule per item. **Losses:** definite clauses cannot say "the patient is allergic to A *or* B", cannot assert a negative fact ("not allergic") except by omission, cannot state uncertainty ("probably"), and cannot conclude a disjunction ("either X or Y is dangerous"). Those statements need non-Horn clauses and resolution, with the exponential worst case seen in Task 2.

Stage 4B – Student B: experimental validation (1.5 marks)

`med_kb.py` generates the KB (the 42-rule base plus synthetic items in the same proportions, up to 2,000 rules); `t7_run.py` runs the experiments. **Metrics:** inference steps (goals expanded + unifications), query latency (mean and 95th percentile, µs), memory (KB build peak and per-query peak), explanation length (proof nodes).

(a) 37 test cases including dangerous combinations

Expected verdicts were written from the clinical specification before the program ran: 31 dangerous cases (11 drug combinations covering all eight interaction pairs in both orders, allergies, contra-indications, age boundaries 11/12 and 74/75, multi-reason cases) and 6 safe cases. The **first draft** KB (39 rules) passed 34 of 37:

Failing case	Expected	Got	Defect and fix
T11 spironolactone prescribed, patient on lisinopril	interaction	none	The interaction was written in one direction only. Added the reverse rule (also for methotrexate/trimethoprim).
T13 trimethoprim prescribed, patient on methotrexate	interaction	none	same defect
T18 diclofenac with NSAID allergy	allergy:nsaid:diclofenac	none	diclofenac was a known NSAID fact, but the allergy list had no rule for it. Added the rule.

After the fixes the final KB (42 rules) passes **37/37**. The three defects were found by tests written *before* the KB; the lesson is that a symmetric relation (interaction) must be encoded in both orders or by a symmetric helper rule.

#	Case	Result	Goals	Unifications	Explanation nodes	µs
T01	warfarin + aspirin (dangerous combination)	pass	9	10	3	129

#	Case	Result	Goals	Unifications	Explanation nodes	µs
T02	aspirin prescribed, patient on warfarin (reverse order)	pass	15	15	3	234
T03	ibuprofen prescribed, patient on warfarin	pass	13	13	3	103
T04	sertraline + tramadol (serotonin syndrome)	pass	7	8	3	64
T05	tramadol prescribed, patient on sertraline	pass	9	9	3	85
T06	sertraline + phenelzine (MAOI)	pass	7	8	3	62
T07	simvastatin + clarithromycin	pass	7	7	3	65
T08	sildenafil + nitroglycerin	pass	5	5	3	48
T09	nitroglycerin prescribed, patient on sildenafil	pass	5	5	3	49
T10	lisinopril + spironolactone (hyperkalaemia)	pass	7	7	3	65
T11	spironolactone prescribed, patient on lisinopril	pass	5	5	3	47
T12	methotrexate + trimethoprim	pass	5	5	3	47
T13	trimethoprim prescribed, patient on methotrexate	pass	5	5	3	73
T14	amoxicillin, penicillin allergy	pass	5	5	3	53
T15	cefalexin, penicillin allergy (cross-reactivity)	pass	5	5	3	54
T16	sulfamethoxazole, sulfa allergy	pass	5	5	3	47
T17	ibuprofen, NSAID allergy	pass	12	12	3	95
T18	diclofenac, NSAID allergy	pass	6	6	3	54
T19	ibuprofen, kidney disease	pass	12	13	3	94
T20	metformin, kidney disease	pass	5	5	3	50
T21	lisinopril, pregnancy	pass	7	7	3	63
T22	warfarin, pregnancy	pass	9	9	3	73
T23	aspirin, child aged 8	pass	14	13	5	110
T24	aspirin, boundary age 12 (not a child)	pass	14	13	0	100
T25	tramadol, boundary age 11 (child)	pass	9	8	5	207
T26	diazepam, age 80	pass	7	6	5	96
T27	diazepam, boundary age 74 (not elderly)	pass	7	6	0	197
T28	diazepam, boundary age 75 (elderly)	pass	7	6	5	104
T29	warfarin + aspirin + pregnancy (two reasons)	pass	9	11	6	136
T30	aspirin, child 9, NSAID allergy, on warfarin, ulcer (four reasons)	pass	15	17	14	164
T31	ibuprofen, patient on naproxen (duplicate NSAID)	pass	13	14	5	167
T32	ibuprofen, patient already on ibuprofen	pass	13	14	5	171
T33	simvastatin, liver disease, on clarithromycin (two reasons)	pass	7	8	6	81
T34	paracetamol, full risky history (safe: no rule applies)	pass	3	2	0	44
T35	amoxicillin, no allergy (safe)	pass	5	4	0	103
T36	amoxicillin, sulfa allergy only (wrong allergen, safe)	pass	5	5	0	59
T37	metformin, patient on ibuprofen, no condition (safe)	pass	5	4	0	48

Dangerous cases need 5–15 goals and give explanations of 3 to 14 nodes (3 per reason: the rule and its two facts; age rules add the Child/Elderly derivation, 5 nodes). Safe cases (T34–T37) are answered in 3–5 goals with **no** explanation: the advisor can only say “no rule applies”, a closed-world answer.

(b) KB sizes of 42, 400 and 2,000 rules; baselines

2,000 patients with one prescription each (40 % of them built to hit a random rule; 48–62 % of all prescriptions turned out dangerous). The **baselines** are (B1) the same backward chainer *without the rule index* – every Danger rule is tried – and (B2) batch **forward chaining** over all patients.

Rules	Mode	Goals/query	Unifications/query	Rule tries/query	Mean µs	p95 µs	Explanation nodes	Build (s)	Query peak memory (KB)
42	indexed	8.0	9.9	3.0	80	167	5.2	0.05	135
42	unindexed (B1)	44.1	82.1	39.2	523	677	5.2	0.04	183
400	indexed	12.3	17.2	5.6	123	233	4.1	0.04	172
400	unindexed (B1)	404.4	801.5	397.8	5,652	8,227	4.1	0.04	187
2,000	indexed	13.4	19.0	6.2	178	353	4.0	0.09	180
2,000	unindexed (B1)	2,005	4,002	1,998	29,625	43,463	4.0	0.06	211

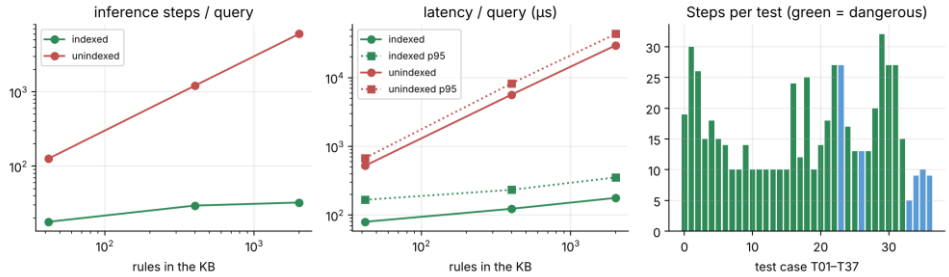


Figure 7.2 Left: inference steps per query. Middle: query latency (mean and p95). Right: steps per test case T01–T37 (green = dangerous). Indexed versus unindexed backward chaining at 42, 400 and 2,000 rules.

Results. (1) Indexed steps grew from 17.9 to 32.4 per query while the KB grew 48×: 1.8× growth against **48× for the unindexed baseline** (126 to 6,007 steps; rule tries 39 → 1,998). The indexed rule tries (3.0 → 6.2) are the number of rules a drug triggers. (2) Mean latency 80 → 178 µs (indexed) against 523 → 29,625 µs (unindexed): at 2,000 rules the index is **166× faster**. (3) All three modes returned identical reasons for every one of the 2,000 patients at every size. (4) Memory is dominated by the facts; the index adds little, and the per-query peak stays at about 0.2 MB. (5) Explanation length stays at 4–5 nodes for the 42-to-2,000-rule KB (the proof of one reason is 3 nodes).

Rules	Forward chaining (all 2,000 patients)	Per patient (amortised)	Backward chaining indexed (per check)	Answers agree
42	4,511 firings, 75,100 unifications, 3,798 facts inferred, 4 iterations, 0.34 s	169 µs	80 µs	yes
400	3,140 firings, 106,840 unifications, 2,923 facts, 4 iterations, 0.43 s	216 µs	123 µs	yes
2,000	2,892 firings, 115,260 unifications, 2,725 facts, 4 iterations, 0.37 s	185 µs	178 µs	yes

Forward chaining is competitive *per patient* (185 µs against 178 µs) when **all** patients are processed in one batch, which supports the design decision to use it for the nightly audit; but it must re-run (or be made incremental) when a single patient is admitted, where backward chaining answers in 178 µs.

(c) Scale test: 2,000 rules and 50,000 patients

Measure	Value
Facts loaded	341,062 (about 6.8 per patient)
KB build time / memory	5.95 s / 120 MB (peak)
3,000 random prescription checks	mean **158 µs**, p95 341 µs, maximum 1,400 µs; mean 32 steps; 48 % flagged dangerous
Forward chaining over all 50,000 patients (baseline B2)	30.9 s, 74,660 firings, 2,887,520 unifications, 69,875 facts inferred
Propositional encoding for comparison	50,000 × 2,000 = **100,000,000** ground rules; the first-order KB keeps 2,000 rules

A check at 50,000 patients (158 µs mean) costs the same as at 2,000 patients (178 µs): the number of patients does not matter because facts are indexed by patient. Even the slowest single check (1.4 ms) is far below any clinical latency limit (the pharmacist's screen response).

Stage 5 – Cross-defence (1 mark)

Student A explains the experimental evidence supporting the group's design

(1) **Correctness:** 37 of 37 test cases pass, including all 11 dangerous drug combinations in both directions, the age boundaries (11/12, 74/75) and multi-reason cases; the three failures of the draft were found by the tests and fixed. (2) **Indexing is what makes it scale:** with 2,000 rules the unindexed baseline needs 2,005 goals per check (29,625 µs) while the indexed design needs 13 goals (178 µs), 166× fewer microseconds; the steps grew 1.8× for a 48× larger KB. (3) **Patients do not matter:** the check on 50,000 patients costs 158 µs, the same as at 2,000 patients. (4) **Forward chaining** is good for batch audits (amortised 185 µs per patient) but takes 30.9 s for all 50,000 patients and must be re-run for changes, which is why backward chaining is the default.

Student B explains why the theoretical complexity result applies to the group's inference

The complexity result is for **definite clauses without function symbols** (Datalog) with SLD resolution: the cost of a query is proportional to the number of rules whose head unifies with the goal and the facts they need, not to R or to the number of patients. The group's inference is exactly SLD backward chaining on such a KB: the rules are definite clauses, the only built-in is a comparison Ge on numbers, there are no function symbols and no recursion (so the search terminates), and the rule index makes the number of rules examined k_d, a constant of the drug rather than of the KB. The measurements match the formula: rule tries per check were 3.0, 5.6 and 6.2 at 42, 400 and 2,000 rules (indexed) against 39.2, 397.8 and 1,998 (unindexed), i.e. O(k_d) against O(R). If the KB contained recursive rules or non-Horn clauses (disjunctions) the polynomial bound would no longer apply and the left-recursion problems of Task 6 and the exponential resolution of Task 2 would reappear.

Stage 6 – Joint evaluation (1 mark)

Why the problem suits a logical agent

- The knowledge is **rule-like and explicit** (pharmacology guidelines are written as "if the patient takes X and is prescribed Y then…"), so it can be encoded directly, reviewed by clinicians, and extended by adding rules.
- **Explanations are mandatory** in medicine: a proof tree is the explanation; the pharmacist can see which rule and which facts led to the alert.
- Decisions are **deterministic and auditable**: the same record gives the same verdict, and a change in the KB can be tested against the 37 cases.
- The reasoning is **cheap** if the KB is Horn and indexed (Section 4B).

Limits of the representation

- **Uncertainty.** Real interactions have severities and probabilities ("major", "moderate", "1 in 1,000"). A definite-clause KB is yes/no. Risk scores or a probabilistic extension (Bayesian networks, Unit IV) are needed for graded alerts.
- **Incomplete data.** Negation as failure treats a missing allergy record as "no allergy" (closed world). A patient who never reported an allergy is declared safe. A real system needs an *unknown* state, an alert like "allergy status not recorded", and data-quality checks.

- **Disjunctive and negative knowledge** (“allergic to A or B”, “not allergic to latex”) is outside definite clauses and requires resolution; **dosage and timing** (numeric reasoning) are only partly handled with the Ge built-in; **temporal** facts (a drug stopped last week) are not modelled.
- **Knowledge acquisition and maintenance.** 2,000 rules must be correct; the one-direction interaction defect shows how easily they are not. Rules need version control, clinical review and regression tests.

What a designer should check before choosing a knowledge-based approach

1. Is the knowledge available in *explicit, stable, expert-agreed* form, and does it change slowly enough to maintain by hand?
2. Are explanations or audit trails required? (If yes, logic is attractive.)
3. Is the data complete and reliable (closed world), or is uncertainty central? If uncertain, use probabilistic or learning methods.
4. What are the expected KB size, number of cases and query pattern? Measure the cost with the *intended* inference method and index, as in Stage 4B, before committing.
5. Is the KB within a tractable fragment (Horn/Datalog), and if not, which of model checking, resolution, SAT or an approximation can serve it? (Task 2.)
6. Can the KB be validated by tests with expected answers written independently, and is there an owner for the rule base?

Contribution Record

Primary contribution does not imply exclusive responsibility: each student has studied the whole assignment and answered the cross-understanding questions for the partner's parts (boxes in each task).

Task	Student A – Saai Vardhan D (192525205) – primary contribution	Student B – [partner name] ([register no.]) – primary contribution	Joint work
Task 1	Symbol and sentence counts, model count 2^m, relevant symbols, soundness/completeness of model checking; implementation of the inference-rule derivation and step counts versus model checking	Instrumented truth-table checker; runs for n = 2, 3, 4 with and without restriction; entailed / negation-entailed / neither queries; truth-table soundness proofs of the rules; "not derived ≠ false"	Wumpus KB encoding, truth-table checker, percept design
Task 2	Manual CNF of three sentences, resolution proof, chains of 10–80 with all counters, non-entailed query and termination argument	Model-checking counts for the same KBs; KB families that favour each method (chains, pigeonhole) and the structural explanation	Both implementations, the 12-rule policy, designed KBs, joint design discussion
Task 3	Vocabulary, 14 axioms with existentials, growth derivation N_prop(s,c), constancy of the FOL axioms	15-query suite, three-defect log, size/time measurements for both encodings, crossover range	Knowledge-engineering process, both encodings, joint conclusion
Task 4	Ground-clause growth in k and arity, function-symbol growth, lifted versus propositional comparison	Hand-derived unifiers for five pairs, soundness of lifted inference, why it avoids ground instances	UNIFY, composition, lifted Modus Ponens, propositionalisation, engineering decision
Task 5	CNF conversions step by step, Skolem justification, proof tree, strategy implementation	Clauses/steps/unifications with and without strategy, non-entailed query under a step cap, propositional comparison	FOL prover design and the four statements
Task 6	Forward chaining to a fixed point; facts inferred, firings, redundancy, iterations, % irrelevant; the work for small queries	Backward chaining with substitutions; left-recursive rule, three remedies and their costs; goals, unifications, depth, repeated subgoals	The 10-rule hierarchy KB, three fact sizes, verification questions
Task 7	Stage 1 proposal (propositional, forward chaining); Stage 4A theoretical validation	Stage 1 proposal (FOL, backward chaining); Stage 4B experimental validation (37 tests, 42/400/2,000 rules, baselines, 50,000 patients)	Stages 2, 3, 5, 6: comparison, final design and pseudocode, cross-defence, joint evaluation

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021 (chapters 7–9: logical agents, first-order logic, inference).

[2] J. A. Robinson, “A machine-oriented logic based on the resolution principle,” *Journal of the ACM*, vol. 12, pp. 23–41, 1965.

[3] W. F. Dowling and J. H. Gallier, “Linear-time algorithms for testing the satisfiability of propositional Horn formulae,” *Journal of Logic Programming*, vol. 1, pp. 267–284, 1984.

[4] A. Haken, “The intractability of resolution,” *Theoretical Computer Science*, vol. 39, pp. 297–308, 1985.

[5] M. Davis, G. Logemann and D. Loveland, “A machine program for theorem proving,” *Communications of the ACM*, vol. 5, pp. 394–397, 1962 (DPLL).

[6] D. S. Warren et al., “Tabling in logic programming” (SLG resolution), *Journal of the ACM*, vol. 43, 1996.

Appendix A – Source Code

The figure script `make_figs3.py`, the raw result files (`t*_results.json`) and these sources are included in the accompanying zip file. All experiments were run with `PYTHONHASHSEED=0`.

A.1 Propositional library: syntax, truth-table checker, CNF, resolution, DPLL (Tasks 1, 2) (logic.py)

```
"""
CSA17 AI Assignment 3 - propositional logic library (Tasks 1 and 2)
Pair report: Student A = Saai Vardhan D (192525205), Student B = partner
Sentences are nested tuples:  ('sym',name) ('not',a) ('and',a,b) ('or',a,b) ('imp',a,b) ('iff',a,b)
"""
import itertools, time


def S(n): return ('sym', n)
def Not(a): return ('not', a)
def And(*xs):
    r = xs[0]
    for x in xs[1:]: r = ('and', r, x)
    return r
def Or(*xs):
    r = xs[0]
    for x in xs[1:]: r = ('or', r, x)
    return r
def Imp(a, b): return ('imp', a, b)
def Iff(a, b): return ('iff', a, b)


def symbols(s, acc=None):
    acc = set() if acc is None else acc
    if s[0] == 'sym': acc.add(s[1])
    else:
        for x in s[1:]: symbols(x, acc)
    return acc


def holds(s, m):
    t = s[0]
    if t == 'sym': return m[s[1]]
    if t == 'not': return not holds(s[1], m)
    if t == 'and': return holds(s[1], m) and holds(s[2], m)
    if t == 'or': return holds(s[1], m) or holds(s[2], m)
    if t == 'imp': return (not holds(s[1], m)) or holds(s[2], m)
    if t == 'iff': return holds(s[1], m) == holds(s[2], m)


def show(s):
    t = s[0]
    if t == 'sym': return s[1]
    if t == 'not': return '~' + show(s[1])
    sym = {'and': ' & ', 'or': ' | ', 'imp': ' => ', 'iff': ' <=> '}[t]
    return '(' + show(s[1]) + sym + show(s[2]) + ')'


# ---------------------------------------------------------------------- truth-table entailment
class MC:
    def __init__(self): self.models = 0; self.kb_evals = 0; self.sentence_evals = 0; self.time = 0.0


def tt_entails(kb, query, stat=None, limit=None):
    """KB |= query ?  Enumerate every model over the symbols of KB and query.  kb = list of sentences (conjunction).
    stat.models = models enumerated; stat.kb_evals = models on which the KB was evaluated; stat.sentence_evals = sentence evaluations
(short-circuit)."""
    stat = stat or MC(); t0 = time.perf_counter()
    syms = sorted(set().union(*[symbols(s) for s in kb], symbols(query)))
    m = len(syms); result = True
    for bits in itertools.product((False, True), repeat=m):
        if limit is not None and stat.models >= limit: stat.time = time.perf_counter() - t0; return None
        model = dict(zip(syms, bits)); stat.models += 1; stat.kb_evals += 1
        ok = True
        for s in kb:
            stat.sentence_evals += 1
            if not holds(s, model): ok = False; break
        if ok and not holds(query, model): result = False; break
    stat.time = time.perf_counter() - t0; stat.nsyms = m
    return result


# ---------------------------------------------------------------------- CNF conversion (each step is a separate function so it can be shown by
hand)
def elim_iff(s):
    t = s[0]
    if t == 'sym': return s
    if t == 'iff':
        a, b = elim_iff(s[1]), elim_iff(s[2]); return ('and', ('imp', a, b), ('imp', b, a))
    return (t,) + tuple(elim_iff(x) for x in s[1:])


def elim_imp(s):
    t = s[0]
    if t == 'sym': return s
    if t == 'imp': return ('or', ('not', elim_imp(s[1])), elim_imp(s[2]))
    return (t,) + tuple(elim_imp(x) for x in s[1:])


def nnf(s):
    t = s[0]
    if t == 'sym': return s
    if t == 'not':
```

```
        a = s[1]
        if a[0] == 'sym': return s
        if a[0] == 'not': return nnf(a[1])
        if a[0] == 'and': return ('or', nnf(('not', a[1])), nnf(('not', a[2])))
        if a[0] == 'or': return ('and', nnf(('not', a[1])), nnf(('not', a[2])))
    return (t,) + tuple(nnf(x) for x in s[1:])


def distribute(s):
    """returns list of clauses (frozenset of literal strings); literal '-x' is the negation of 'x'"""
    t = s[0]
    if t == 'sym': return [frozenset([s[1]])]
    if t == 'not': return [frozenset(['-' + s[1][1]])]
    if t == 'and': return distribute(s[1]) + distribute(s[2])
    if t == 'or':
        A, B = distribute(s[1]), distribute(s[2]); return [a | b for a in A for b in B]


def to_cnf(s): return distribute(nnf(elim_imp(elim_iff(s))))
def neg(l): return l[1:] if l[0] == '-' else '-' + l


def is_taut(c): return any(neg(l) in c for l in c)


# ------------------------------------------------------------------- propositional resolution (saturation with literal index)
class RS:
    def __init__(self): self.generated = 0; self.attempted = 0; self.tautologies = 0; self.subsumed = 0; self.initial = 0; self.rounds = 0;
self.time = 0.0; self.kept = 0; self.proof = None


def resolve(c1, c2, lit):
    return (c1 - {lit}) | (c2 - {neg(lit)})


def pl_resolution(kb, query, stat=None, max_clauses=200000, time_limit=None, subsumption=True, record=False):
    """Refutation: CNF(KB & ~query), saturate with binary resolution, level by level.  Returns True (entailed), False (saturated, not
entailed) or None (limit)."""
    stat = stat or RS(); t0 = time.perf_counter()
    clauses = []
    for s in kb: clauses += to_cnf(s)
    clauses += to_cnf(('not', query))
    seen = set(); work = []
    for c in clauses:
        if is_taut(c): stat.tautologies += 1; continue
        if c not in seen: seen.add(c); work.append(c)
    stat.initial = len(work); parents = {c: None for c in work}
    allc = list(work); index = {}
    for c in allc:
        for l in c: index.setdefault(l, []).append(c)
    frontier = list(allc); alive = set(allc)
    def subsumed_by_existing(c):
        for l in c:
            for d in index.get(l, ()):
                if d in alive and d <= c and d != c: return True
        return False
    while frontier:
        stat.rounds += 1; new = []
        for c1 in frontier:
            for lit in c1:
                for c2 in list(index.get(neg(lit), ())):
                    if c2 not in alive and subsumption: continue
                    stat.attempted += 1
                    r = frozenset(resolve(c1, c2, lit))
                    if is_taut(r): stat.tautologies += 1; continue
                    if r in seen: continue
                    if not r:
                        stat.generated += 1; stat.kept = len(alive); stat.time = time.perf_counter() - t0
                        parents[r] = (c1, c2, lit); stat.proof = (parents, r); return True
                    if subsumption and subsumed_by_existing(r): stat.subsumed += 1; seen.add(r); continue
                    seen.add(r); stat.generated += 1; parents[r] = (c1, c2, lit); new.append(r)
                    if len(seen) > max_clauses or (time_limit and time.perf_counter() - t0 > time_limit):
                        stat.kept = len(alive); stat.time = time.perf_counter() - t0; return None
        if subsumption:
            for r in new:                                   # backward subsumption: delete older clauses that r subsumes
                for l in r:
                    for d in index.get(l, ()):
                        if d in alive and r < d: alive.discard(d); stat.subsumed += 1
        for r in new:
            alive.add(r)
            for l in r: index.setdefault(l, []).append(r)
        frontier = new
    stat.kept = len(alive); stat.time = time.perf_counter() - t0
    return False


def proof_steps(stat):
    parents, goal = stat.proof; order = []; seen = set()
    def walk(c):
        if c in seen: return
        seen.add(c); p = parents[c]
        if p is not None: walk(p[0]); walk(p[1])
        order.append(c)
    walk(goal); return order, parents
def fmt_clause(c): return '{' + ', '.join(sorted(c)) + '}' if c else '{}  (empty clause)'


# ------------------------------------------------------------------- DPLL satisfiability (independent check on large KBs)
def dpll_sat(clauses):
    clauses = [frozenset(c) for c in clauses if not is_taut(c)]
    def go(cls, assign):
        while True:
            unit = next((next(iter(c)) for c in cls if len(c) == 1), None)
            if unit is None: break
            assign = assign | {unit}; new = []
            for c in cls:
                if unit in c: continue
                if neg(unit) in c:
                    c = c - {neg(unit)}
                    if not c: return None
```

```
                new.append(c)
            cls = new
        if not cls: return assign
        lit = next(iter(cls[0]))
        for choice in (lit, neg(lit)):
            r = go([c for c in cls if choice not in c] if False else [ (c - {neg(choice)}) for c in cls if choice not in c ], assign |
{choice})
            if r is not None and all(len(c - {neg(choice)}) > 0 for c in cls if choice not in c): return r
        return None
    return go(clauses, frozenset())
def dpll_entails(kb, q):
    cls = []
    for s in kb: cls += to_cnf(s)
    cls += to_cnf(('not', q)); return dpll_sat(cls) is None
```

A.2 Task 1 – Wumpus-world KB and inference-rule derivation (wumpus.py)

```
"""CSA17 A3 Task 1 - n x n Wumpus world in propositional logic, model checking, inference-rule derivation."""
from logic import *
def nb(n, x, y): return [(a, b) for a, b in ((x-1, y), (x+1, y), (x, y-1), (x, y+1)) if 1 <= a <= n and 1 <= b <= n]
def P(x, y): return S(f'P{x}{y}')
def W(x, y): return S(f'W{x}{y}')
def B(x, y): return S(f'B{x}{y}')
def St(x, y): return S(f'S{x}{y}')
def squares(n): return [(x, y) for y in range(1, n + 1) for x in range(1, n + 1)]


def rules(n):
    kb = {}
    kb['R1 no pit at start'] = Not(P(1, 1)); kb['R2 no wumpus at start'] = Not(W(1, 1))
    for (x, y) in squares(n):
        kb[f'Rb{x}{y} breeze iff pit nearby'] = Iff(B(x, y), Or(*[P(a, b) for a, b in nb(n, x, y)]))
        kb[f'Rs{x}{y} stench iff wumpus nearby'] = Iff(St(x, y), Or(*[W(a, b) for a, b in nb(n, x, y)]))
    kb['Rw1 at least one wumpus'] = Or(*[W(x, y) for x, y in squares(n)])
    sq = squares(n)
    for i in range(len(sq)):
        for j in range(i + 1, len(sq)):
            kb[f'Rw2 at most one wumpus {sq[i]}{sq[j]}'] = Or(Not(W(*sq[i])), Not(W(*sq[j])))
    return kb


def nsymbols(n): return 4 * n * n
def nsentences(n):
    m = n * n
    return 2 + 2 * m + 1 + m * (m - 1) // 2
def percepts(n):
    """percept sequence: (1,1) nothing; (2,1) breeze; (1,2) stench   (n>=3)"""
    if n == 2: return [Not(B(1, 1)), Not(St(1, 1))]
    return [Not(B(1, 1)), Not(St(1, 1)), B(2, 1), Not(St(2, 1)), Not(B(1, 2)), St(1, 2)]
def queries(n):
    if n == 2: return {'entailed: [1,2] has no pit': Not(P(1, 2)), 'negation entailed: pit at [2,1]': P(2, 1), 'neither: pit at [2,2]':
P(2, 2)}
    q = {'entailed: pit at [3,1]': P(3, 1), 'negation entailed: pit at [2,2]': P(2, 2), 'neither: pit at [3,3]' if n >= 3 else '': P(min(3,
n), min(3, n))}
    q['entailed: [2,2] is safe (no pit, no wumpus)'] = And(Not(P(2, 2)), Not(W(2, 2)))
    return q


def relevant_kb(n, perc, query):
    """Relevant-symbol restriction: keep the percept sentences, the breeze/stench rules of the squares that were perceived, R1/R2,
    and the at-most-one-wumpus rules among the relevant wumpus symbols.  (A SUBSET of the KB: sound, possibly incomplete.)"""
    full = rules(n); pers = set()
    for p in perc: pers |= symbols(p)
    perceived = {(int(s[1]), int(s[2])) for s in pers if s[0] in 'BS'}
    kb = [full['R1 no pit at start'], full['R2 no wumpus at start']]
    for (x, y) in sorted(perceived):
        kb.append(full[f'Rb{x}{y} breeze iff pit nearby']); kb.append(full[f'Rs{x}{y} stench iff wumpus nearby'])
    kb += perc
    wsyms = sorted({s for k in kb[:] for s in symbols(k) if s[0] == 'W'} | symbols(query) & {s for s in symbols(query) if s[0] == 'W'})
    for i in range(len(wsyms)):
        for j in range(i + 1, len(wsyms)): kb.append(Or(Not(S(wsyms[i])), Not(S(wsyms[j]))))
    return kb


# ---------------------------------------------------------------- Part B : inference-rule derivation with step counting
def derive(n, perc, goal_lits):
    """Applies Biconditional-Elimination, And-Elimination, Modus Ponens and Unit Resolution to the rules of the perceived squares.
    Every rule application is one step with an id and the ids it depends on.  After the goal literals are derived, steps that
    the goal does not depend on are pruned, so the returned list is the derivation of THIS goal.   Returns (steps, goal_ok)."""
    steps = []                                          # (id, rule, text, deps)
    def step(rule, text, deps): steps.append((len(steps) + 1, rule, text, tuple(deps))); return len(steps)
    units = {}                                          # literal -> step id (0 = given premise)
    for p in perc: units[('-' + p[1][1]) if p[0] == 'not' else p[1]] = 0
    units['-P11'] = 0; units['-W11'] = 0
    full = rules(n); perceived = sorted({(int(k.lstrip('-')[1]), int(k.lstrip('-')[2])) for k in units if k.lstrip('-')[0] in 'BS'})
    clauses = []                                        # [literal set, producing step id, text]
    for (x, y) in perceived:
        for tag, kind in (('b', 'breeze iff pit nearby'), ('s', 'stench iff wumpus nearby')):
            sent = full[f'R{tag}{x}{y} {kind}']; be = step('Biconditional-Elimination', show(sent) + '  gives  ' + show(elim_iff(sent)),
[])
            a = elim_iff(sent)
            for k, imp in enumerate((a[1], a[2])):
                ae = step('And-Elimination', show(a) + '  gives  ' + show(imp), [be])
                cl = to_cnf(imp)
                for j, c in enumerate(cl):
                    sid = ae if len(cl) == 1 else step('And-Elimination', 'split ' + show(imp) + ' (as clauses) gives ' + fmt_clause(c),
[ae])
                    clauses.append([frozenset(c), sid, fmt_clause(c)])
    changed = True
    while changed and not all(g in units for g in goal_lits):
        changed = False
        for cl in clauses:
            lits, sid, _ = cl
            for l in sorted(lits):
                if neg(l) in units and len(lits) > 1:
                    new = lits - {l}
```

```
                    rule = 'Modus Ponens' if l[0] == '-' else 'Unit Resolution'
                    sid = step(rule, f'{fmt_clause(lits)} with {neg(l)}  gives  {fmt_clause(new)}', [sid, units[neg(l)]] if units[neg(l)]
else [sid])
                    cl[0], cl[1] = new, sid; lits = new; changed = True
                    if len(new) == 1:
                        u = next(iter(new))
                        if u not in units: units[u] = sid
            if len(cl[0]) == 1 and next(iter(cl[0])) not in units: units[next(iter(cl[0]))] = cl[1]; changed = True
    ok = all(g in units for g in goal_lits)
    need = set(); todo = [units[g] for g in goal_lits if g in units]
    while todo:
        t = todo.pop()
        if t in need or t == 0: continue
        need.add(t); todo += list(steps[t - 1][3])
    pruned = [st for st in steps if st[0] in need]
    return pruned, ok
```

A.3 Task 2 – the 12-rule access-control policy and the six cases (policy.py)

```
"""Smart-building access-control policy (Task 2): 12 rules with implication, biconditional and disjunction."""
from logic import *
def sy(n): return S(n)
RULES = {
 'R1  a badge is valid exactly when it is not expired':                    Iff(sy('BadgeValid'), Not(sy('BadgeExpired'))),
 'R2  every person is an employee, a visitor or a contractor':             Or(sy('Employee'), sy('Visitor'), sy('Contractor')),
 'R3  a visitor is not an employee':                                       Imp(sy('Visitor'), Not(sy('Employee'))),
 'R4  work hours exactly when it is not after hours':                      Iff(sy('WorkHours'), Not(sy('AfterHours'))),
 'R5  employee with valid badge in work hours may enter':                  Imp(And(sy('Employee'), sy('BadgeValid'), sy('WorkHours')),
sy('MayEnter')),
 'R6  escorted visitor with valid badge in work hours may enter':          Imp(And(sy('Visitor'), sy('Escorted'), sy('BadgeValid'),
sy('WorkHours')), sy('MayEnter')),
 'R7  after hours, no emergency, not security staff: may not enter':       Imp(And(sy('AfterHours'), Not(sy('Emergency')),
Not(sy('Security'))), Not(sy('MayEnter'))),
 'R8  emergency override: everybody may enter':                            Imp(sy('Emergency'), sy('MayEnter')),
 'R9  the door is unlocked exactly when entry is allowed':                 Iff(sy('MayEnter'), sy('DoorUnlocked')),
 'R10 an emergency activates the alarm':                                   Imp(sy('Emergency'), sy('Alarm')),
 'R11 server-room access needs entry permission and employee status':      Imp(sy('ServerRoom'), And(sy('MayEnter'), sy('Employee'))),
 'R12 a contractor needs an escort or may not enter':                      Imp(sy('Contractor'), Or(sy('Escorted'),
Not(sy('MayEnter')))),
}
CASES = {
 'C1 employee, valid badge, work hours  |=  DoorUnlocked':     ([sy('Employee'), Not(sy('BadgeExpired')), sy('WorkHours')],
sy('DoorUnlocked')),
 'C2 visitor, unescorted, work hours    |=  MayEnter ?':        ([sy('Visitor'), Not(sy('Escorted')), Not(sy('BadgeExpired')),
sy('WorkHours')], sy('MayEnter')),
 'C3 after hours, no emergency, not security |= ~DoorUnlocked': ([sy('AfterHours'), Not(sy('Emergency')), Not(sy('Security'))],
Not(sy('DoorUnlocked'))),
 'C4 emergency |= DoorUnlocked & Alarm':                          ([sy('Emergency')], And(sy('DoorUnlocked'), sy('Alarm'))),
 'C5 server-room request by a visitor |= false (inconsistent)':   ([sy('ServerRoom'), sy('Visitor')], Not(sy('ServerRoom'))),
 'C6 contractor, unescorted, work hours |= ~MayEnter':            ([sy('Contractor'), Not(sy('Escorted')), sy('WorkHours')],
Not(sy('MayEnter'))),
}
```

A.4 First-order engine: unification, backward and forward chaining, propositionalisation (Tasks 3, 4, 6, 7) (fol.py)

```
"""
CSA17 AI Assignment 3 - first-order logic engine (Tasks 3-7)
Terms : variable '?x' (string starting with '?'), constant (any other str/int), compound ('f', t1, ..., tn)
Atoms : ('Pred', t1, ..., tn).  Rule = (head, [body literals]); a body literal is an atom or ('not', atom) (negation as failure).
"""
import itertools, time, sys, itertools
sys.setrecursionlimit(20000)


def is_var(t): return isinstance(t, str) and t.startswith('?')
def is_compound(t): return isinstance(t, tuple)


# ---------------------------------------------------------------- substitution and unification
def walk(t, th):
    while is_var(t) and t in th: t = th[t]
    return t
def subst(t, th):
    t = walk(t, th)
    if is_compound(t): return (t[0],) + tuple(subst(a, th) for a in t[1:])
    return t
def occurs(v, t, th):
    t = walk(t, th)
    if t == v: return True
    if is_compound(t): return any(occurs(v, a, th) for a in t[1:])
    return False
class UStat:
    def __init__(self): self.calls = 0; self.bindings = 0; self.occurs_checks = 0
def unify(x, y, th, occurs_check=True, stat=None):
    """returns an (extended) substitution dict or None.  th is not modified."""
    if stat is not None: stat.calls += 1
    return _unify(x, y, dict(th), occurs_check, stat)
def _unify(x, y, th, oc, stat):
    if th is None: return None
    x = walk(x, th); y = walk(y, th)
    if x == y: return th
    if is_var(x): return _bind(x, y, th, oc, stat)
    if is_var(y): return _bind(y, x, th, oc, stat)
    if is_compound(x) and is_compound(y):
        if x[0] != y[0] or len(x) != len(y): return None
        for a, b in zip(x[1:], y[1:]):
            th = _unify(a, b, th, oc, stat)
            if th is None: return None
        return th
    return None
```

```
def _bind(v, t, th, oc, stat):
    if oc:
        if stat is not None: stat.occurs_checks += 1
        if occurs(v, t, th): return None
    th[v] = t
    if stat is not None: stat.bindings += 1
    return th
def compose(s1, s2):
    """s1 then s2:  apply(apply(t, s1), s2) == apply(t, compose(s1, s2))"""
    r = {v: subst(t, s2) for v, t in s1.items()}
    for v, t in s2.items():
        if v not in r: r[v] = t
    return {v: t for v, t in r.items() if v != t}
def mgu_full(x, y, occurs_check=True):
    th = unify(x, y, {}, occurs_check)
    if th is None: return None
    try: return {v: subst(v, th) for v in th}
    except RecursionError: return 'CYCLIC'                    # fully resolved (idempotent) form
def tstr(t):
    if is_compound(t): return t[0] + '(' + ', '.join(tstr(a) for a in t[1:]) + ')'
    return str(t)
def sstr(th):
    if th == 'CYCLIC': return 'CYCLIC binding (infinite term, e.g. ?x = f(f(f(...))))'
    return '{' + ', '.join(f'{v}/{tstr(t)}' for v, t in sorted(th.items())) + '}' if th else '{}  (empty)'
def variables(t, acc=None):
    acc = [] if acc is None else acc
    if is_var(t):
        if t not in acc: acc.append(t)
    elif is_compound(t):
        for a in t[1:]: variables(a, acc)
    return acc
_cnt = itertools.count(1)
def rename_rule(rule):
    """standardise apart: fresh variable names for a rule (head, body)"""
    head, body = rule; m = {}
    def ren(t):
        if is_var(t):
            if t not in m: m[t] = f'{t}_{next(_cnt)}'
            return m[t]
        if is_compound(t): return (t[0],) + tuple(ren(a) for a in t[1:])
        return t
    return ren(head), [(ren(b) if b[0] != 'not' else ('not', ren(b[1]))) for b in body]


# ---------------------------------------------------------------- knowledge base with indexes
class KB:
    def __init__(self, rules=(), facts=(), index2=False):
        self.index2 = index2; self.facts_by_key2 = {}
        self.rules = []; self.rules_by_pred = {}; self.facts = set(); self.facts_by_pred = {}; self.facts_by_key = {}
        for r in rules: self.add_rule(r)
        for f in facts: self.add_fact(f)
    def add_rule(self, r):
        self.rules.append(r); self.rules_by_pred.setdefault(r[0][0], []).append(r)
    def add_fact(self, f):
        if f in self.facts: return False
        self.facts.add(f); self.facts_by_pred.setdefault(f[0], []).append(f)
        if len(f) > 1: self.facts_by_key.setdefault((f[0], f[1]), []).append(f)
        if self.index2 and len(f) > 2: self.facts_by_key2.setdefault((f[0], f[2]), []).append(f)
        return True
    def candidates(self, goal):
        """index on predicate and, when bound, first argument"""
        if len(goal) > 1 and not is_var(goal[1]) and not is_compound(goal[1]): return self.facts_by_key.get((goal[0], goal[1]), [])
        if self.index2 and len(goal) > 2 and not is_var(goal[2]) and not is_compound(goal[2]): return self.facts_by_key2.get((goal[0],
goal[2]), [])
        return self.facts_by_pred.get(goal[0], [])


# ---------------------------------------------------------------- backward chaining with substitutions
class BCStat:
    def __init__(self): self.goals = 0; self.unifications = 0; self.rule_tries = 0; self.max_depth = 0; self.repeated = 0; self.solutions =
0; self.facts_matched = 0; self.cut = 0; self.time = 0.0; self.exhausted = False


def variant_key(goal, th):
    g = subst(goal, th); m = {}
    def f(t):
        if is_var(t): return m.setdefault(t, f'_{len(m)}')
        if is_compound(t): return (t[0],) + tuple(f(a) for a in t[1:])
        return t
    return f(g)


class Prover:
    """Backward chaining (SLD resolution with negation as failure on ground literals).
       mode: 'plain' | 'loopcheck' | 'depth' (iterative/limit) ; goal-count budget stops runaway searches."""
    def __init__(self, kb, mode='plain', depth_limit=None, budget=2_000_000, occurs_check=False, index=True, trace=False):
        self.kb = kb; self.mode = mode; self.depth_limit = depth_limit; self.budget = budget; self.oc = occurs_check
        self.index = index; self.stat = BCStat(); self.seen_goals = {}; self.trace = trace
    def prove_all(self, goals, th, depth, ancestors):
        if not goals: yield th, []; return
        first, rest = goals[0], goals[1:]
        for th1, p1 in self.prove(first, th, depth, ancestors):
            for th2, p2 in self.prove_all(rest, th1, depth, ancestors): yield th2, [p1] + p2
    def prove(self, goal, th, depth, ancestors):
        st = self.stat
        if st.goals >= self.budget: st.exhausted = True; return
        if goal[0] == 'not':
            g = subst(goal[1], th)
            if any(is_var(a) for a in g[1:]): raise ValueError('floundering: non-ground negative literal ' + str(g))
            sub = Prover(self.kb, self.mode, self.depth_limit, 200000, self.oc, self.index)
            ok = any(True for _ in sub.prove(g, {}, 0, ()))
            st.goals += sub.stat.goals; st.unifications += sub.stat.unifications
            if not ok: yield th, ('naf', g)
            return
        if goal[0] == 'Ge':                                   # built-in arithmetic comparison (both arguments must be numbers)
            a, b = subst(goal[1], th), subst(goal[2], th)
            if isinstance(a, (int, float)) and isinstance(b, (int, float)):
                st.goals += 1
                if a >= b: yield th, ('fact', ('Ge', a, b))
                return
```

```
            raise ValueError('floundering: Ge needs bound numbers: ' + str((a, b)))
        st.goals += 1; st.max_depth = max(st.max_depth, depth)
        key = variant_key(goal, th)
        if key in self.seen_goals: st.repeated += 1
        self.seen_goals[key] = self.seen_goals.get(key, 0) + 1
        if self.depth_limit is not None and depth >= self.depth_limit: st.cut += 1; return
        if self.mode == 'loopcheck' and key in ancestors: st.cut += 1; return
        anc = ancestors | {key} if self.mode == 'loopcheck' else ancestors
        g = subst(goal, th)
        facts = self.kb.candidates(g) if self.index else self.kb.facts_by_pred.get(g[0], [])
        for f in facts:
            st.unifications += 1
            th1 = unify(g, f, th, self.oc)
            if th1 is not None: st.facts_matched += 1; yield th1, ('fact', f)
        for rule in (self.kb.rules_by_pred.get(g[0], []) if self.index else [r for r in self.kb.rules if r[0][0] == g[0]]):
            st.rule_tries += 1; head, body = rename_rule(rule); st.unifications += 1
            th1 = unify(g, head, th, self.oc)
            if th1 is None: continue
            for th2, subs in self.prove_all(body, th1, depth + 1, anc): yield th2, ('rule', rule, subs)
    def ask(self, goal, limit=None):
        t0 = time.perf_counter(); out = []; seen = set()
        for th, proof in self.prove(goal, {}, 0, frozenset()):
            ans = subst(goal, th)
            if ans not in seen: seen.add(ans); out.append((ans, proof))
            if limit and len(out) >= limit: break
        self.stat.time = time.perf_counter() - t0; self.stat.solutions = len(out); return out


def proof_size(p):
    if p is None: return 0
    if p[0] in ('fact', 'naf'): return 1
    return 1 + sum(proof_size(x) for x in p[2])
def proof_lines(p, indent=0, out=None):
    out = [] if out is None else out
    if p[0] == 'fact': out.append('  ' * indent + 'fact   ' + tstr(p[1]))
    elif p[0] == 'naf': out.append('  ' * indent + 'not     ' + tstr(p[1]) + '  (no proof found: negation as failure)')
    else:
        h, b = p[1]; out.append('  ' * indent + 'rule   ' + tstr(h) + ' :- ' + ', '.join(tstr(x) if x[0] != 'not' else 'not ' + tstr(x[1])
for x in b))
        for s in p[2]: proof_lines(s, indent + 1, out)
    return out


# ---------------------------------------------------------------- forward chaining
class FCStat:
    def __init__(self): self.iterations = 0; self.firings = 0; self.new_facts = 0; self.redundant = 0; self.unifications = 0; self.time =
0.0; self.initial = 0
def match_body(kb, body, th, stat, delta=None, used_delta=False):
    if not body:
        if delta is None or used_delta: yield th
        return
    lit, rest = body[0], body[1:]
    if lit[0] == 'not':
        g = subst(lit[1], th)
        if g not in kb.facts: yield from match_body(kb, rest, th, stat, delta, used_delta)
        return
    g = subst(lit, th)
    if g[0] == 'Ge':                                            # built-in comparison, both arguments must be bound numbers
        if isinstance(g[1], (int, float)) and isinstance(g[2], (int, float)) and g[1] >= g[2]:
            yield from match_body(kb, rest, th, stat, delta, used_delta)
        return
    for f in kb.candidates(g):
        stat.unifications += 1
        th1 = unify(g, f, th, False)
        if th1 is not None: yield from match_body(kb, rest, th1, stat, delta, used_delta or (delta is not None and f in delta))
def forward_chain(kb, seminaive=True, max_iter=100000):
    """Applies all rules until no new fact is produced.  Facts are added to kb.  semi-naive: a rule instance must use a fact derived in the
previous iteration."""
    st = FCStat(); t0 = time.perf_counter(); st.initial = len(kb.facts); delta = None
    while st.iterations < max_iter:
        st.iterations += 1; new = []
        for rule in kb.rules:
            head, body = rule
            for th in match_body(kb, body, {}, st, delta if seminaive else None):
                st.firings += 1; h = subst(head, th)
                if h in kb.facts or h in new: st.redundant += 1
                else: new.append(h)
        if not new: break
        for f in new: kb.add_fact(f)
        st.new_facts += len(new); delta = set(new)
    st.time = time.perf_counter() - t0; return st


# ---------------------------------------------------------------- propositionalisation (instantiate every rule with all ground terms)
def ground_terms(constants, functions, depth):
    """functions = {name: arity}.  returns list of ground terms of nesting depth <= depth"""
    terms = list(constants); level = list(constants)
    for _ in range(depth):
        new = []
        for fname, ar in functions.items():
            for args in itertools.product(terms, repeat=ar):
                t = (fname,) + args
                if t not in terms and t not in new: new.append(t)
        terms += new
    return terms
def propositionalise(rules, terms, facts=()):
    """ground instances of every rule (variables replaced by all combinations of terms).  returns list of (head, body) ground clauses"""
    out = []
    for head, body in rules:
        vs = []
        for a in [head] + [b if b[0] != 'not' else b[1] for b in body]: variables(a, vs)
        for combo in itertools.product(terms, repeat=len(vs)):
            th = dict(zip(vs, combo)); out.append((subst(head, th), [subst(b, th) if b[0] != 'not' else ('not', subst(b[1], th)) for b in
body]))
    return out
```

A.5 Task 5 – first-order resolution prover with strategies (fol_res.py)

```
"""CSA17 A3 Task 5 - first-order resolution: CNF conversion with Skolemisation, and a given-clause resolution prover using the Task 4
unifier"""
import itertools, time
from fol import *
CONN = {'forall', 'exists', 'and', 'or', 'imp', 'iff', 'not'}
def Forall(v, f): return ('forall', v, f)
def Exists(v, f): return ('exists', v, f)
def fstr(f):
    t = f[0]
    if t in ('forall', 'exists'): return ('A' if t == 'forall' else 'E') + f[1][1:] + ' ' + fstr(f[2])
    if t == 'not': return '~' + fstr(f[1])
    if t in ('and', 'or', 'imp', 'iff'): return '(' + fstr(f[1]) + {'and': ' & ', 'or': ' | ', 'imp': ' => ', 'iff': ' <=> '}[t] +
fstr(f[2]) + ')'
    return tstr(f)
def elim(f):
    t = f[0]
    if t in ('forall', 'exists'): return (t, f[1], elim(f[2]))
    if t == 'not': return ('not', elim(f[1]))
    if t == 'imp': return ('or', ('not', elim(f[1])), elim(f[2]))
    if t == 'iff': a, b = elim(f[1]), elim(f[2]); return ('and', ('or', ('not', a), b), ('or', ('not', b), a))
    if t in ('and', 'or'): return (t, elim(f[1]), elim(f[2]))
    return f
def nnf1(f, neg=False):
    t = f[0]
    if t == 'not': return nnf1(f[1], not neg)
    if t == 'and': return ('or' if neg else 'and', nnf1(f[1], neg), nnf1(f[2], neg))
    if t == 'or': return ('and' if neg else 'or', nnf1(f[1], neg), nnf1(f[2], neg))
    if t == 'forall': return ('exists' if neg else 'forall', f[1], nnf1(f[2], neg))
    if t == 'exists': return ('forall' if neg else 'exists', f[1], nnf1(f[2], neg))
    return ('not', f) if neg else f
_n = itertools.count(1)
def rename_vars(f, m=None):
    m = m or {}; t = f[0]
    if t in ('forall', 'exists'):
        nv = f'{f[1]}{next(_n)}'; m2 = dict(m); m2[f[1]] = nv; return (t, nv, rename_vars(f[2], m2))
    if t == 'not': return ('not', rename_vars(f[1], m))
    if t in ('and', 'or'): return (t, rename_vars(f[1], m), rename_vars(f[2], m))
    def r(x):
        if is_var(x): return m.get(x, x)
        if is_compound(x): return (x[0],) + tuple(r(a) for a in x[1:])
        return x
    return r(f)
_sk = itertools.count(1)
def skolemise(f, univ=(), log=None):
    t = f[0]
    if t == 'forall': return ('forall', f[1], skolemise(f[2], univ + (f[1],), log))
    if t == 'exists':
        name = f'Sk{next(_sk)}'; term = name if not univ else (name,) + tuple(univ)
        if log is not None: log.append(f'{f[1]} -> {tstr(term)}' + ('   (a constant: the existential is not inside any universal)' if not
univ else f'   (a function of the universal variables {", ".join(univ)} in whose scope it occurs)'))
        return skolemise(substf(f[2], f[1], term), univ, log)
    if t in ('and', 'or'): return (t, skolemise(f[1], univ, log), skolemise(f[2], univ, log))
    return f
def substf(f, v, term):
    t = f[0]
    if t in ('forall', 'exists'): return (t, f[1], substf(f[2], v, term))
    if t == 'not': return ('not', substf(f[1], v, term))
    if t in ('and', 'or'): return (t, substf(f[1], v, term), substf(f[2], v, term))
    return subst(f, {v: term})
def drop_forall(f):
    return drop_forall(f[2]) if f[0] == 'forall' else ((f[0],) + tuple(drop_forall(x) for x in f[1:]) if f[0] in ('and', 'or') else
(('not', drop_forall(f[1])) if f[0] == 'not' else f))
def dist(f):
    if f[0] == 'and': return dist(f[1]) + dist(f[2])
    if f[0] == 'or': return [a + b for a in dist(f[1]) for b in dist(f[2])]
    if f[0] == 'not': return [((False, f[1]),)]
    return [((True, f),)]
def to_clauses(f, verbose=False, label=''):
    steps = []; a = elim(f); steps.append(('1. eliminate => and <=>', fstr(a)))
    b = nnf1(a); steps.append(('2. move ~ inwards (quantifier duality)', fstr(b)))
    c = rename_vars(b); steps.append(('3. standardise variables apart', fstr(c)))
    log = []; d = skolemise(c, (), log); steps.append(('4. Skolemise (remove existential quantifiers)', fstr(d) + ('      Skolem: ' + ';
'.join(log) if log else '')))
    e = drop_forall(d); steps.append(('5. drop universal quantifiers', fstr(e)))
    cl = [dedupe(c) for c in dist(e)]; steps.append(('6. distribute | over & : clauses', '   '.join(cstr(c) for c in cl)))
    if verbose:
        print(f'  {label}: {fstr(f)}')
        for s, t in steps: print(f'      {s:44s} {t}')
    return cl
def dedupe(c):
    out = []
    for l in c:
        if l not in out: out.append(l)
    return tuple(out)
def lstr(l): return ('' if l[0] else '~') + tstr(l[1])
def cstr(c): return '[' + ' | '.join(lstr(l) for l in c) + ']' if c else '[]'
def is_tautology(c): return any(l1[0] != l2[0] and l1[1] == l2[1] for l1 in c for l2 in c)


# ------------------------------------------------------------------- given-clause resolution prover
class RStat:
    def __init__(self): self.generated = 0; self.steps = 0; self.unifications = 0; self.tautologies = 0; self.duplicates = 0; self.given =
0; self.time = 0.0; self.proof_len = None; self.result = None; self.max_clause_size = 0; self.subsumed = 0


def canon(c):
    m = {}
    def f(t):
        if is_var(t): return m.setdefault(t, f'_{len(m)}')
        if is_compound(t): return (t[0],) + tuple(f(a) for a in t[1:])
        return t
    items = sorted(c, key=lambda l: (l[0], l[1][0], len(l[1])))
    return tuple((s, f(a)) for s, a in items)
def fresh(c):
    m = {}
    def f(t):
```

```
        if is_var(t): return m.setdefault(t, f'?v{next(_n)}')
        if is_compound(t): return (t[0],) + tuple(f(a) for a in t[1:])
        return t
    return tuple((s, f(a)) for s, a in c)
def apply_th(c, th): return tuple((s, subst(a, th)) for s, a in c)


def prove_resolution(*a, **k):
    try: return _prove(*a, **k)
    except RecursionError:
        st = RStat(); st.result = 'DIVERGED (term nesting exceeded the recursion limit)'; return None, st, {}


def _prove(axioms, negated_goal, strategy='plain', max_steps=20000, max_clauses=20000, time_limit=60, occurs_check=True):
    """axioms, negated_goal: lists of clauses.  strategy 'plain' (FIFO, breadth first) | 'unit' (smallest clause first) | 'sos' (set of
support: one parent descends from the negated goal)"""
    st = RStat(); t0 = time.perf_counter(); ustat = UStat()
    info = {}                                       # clause -> (parents, (index of literals)) for the proof
    def register(c, parents, sos):
        c = fresh(c); k = canon(c)
        if k in seen: st.duplicates += 1; return None
        seen.add(k); info[c] = (parents, sos); return c
    seen = set(); active = []; passive = []
    for c in axioms:
        if is_tautology(c): continue
        c2 = register(c, None, False)
        if c2 is not None: (active if strategy == 'sos' else passive).append(c2)
    for c in negated_goal:
        c2 = register(c, None, True)
        if c2 is not None: passive.append(c2)
    age = {c: i for i, c in enumerate(list(info))}; cnt = len(age)
    while passive:
        if strategy == 'unit' or (strategy == 'unitfair' and st.given % 5 != 4): g = min(passive, key=lambda c: (len(c), age[c]))
        else: g = passive[0]
        passive.remove(g); st.given += 1
        if st.steps >= max_steps or len(info) >= max_clauses or time.perf_counter() - t0 > time_limit:
            st.result = 'CAP'; st.time = time.perf_counter() - t0; st.unifications = ustat.calls; st.generated = len(info) - len(axioms) -
len(negated_goal); return None, st, info
        for h0 in active + [g]:
            if strategy == 'sos' and not (info[g][1] or info[h0][1]): continue
            h = fresh(h0) if h0 is g else h0                # a clause resolved with itself needs a renamed copy (standardising apart)
            for i, (s1, a1) in enumerate(g):
                for j, (s2, a2) in enumerate(h):
                    if s1 == s2: continue
                    th = unify(a1, a2, {}, occurs_check, ustat)
                    if th is None: continue
                    st.steps += 1
                    r = tuple(l for k_, l in enumerate(g) if k_ != i) + tuple(l for k_, l in enumerate(h) if k_ != j)
                    r = dedupe(apply_th(r, th))
                    if is_tautology(r): st.tautologies += 1; continue
                    sos = info[g][1] or info[h0][1]
                    if not r:
                        info[r] = ((g, h0), sos); st.generated = len(info) - len(axioms) - len(negated_goal); st.time = time.perf_counter()
- t0; st.unifications = ustat.calls; st.result = 'PROVED'
                        st.proof_len = len(proof_dag(r, info)); return r, st, info
                    r2 = register(r, (g, h0), sos)
                    if r2 is not None:
                        age[r2] = cnt; cnt += 1; passive.append(r2); st.max_clause_size = max(st.max_clause_size, len(r2))
        active.append(g)
    st.result = 'SATURATED (no proof exists: the clause set is satisfiable)'; st.time = time.perf_counter() - t0; st.unifications =
ustat.calls; st.generated = len(info) - len(axioms) - len(negated_goal)
    return None, st, info


def proof_dag(goal, info):
    order = []; seen = set()
    def walk(c):
        if c in seen: return
        seen.add(c); p = info[c][0]
        if p is not None: walk(p[0]); walk(p[1])
        order.append(c)
    walk(goal); return order
def proof_text(goal, info):
    order = proof_dag(goal, info); idx = {c: i + 1 for i, c in enumerate(order)}; lines = []
    for c in order:
        p = info[c][0]; lines.append(f"  {idx[c]:>2}. {cstr(c):52s}" + ('given' if p is None else f'resolve {idx[p[0]]}, {idx[p[1]]}'))
    return lines
```

A.6 Task 3 – exam-eligibility KB, drafts, query suite (exam_kb.py)

```
"""Task 3 - University exam-eligibility advisor: instance, draft and final first-order KBs, propositional encoding"""
import itertools
from fol import *
v = lambda n: '?' + n
Req = lambda c, p: ('Requires', c, p)
INSTANCE = [
 *[('Student', s) for s in ('alice', 'bob', 'carol', 'dave', 'erin', 'frank', 'gina', 'hari', 'ida')],
 *[('Course', c) for c in ('CS101', 'CS201', 'CS301', 'MA101')],
 Req('CS201', 'CS101'), Req('CS301', 'CS201'),
 ('MinAtt', 'CS101', 75), ('MinAtt', 'CS201', 75), ('MinAtt', 'CS301', 75), ('MinAtt', 'MA101', 75),
 ('Enrolled', 'alice', 'CS201'), ('Grade', 'alice', 'CS101', 70), ('Attendance', 'alice', 'CS201', 82),
 ('Enrolled', 'bob', 'CS201'), ('Grade', 'bob', 'CS101', 55), ('Attendance', 'bob', 'CS201', 60),
 ('Enrolled', 'carol', 'CS201'), ('Grade', 'carol', 'CS101', 66), ('Attendance', 'carol', 'CS201', 60), ('ApprovedLeave', 'carol',
'CS201'),
 ('Enrolled', 'dave', 'CS301'), ('Grade', 'dave', 'CS201', 45), ('Grade', 'dave', 'CS101', 30), ('Attendance', 'dave', 'CS301', 90),
 ('Enrolled', 'erin', 'MA101'), ('Attendance', 'erin', 'MA101', 88), ('FeeDue', 'erin'),
 ('Enrolled', 'frank', 'MA101'), ('Attendance', 'frank', 'MA101', 91), ('DisciplinaryCase', 'frank', 'k1'), ('Active', 'k1'),
 ('Enrolled', 'gina', 'MA101'), ('Attendance', 'gina', 'MA101', 85), ('DisciplinaryCase', 'gina', 'k2'),
 ('Enrolled', 'hari', 'CS101'), ('Attendance', 'hari', 'CS101', 90),
]
def R(h, *b): return (h, list(b))
# ---------------------------------------------------------------------- DRAFT 1 (first attempt)
DRAFT1 = [
 R(('AttendanceOK', v('s'), v('c')), ('Enrolled', v('s'), v('c')), ('Attendance', v('s'), v('c'), v('p')), ('MinAtt', v('c'), v('m')),
('Ge', v('p'), v('m'))),
 R(('AttendanceOK', v('s'), v('c')), ('Enrolled', v('s'), v('c')), ('ApprovedLeave', v('s'), v('c'))),
```

```
 R(('Passed', v('s'), v('c')), ('Grade', v('s'), v('c'), v('g')), ('Ge', v('g'), 40)),
 R(('MissingPrereq', v('s'), v('c')), ('Enrolled', v('s'), v('c')), Req(v('c'), v('p')), ('not', ('Passed', v('s'), v('p')))),
 R(('PrereqsCleared', v('s'), v('c')), ('Enrolled', v('s'), v('c')), ('not', ('MissingPrereq', v('s'), v('c')))),
 R(('FeesCleared', v('s')), ('not', ('FeeDue', v('s'))), ('Student', v('s'))),
 R(('Barred', v('s')), ('DisciplinaryCase', v('s'), v('k'))),
 R(('Eligible', v('s'), v('c')), ('AttendanceOK', v('s'), v('c')), ('PrereqsCleared', v('s'), v('c')), ('FeesCleared', v('s')), ('not',
('Barred', v('s')))),
]
# ------------------------------------------------------------------ FINAL (after the three fixes found by the query suite)
FINAL = [
 DRAFT1[0], DRAFT1[1], DRAFT1[2],
 R(('Needs', v('c'), v('p')), Req(v('c'), v('p'))),                                                    # fix 2: transitive prerequisites
 R(('Needs', v('c'), v('p')), Req(v('c'), v('q')), ('Needs', v('q'), v('p'))),
 R(('MissingPrereq', v('s'), v('c')), ('Enrolled', v('s'), v('c')), ('Needs', v('c'), v('p')), ('not', ('Passed', v('s'), v('p')))),
 DRAFT1[4],
 R(('FeesCleared', v('s')), ('Student', v('s')), ('not', ('FeeDue', v('s')))),                         # fix 3: bind s before the negative
literal
 R(('Barred', v('s')), ('DisciplinaryCase', v('s'), v('k')), ('Active', v('k'))),                      # fix 1: only an ACTIVE case bars
 DRAFT1[7],
]
# the 14 axioms of the full first-order theory (those marked * are executed as definite clauses with negation as failure; 10 and 14 contain
an existential)
AXIOMS = [
 "1*  Ax,c,p,m  Enrolled(s,c) & Attendance(s,c,p) & MinAtt(c,m) & Ge(p,m)  =>  AttendanceOK(s,c)",
 "2*  Ax,c  Enrolled(s,c) & ApprovedLeave(s,c)  =>  AttendanceOK(s,c)",
 "3*  Ax,c,g  Grade(s,c,g) & Ge(g,40)  =>  Passed(s,c)",
 "4*  Ac,p  Requires(c,p)  =>  Needs(c,p)",
 "5*  Ac,p,q  Requires(c,q) & Needs(q,p)  =>  Needs(c,p)",
 "6*  As,c  Enrolled(s,c) & Ep (Needs(c,p) & ~Passed(s,p))  =>  MissingPrereq(s,c)",
 "7*  As,c  Enrolled(s,c) & ~MissingPrereq(s,c)  =>  PrereqsCleared(s,c)",
 "8*  As  Student(s) & ~FeeDue(s)  =>  FeesCleared(s)",
 "9*  As  Ek (DisciplinaryCase(s,k) & Active(k))  =>  Barred(s)",
 "10  As  Barred(s)  =>  Ek (DisciplinaryCase(s,k) & Active(k))                    [existential: the 'only if' half of rule 9]",
 "11* As,c  AttendanceOK(s,c) & PrereqsCleared(s,c) & FeesCleared(s) & ~Barred(s)  =>  Eligible(s,c)",
 "12  As,c  Eligible(s,c)  =>  Enrolled(s,c)                                      [typing / sanity axiom]",
 "13  As,c  Passed(s,c)  =>  Ee Grade(s,c,e)                                      [existential: a pass has a recorded grade]",
 "14  Ac  Course(c)  =>  Ef Coordinator(c,f)                                      [existential: every course has a coordinator]",
]
# ------------------------------------------------------------------ the competency query suite: (label, goal, expected)   expected: True /
False / set of binding tuples
QUERIES = [
 ('S1 Eligible(alice, CS201)', ('Eligible', 'alice', 'CS201'), True), ('S2 Eligible(carol, CS201)  [approved leave]', ('Eligible', 'carol',
'CS201'), True),
 ('S3 Eligible(hari, CS101)', ('Eligible', 'hari', 'CS101'), True), ('S4 Eligible(gina, MA101)  [closed case]', ('Eligible', 'gina',
'MA101'), True),
 ('S5 Passed(alice, CS101)', ('Passed', 'alice', 'CS101'), True),
 ('F1 Eligible(bob, CS201)  [attendance]', ('Eligible', 'bob', 'CS201'), False), ('F2 Eligible(dave, CS301)  [indirect prerequisite]',
('Eligible', 'dave', 'CS301'), False),
 ('F3 Eligible(erin, MA101)  [fees due]', ('Eligible', 'erin', 'MA101'), False), ('F4 Eligible(frank, MA101)  [active case]', ('Eligible',
'frank', 'MA101'), False),
 ('F5 Eligible(ida, CS101)  [not enrolled]', ('Eligible', 'ida', 'CS101'), False),
 ('B1 Eligible(?s, CS201)', ('Eligible', '?s', 'CS201'), {('alice',), ('carol',)}), ('B2 Needs(CS301, ?p)', ('Needs', 'CS301', '?p'),
{('CS201',), ('CS101',)}),
 ('B3 Barred(?s)', ('Barred', '?s'), {('frank',)}), ('B4 FeesCleared(?s)', ('FeesCleared', '?s'), {(x,) for x in ('alice', 'bob', 'carol',
'dave', 'frank', 'gina', 'hari', 'ida')}),
 ('B5 Eligible(?s, MA101)', ('Eligible', '?s', 'MA101'), {('gina',)}),
]
def run_suite(rules):
    kb = KB(rules, INSTANCE); rows = []
    for lab, goal, exp in QUERIES:
        p = Prover(kb)
        try:
            ans = p.ask(goal); vars_ = variables(goal)
            if isinstance(exp, bool): got = bool(ans)
            else: got = {tuple(subst(v_, {}) if False else a[1:][goal[1:].index(v_)] for v_ in vars_) for a, _ in ans}
            rows.append((lab, exp, got, got == exp, p.stat.goals, p.stat.unifications, None))
        except ValueError as e:
            rows.append((lab, exp, 'ERROR', False, p.stat.goals, p.stat.unifications, str(e)))
    return rows
```

A.7 Task 6 – organisation hierarchy KB (org_kb.py)

```
"""Task 6 - organisation hierarchy knowledge base (10 definite clauses) and fact generators"""
from fol import *
v = lambda n: '?' + n
def R(h, *b): return (h, list(b))
RULES = [
 R(('Above', v('x'), v('y')), ('ReportsTo', v('y'), v('x'))),                                          # r1
 R(('Above', v('x'), v('y')), ('ReportsTo', v('y'), v('z')), ('Above', v('x'), v('z'))),               # r2 (right-recursive: the recursive
call is last)
 R(('Peer', v('x'), v('y')), ('ReportsTo', v('x'), v('m')), ('ReportsTo', v('y'), v('m'))),           # r3
 R(('SameDept', v('x'), v('y')), ('DeptMember', v('d'), v('x')), ('DeptMember', v('d'), v('y'))),     # r4
 R(('CanApprove', v('x'), v('y')), ('Above', v('x'), v('y')), ('Senior', v('x'))),                     # r5  approval rule
 R(('Escalation', v('x'), v('m')), ('ReportsTo', v('x'), v('m'))),                                     # r6
 R(('SkipLevel', v('x'), v('g')), ('Escalation', v('x'), v('m')), ('Escalation', v('m'), v('g'))),    # r7
 R(('TeamMember', v('x'), v('m')), ('Above', v('m'), v('x')), ('Lead', v('m'))),                       # r8
 R(('Collab', v('x'), v('y')), ('SameDept', v('x'), v('y')), ('Peer', v('x'), v('y'))),               # r9
 R(('Reviewer', v('x'), v('y')), ('Peer', v('x'), v('y')), ('Senior', v('x'))),                        # r10
]
def emp(i): return f'e{i}'
def make_facts(target, fanout=4, dag=False):
    """employees e0 (root) .. e(n-1); parent of e_i is e_((i-1)//fanout).  dag=True adds a second ('dotted-line') manager.  stops once the
fact count reaches target"""
    facts = []; i = 0
    while len(facts) < target:
        if i > 0:
            facts.append(('ReportsTo', emp(i), emp((i - 1) // fanout)))
            if dag and (i - 1) // fanout > 0: facts.append(('ReportsTo', emp(i), emp(((i - 1) // fanout - 1) // fanout if False else ((i -
1) // fanout) // 2)))
        facts.append(('DeptMember', f'd{i // 10}', emp(i)))
        if i % 7 == 0: facts.append(('Lead', emp(i)))
        i += 1
    n = i
    facts += [('Senior', emp(k)) for k in range(n) if fanout * k + 1 < n]           # every employee who manages somebody is senior
    return facts, n
def depth_of(i, fanout=4):
```

```
    d = 0
    while i > 0: i = (i - 1) // fanout; d += 1
    return d
LEFT = [
 R(('AboveL', v('x'), v('y')), ('ReportsTo', v('y'), v('x'))),
 R(('AboveL', v('x'), v('y')), ('AboveL', v('x'), v('z')), ('ReportsTo', v('y'), v('z'))),           # LEFT-recursive: calls itself first,
with the same first argument and an unbound second
]
```

A.8 Task 7 – medication-safety KB and advisor (med_kb.py)

```
"""Task 7 - hospital medication-safety advisor: knowledge base generator (rule-per-item Datalog / definite clauses over one patient variable)"""
import random
from fol import *
v = lambda n: '?' + n
def R(h, *b): return (h, list(b))
P = v('p')
ALLERGY =
[('penicillin','amoxicillin'),('penicillin','ampicillin'),('penicillin','penicillinV'),('penicillin','cefalexin'),('sulfa','sulfamethoxazole'),('nsaid','ibuprof
INTER_SYM = [('warfarin','aspirin'),('warfarin','ibuprofen'),('sertraline','tramadol'),('sertraline','phenelzine'),('simvastatin','clarithromycin'),('sildenafil
INTER_ONE = [('lisinopril','spironolactone'),('methotrexate','trimethoprim')]
CONTRA =
[('ibuprofen','kidney_disease'),('ibuprofen','ulcer'),('naproxen','ulcer'),('aspirin','ulcer'),('metformin','kidney_disease'),('lisinopril','pregnancy'),('warfa
CHILD = ['aspirin','tramadol','codeine']; ELDERLY = ['diazepam','amitriptyline']
NSAIDS = ['ibuprofen','naproxen','aspirin','diclofenac']
# a rule is (key_drug or None, rule): key_drug = the prescribed drug that triggers it (used by the rule index)
def allergy_rule(s, d): return (d, R(('Danger', P, f'allergy:{s}:{d}'), ('Rx', P, d), ('Allergy', P, s)))
def inter_rule(d, e): return (d, R(('Danger', P, f'interaction:{d}+{e}'), ('Rx', P, d), ('Takes', P, e)))
def contra_rule(d, c): return (d, R(('Danger', P, f'contra:{d}:{c}'), ('Rx', P, d), ('Condition', P, c)))
def child_rule(d): return (d, R(('Danger', P, f'age:child:{d}'), ('Rx', P, d), ('Child', P)))
def elderly_rule(d): return (d, R(('Danger', P, f'age:elderly:{d}'), ('Rx', P, d), ('Elderly', P)))
HELPERS = [R(('Child', P), ('Age', P, v('a')), ('Ge', 11, v('a'))), R(('Elderly', P), ('Age', P, v('a')), ('Ge', v('a'), 75)),
           R(('Unsafe', P), ('Danger', P, v('w')))]
GENERAL = [R(('Danger', P, 'duplicate:nsaid'), ('Rx', P, v('d')), ('NSAID', v('d')), ('Takes', P, v('e')), ('NSAID', v('e')))]   # not tied to one drug: always
def base_rules(draft=False):
    rs = [allergy_rule(s, d) for s, d in ALLERGY + ([] if draft else [('nsaid', 'diclofenac')])]
    for d, e in INTER_SYM: rs += [inter_rule(d, e), inter_rule(e, d)]
    rs += [inter_rule(d, e) for d, e in INTER_ONE] + ([] if draft else [inter_rule(e, d) for d, e in INTER_ONE]) + [contra_rule(d, c) for d, c in CONTRA]
    rs += [child_rule(d) for d in CHILD] + [elderly_rule(d) for d in ELDERLY]
    return rs
BASE_FACTS = [('NSAID', d) for d in NSAIDS]
def grown_rules(total, seed=7):
    """base rules plus synthetic items in the same proportions until the KB has `total` rules (helpers and general rules included in the count)"""
    rnd = random.Random(seed); rs = base_rules(); need = total - len(HELPERS) - len(GENERAL) - len(rs)
    nd = max(20, total // 4); drugs = [f'X{i}' for i in range(nd)]; allerg = [f'A{i}' for i in range(max(5, total // 20))]; conds = [f'C{i}' for i in range(max(
    for k in range(max(need, 0)):
        t = k % 10
        if t < 2: rs.append(allergy_rule(rnd.choice(allerg), rnd.choice(drugs)))
        elif t < 6: rs.append(inter_rule(rnd.choice(drugs), rnd.choice(drugs)))
        elif t < 8: rs.append(contra_rule(rnd.choice(drugs), rnd.choice(conds)))
        elif t < 9: rs.append(child_rule(rnd.choice(drugs)))
        else: rs.append(elderly_rule(rnd.choice(drugs)))
    return rs, drugs, allerg, conds
class Advisor:
    """final design: generic definite-clause KB, rules indexed by trigger drug, facts indexed by patient (first argument); backward chaining answers Danger(pati
    def __init__(self, keyed, facts, indexed=True):
        self.indexed = indexed; self.rules = HELPERS + GENERAL + [r for _, r in keyed]
        self.kb = KB(self.rules, BASE_FACTS + list(facts), index2=True)
        self.by_drug = {}
        for d, r in keyed: self.by_drug.setdefault(d, []).append(r)
        self.kbs = {}
    def _kb_for(self, drug):
        if not self.indexed: return self.kb
        k = self.kbs.get(drug)
        if k is None:
            import copy; k = copy.copy(self.kb); rp = dict(self.kb.rules_by_pred); rp['Danger'] = GENERAL + self.by_drug.get(drug, []); k.rules_by_pred = rp; se
        return k
    def check(self, patient, drug=None):
        """returns (set of reasons, proofs, BCStat).  drug: the prescribed drug (the Rx fact must already be in the KB for the patient)"""
        if drug is None: drug = next(f[2] for f in self.kb.facts_by_key[('Rx', patient)])
        p = Prover(self._kb_for(drug), 'plain'); ans = p.ask(('Danger', patient, '?w'))
        return {a[2] for a, _ in ans}, [pr for _, pr in ans], p.stat
```

A.9 Task 1 experiment driver (t1_run.py)

```
import json, time, sys
from wumpus import *
out = {}
print("TASK 1 - Wumpus world, propositional KB and model checking\n")
print("A  SYMBOL AND SENTENCE COUNTS (derived formulas: symbols = 4n^2;  sentences = 2 + 2n^2 + 1 + n^2(n^2-1)/2  (+ percept sentences))")
print(f"{'n':>3} {'symbols m':>10} {'formula':>8} {'sentences':>10} {'formula':>8} | {'models 2^m':>26}")
for n in (2, 3, 4, 5, 6):
    r = rules(n); print(f"{n:>3} {len({s for k in r.values() for s in symbols(k)}):>10} {nsymbols(n):>8} {len(r):>10} {nsentences(n):>8} |
{2**nsymbols(n):>26,}")
print("\nB  MODEL CHECKING, FULL SYMBOL SET vs RELEVANT-SYMBOL RESTRICTION")
print(f"{'n':>2} {'query':46s} {'method':10s} {'symbols':>7} {'models enumerated':>18} {'sentence evals':>14} {'time s':>9} | {'KB|=q':>6}
{'KB|=~q':>7} verdict")
res = []
rate = None
for n in (2, 3, 4):
    perc = percepts(n); Q = {k: v for k, v in queries(n).items() if k}
    for name, q in Q.items():
        row = {}
        for method in ('full', 'relevant'):
            if method == 'full':
                kb = list(rules(n).values()) + perc
                m = len({s for k in kb for s in symbols(k)} | symbols(q))
                if m > 20:
                    est = rate * 2 ** m if rate else float('nan')
                    print(f"{n:>2} {name:46s} {'full':10s} {m:>7} {'2^%d = %s' % (m, format(2**m, ',')):>18} {'-':>14} {'~%.1e'%est:>9} |
infeasible (rate extrapolation)"); continue
            else:
```

```
                kb = relevant_kb(n, perc, q); m = len({s for k in kb for s in symbols(k)} | symbols(q))
            a, b = MC(), MC(); e1 = tt_entails(kb, q, a); e2 = tt_entails(kb, Not(q), b)
            if method == 'full' and m == 16: rate = (a.time + b.time) / (a.models + b.models)
            verdict = 'ENTAILED' if e1 else ('NEGATION ENTAILED' if e2 else 'NEITHER')
            print(f"{n:>2} {name:46s} {method:10s} {m:>7} {a.models + b.models:>18,} {a.sentence_evals + b.sentence_evals:>14,} {a.time +
b.time:>9.4f} | {str(e1):>6} {str(e2):>7} {verdict}")
            res.append(dict(n=n, q=name, method=method, m=m, models=a.models + b.models, evals=a.sentence_evals + b.sentence_evals,
time=a.time + b.time, e1=e1, e2=e2, verdict=verdict))
out['mc'] = res
print("\nC  INDEPENDENT CROSS-CHECK on the FULL KB with a DPLL satisfiability solver (KB & ~q unsatisfiable <=> KB |= q); n=3, 4 have 36 /
64 symbols")
for n in (3, 4):
    kb = list(rules(n).values()) + percepts(n)
    for name, q in {k: v for k, v in queries(n).items() if k}.items():
        t = time.perf_counter(); e1 = dpll_entails(kb, q); e2 = dpll_entails(kb, Not(q))
        v = 'ENTAILED' if e1 else ('NEGATION ENTAILED' if e2 else 'NEITHER')
        mc = [x for x in res if x['n'] == n and x['q'] == name and x['method'] == 'relevant'][0]['verdict']
        print(f"  n={n}  {name:46s} full-KB verdict {v:18s} relevant-symbol verdict {mc:18s} agree: {v == mc}   ({time.perf_counter()-
t:.2f}s)")
        out.setdefault('dpll', []).append(dict(n=n, q=name, full=v, rel=mc, agree=v == mc))
print("\nD  PART B - INFERENCE-RULE DERIVATIONS (n=4, same KB and percepts)")
n = 4; perc = percepts(n)
for goal, desc, q in ((['-P22', '-W22'], '[2,2] is safe', And(Not(P(2, 2)), Not(W(2, 2)))), (['-P12', '-P21'], '[1,2] and [2,1] have no
pit', And(Not(P(1, 2)), Not(P(2, 1)))), (['P31'], 'pit at [3,1]', P(3, 1))):
    st, ok = derive(n, perc, goal)
    print(f"\n  Goal: {desc}  ->  derived: {ok};  inference steps needed = {len(st)}")
    for k, (i, rule, text, deps) in enumerate(st, 1): print(f"    {k:>2}. {rule:26s} {text[:150]}")
    kb = relevant_kb(n, perc, q); mm = MC(); tt_entails(kb, q, mm)
    print(f"    -> {len(st)} inference steps   versus   model checking (18 relevant symbols): {mm.models:,} models, {mm.sentence_evals:,}
sentence evaluations   (ratio {mm.sentence_evals/len(st):,.0f}x)")
    by = {}
    for st_ in st: by[st_[1]] = by.get(st_[1], 0) + 1
    out.setdefault('derive', []).append(dict(goal=desc, steps=len(st), ok=ok, by_rule=by, mc_models=mm.models, mc_evals=mm.sentence_evals))
json.dump(out, open('t1_results.json', 'w'), indent=1)
```

A.10 Task 2 experiment driver (t2_run.py)

```
import json, time, sys
from logic import *
from policy import RULES, CASES
out = {}
def chain(n):
    kb = [S('A0')] + [Imp(S(f'A{i}'), S(f'A{i+1}')) for i in range(n)]
    return kb, S(f'A{n}')
def php(k):                                  # pigeonhole: k+1 pigeons, k holes (unsatisfiable)
    v = lambda p, h: S(f'p{p}h{h}')
    kb = [Or(*[v(p, h) for h in range(k)]) for p in range(k + 1)]
    for h in range(k):
        for p in range(k + 1):
            for q in range(p + 1, k + 1): kb.append(Or(Not(v(p, h)), Not(v(q, h))))
    return kb
print("TASK 2 - Model checking versus resolution\n")
print("A  MANUAL CNF CONVERSION, stage by stage (the hand derivations in the report are checked against these outputs)")
for k in ('R1  a badge is valid exactly when it is not expired', 'R5  employee with valid badge in work hours may enter', 'R7  after hours,
no emergency, not security staff: may not enter'):
    s = RULES[k]; print(f"  {k[:3]}: {show(s)}")
    a = elim_iff(s); print(f"      1 eliminate <=>:  {show(a)}"); b = elim_imp(a); print(f"      2 eliminate =>:   {show(b)}"); c = nnf(b);
print(f"      3 push ~ inward:  {show(c)}"); cl = distribute(c); print(f"      4 distribute | over &:  " + '  &  '.join(fmt_clause(x) for x
in cl))
print("\nB  THE 12-RULE POLICY: six designed cases, resolution and model checking")
print(f"{'case':66s} | {'models':>8} {'res: gen':>8} {'attempted':>9} {'taut':>5} {'subs':>5} | verdict MC / RES")
kb = list(RULES.values()); res = []
for name, (facts, q) in CASES.items():
    k2 = kb + facts; m = MC(); r = RS(); e = tt_entails(k2, q, m); e2 = pl_resolution(k2, q, r)
    inc = tt_entails(k2, ('and', S('X'), Not(S('X'))), MC()); en = tt_entails(k2, Not(q), MC())           # is the KB inconsistent?
(entails a contradiction)
    print(f"{name:66s} | {m.models:>8,} {r.generated:>8} {r.attempted:>9} {r.tautologies:>5} {r.subsumed:>5} | {e} / {e2}" + ('   (also
~query not entailed: the KB leaves it OPEN)' if (e is False and en is False) else '') + ('   (KB+facts inconsistent: entails everything)'
if inc else ''))
    res.append(dict(case=name, models=m.models, gen=r.generated, att=r.attempted, taut=r.tautologies, subs=r.subsumed, mc=e, rs=e2,
inconsistent=inc))
out['policy'] = res
print("\n  Resolution proof of case C1 (clause, parents):")
facts, q = CASES['C1 employee, valid badge, work hours  |=  DoorUnlocked']; r = RS(); pl_resolution(kb + facts, q, r)
order, par = proof_steps(r)
idx = {c: i + 1 for i, c in enumerate(order)}
for c in order:
    p = par[c]; print(f"    {idx[c]:>2}. {fmt_clause(c):55s}" + ('given' if p is None else f'resolve {idx[p[0]]} and {idx[p[1]]} on
{p[2]}'))
out['proof_len'] = len(order)
print("\nC  RESOLUTION AS THE KB GROWS: implication chains A0, A0=>A1, ..., A(n-1)=>An ; query An (entailed); NOT-entailed test: the same
chain WITHOUT the fact A0, query An")
print(f"{'n':>4} {'clauses':>7} | {'RES entailed: generated':>23} {'attempted':>9} {'taut':>5} {'subsumed':>8} {'rounds':>6} {'time s':>8}
| {'RES not entailed: generated':>27} {'attempted':>9} {'terminates':>10} {'time s':>8} | {'MC models':>12} {'MC time s':>9}")
C = []
for n in (10, 20, 40, 80):
    kb, q = chain(n); r = RS(); e = pl_resolution(kb, q, r)
    r2 = RS(); e2 = pl_resolution(kb[1:], q, r2)
    if n <= 20:
        m = MC(); em = tt_entails(kb, q, m); mt = f"{m.models:>12,} {m.time:>9.2f}"; mm, mtt = m.models, m.time
    else:
        mm = 2 ** (n + 1); mtt = None; mt = f"{'2^%d = %.1e' % (n+1, mm):>12} {'infeasible':>9}"
    print(f"{n:>4} {len(kb):>7} | {r.generated:>23,} {r.attempted:>9,} {r.tautologies:>5} {r.subsumed:>8} {r.rounds:>6} {r.time:>8.3f} |
{r2.generated:>27,} {r2.attempted:>9,} {str(e2 is False):>10} {r2.time:>8.3f} | {mt}")
    C.append(dict(n=n, clauses=len(kb), gen=r.generated, att=r.attempted, taut=r.tautologies, subs=r.subsumed, rounds=r.rounds,
time=r.time, gen2=r2.generated, att2=r2.attempted, term2=(e2 is False), time2=r2.time, mc_models=mm, mc_time=mtt, entailed=e))
out['chains'] = C
print("\n  Same chains with and without subsumption (n = 40):")
kb, q = chain(40)
for sub in (True, False):
    r = RS(); e = pl_resolution(kb, q, r, subsumption=sub); print(f"    subsumption {str(sub):5s}: generated {r.generated:>6,}, attempted
{r.attempted:>7,}, subsumed {r.subsumed:>5}, time {r.time:.3f}s, result {e}")
print("\nD  KB FAMILIES THAT FAVOUR EACH METHOD")
print("  D1. Model checking time against the number of symbols m (chain KB, query entailed; every model enumerated: 2^m)")
D1 = []
for n in (8, 12, 16, 18):
```

```
    kb, q = chain(n); m = MC(); tt_entails(kb, q, m); r = RS(); pl_resolution(kb, q, r)
    print(f"     m={n+1:>2}: MC {m.models:>9,} models {m.time:>8.3f}s     resolution {r.generated:>5} clauses {r.time:>8.4f}s")
    D1.append(dict(m=n + 1, mc_models=m.models, mc_time=m.time, rs_gen=r.generated, rs_time=r.time))
out['d1'] = D1
print("\n   D2. Pigeonhole PHP(k+1, k): dense, unsatisfiable, few symbols.  KB entails every query; query = contradiction 'Z & ~Z'.")
print(f"{'k':>3} {'symbols m':>9} {'clauses':>8} | {'MC models':>10} {'MC time s':>9} | {'RES generated':>14} {'attempted':>10}
{'subsumed':>9} {'time s':>8} {'result':>8}")
D2 = []
for k in (2, 3, 4, 5):
    kb = php(k); q = And(S('Z'), Not(S('Z'))); m = MC(); r = RS()
    m_ = len(set().union(*[symbols(s) for s in kb]) | {'Z'})
    if m_ <= 22: tt_entails(kb, q, m); mc = f"{m.models:>10,} {m.time:>9.3f}"
    else: mc = f"{'2^%d' % m_:>10} {'infeasible':>9}"
    e = pl_resolution(kb, q, r, max_clauses=400000, time_limit=60)
    print(f"{k:>3} {m_:>9} {len(kb):>8} | {mc} | {r.generated:>14,} {r.attempted:>10,} {r.subsumed:>9,} {r.time:>8.2f} {('proved' if e else
('LIMIT' if e is None else 'no')):>8}")
    D2.append(dict(k=k, m=m_, clauses=len(kb), mc_models=m.models, mc_time=m.time, gen=r.generated, att=r.attempted, subs=r.subsumed,
time=r.time, res=e))
out['d2'] = D2
print("\n   D3. Horn chains with many symbols: resolution is polynomial, model checking is not even runnable beyond m = 25")
print(f"     chain n=80 (m=81):  resolution {C[-1]['gen']:,} clauses in {C[-1]['time']:.3f}s;  model checking would need 2^81 = {2**81:.2e}
models; at the measured {D1[-1]['mc_time']/D1[-1]['mc_models']*1e6:.2f} us per model = {2**81*D1[-1]['mc_time']/D1[-
1]['mc_models']/3.15e7:.1e} years")
json.dump(out, open('t2_results.json', 'w'), indent=1, default=str)
```

A.11 Task 3 experiment driver (t3_run.py)

```
import json, time, tracemalloc, random, math
from exam_kb import *
out = {}
print("TASK 3 - University exam-eligibility advisor: knowledge engineering\n")
print("A  THE FOL AXIOMS (14; * = executed as a definite clause with negation as failure)")
for a in AXIOMS: print('  ', a)
def table(rows):
    ok = 0
    for lab, exp, got, good, g, u, err in rows:
        e = exp if isinstance(exp, bool) else sorted(x[0] for x in exp); gg = got if isinstance(got, (bool, str)) else sorted(x[0] for x in
got)
        print(f"   {'PASS' if good else 'FAIL'}  {lab:50s} expected {str(e)[:60]:60s} got {str(gg)[:60]:60s} goals {g:>4}" + (f"   [{err}]"
if err else '')); ok += good
    return ok
print("\nB  QUERY SUITE ON DRAFT 1 (the first version of the KB; expected answers were written from the regulations before running)")
r1 = run_suite(DRAFT1); n1 = table(r1); print(f"   -> {n1}/15 pass; {15 - n1} fail")
print("\n   DEFECT LOG (each defect found by a failing query, diagnosed and fixed):")
print("   D1  S4 Eligible(gina, MA101) fails and B3 Barred(?s) returns gina as well as frank: rule 'Barred(s) :- DisciplinaryCase(s,k)'
ignored whether the case is still ACTIVE (closed case wrongly bars).  FIX: add Active(k) to the body.")
print("   D2  F2 Eligible(dave, CS301) succeeds although dave failed CS101, and B2 Needs(CS301, ?p) has no answer: the draft only checked
DIRECT prerequisites (no transitive closure).  FIX: add predicate Needs with a base rule and a recursive rule; MissingPrereq uses Needs.")
print("   D3  B4 FeesCleared(?s) raises an error (floundering): the body 'not FeeDue(s), Student(s)' evaluates the negative literal while s
is unbound.  FIX: put Student(s) first so that s is bound before the negation is evaluated.")
print("\n   QUERY SUITE ON THE FINAL KB")
r2 = run_suite(FINAL); n2 = table(r2); print(f"   -> {n2}/15 pass")
out['suite'] = dict(draft=[dict(q=x[0], ok=x[3], err=x[6]) for x in r1], final=[dict(q=x[0], ok=x[3], goals=x[4], unif=x[5]) for x in r2],
draft_pass=n1, final_pass=n2)
print("\n   Proof (explanation) for Eligible(alice, CS201):")
kb = KB(FINAL, INSTANCE); p = Prover(kb); a = p.ask(('Eligible', 'alice', 'CS201'))
for l in proof_lines(a[0][1]): print('     ' + l)
print("   Proof size", proof_size(a[0][1]), "nodes")
# ---------------------------------------------------------------------- growth derivation + measurement
def fol_instance(s, c, seed=1):
    rnd = random.Random(seed); facts = []; courses = [f'C{j}' for j in range(c)]
    for j in range(c):
        facts += [('Course', courses[j]), ('MinAtt', courses[j], 75)]
        if j >= 1: facts.append(Req(courses[j], courses[j - 1]))         # linear prerequisite chain
    for i in range(s):
        st = f'S{i}'; facts.append(('Student', st))
        if rnd.random() < 0.1: facts.append(('FeeDue', st))
        if rnd.random() < 0.05: facts += [('DisciplinaryCase', st, f'K{i}')] + ([('Active', f'K{i}')] if rnd.random() < 0.5 else [])
        for j in range(c):
            if rnd.random() < 0.5:
                facts += [('Enrolled', st, courses[j]), ('Attendance', st, courses[j], rnd.randint(55, 100))]
                if rnd.random() < 0.1: facts.append(('ApprovedLeave', st, courses[j]))
            if j >= 1 and rnd.random() < 0.8: facts.append(('Grade', st, courses[j - 1], rnd.randint(30, 90)))
    return facts, courses
def prop_encoding(facts, s, c):
    """one set of propositional (ground) rules per student and course.  Atoms are strings.  The arithmetic and the prerequisite closure are
precomputed (propositional logic cannot compare numbers or quantify)."""
    F = set(facts); rules = {}; n = 0
    def add(h, b):
        nonlocal n; rules.setdefault(h, []).append(b); n += 1
    need = {}
    for j in range(c): need[f'C{j}'] = [f'C{k}' for k in range(j)]
    for i in range(s):
        st = f'S{i}'
        add(f'FeesCleared_{st}', [f'Student_{st}', '~FeeDue_' + st]); add(f'Barred_{st}', [f'Case_{st}', f'ActiveCase_{st}'])
        for j in range(c):
            co = f'C{j}'; k = f'{st}_{co}'
            add(f'AttendanceOK_{k}', [f'Enrolled_{k}', f'AttGEmin_{k}']); add(f'AttendanceOK_{k}', [f'Enrolled_{k}', f'Leave_{k}'])
            for p in need[co]: add(f'MissingPrereq_{k}', [f'Enrolled_{k}', f'~Passed_{st}_{p}'])
            add(f'PrereqsCleared_{k}', [f'Enrolled_{k}', f'~MissingPrereq_{k}'])
            add(f'Eligible_{k}', [f'AttendanceOK_{k}', f'PrereqsCleared_{k}', f'FeesCleared_{st}', f'~Barred_{st}'])
    pf = set()
    for f in F:
        if f[0] == 'Student': pf.add(f'Student_{f[1]}')
        elif f[0] == 'FeeDue': pf.add(f'FeeDue_{f[1]}')
        elif f[0] == 'DisciplinaryCase': pf.add(f'Case_{f[1]}')
        elif f[0] == 'Active': pf.add(f'ActiveCase_{f[1][1:]}' if False else f'ActiveCase_S{f[1][1:]}')
        elif f[0] == 'Enrolled': pf.add(f'Enrolled_{f[1]}_{f[2]}')
        elif f[0] == 'Attendance' and f[3] >= 75: pf.add(f'AttGEmin_{f[1]}_{f[2]}')
        elif f[0] == 'ApprovedLeave': pf.add(f'Leave_{f[1]}_{f[2]}')
        elif f[0] == 'Grade' and f[3] >= 40: pf.add(f'Passed_{f[1]}_{f[2]}')
    return rules, pf, n
def prop_ask(rules, pf, atom, memo, ctr):
    if atom in memo: return memo[atom]
    ctr[0] += 1
    if atom[0] == '~': r = not prop_ask(rules, pf, atom[1:], memo, ctr)
```

```
    elif atom in pf: r = True
    else: r = any(all(prop_ask(rules, pf, b, memo, ctr) for b in body) for body in rules.get(atom, []))
    memo[atom] = r; return r
print("\nC  GROWTH OF THE KB WITH s STUDENTS AND c COURSES")
print("   Propositional encoding, rules per student s and course c (counted by the program):  per (s,c): 2 attendance rules + (c_index)
missing-prerequisite rules + 1 prerequisite rule + 1 eligibility rule;  per student: 2 rules (fees, barred)")
print("      N_rules(s, c) = s * [ 2 + sum_{j=0}^{c-1} (4 + j) ]  =  s * [ 2 + 4c + c(c-1)/2 ]       (the dependence is linear in s and
quadratic in c because of the prerequisite chain)")
print("   First-order encoding: the number of AXIOMS is the constant 10 executable rules (14 axioms in the full theory), whatever s and c
are; only the FACTS (data) grow with s and c.\n")
print(f"{'s':>6} {'c':>3} | {'prop rules':>10} {'formula':>9} | {'FOL axioms':>10} {'FOL facts':>9} | {'prop sentences (rules+facts)':>28}
{'FOL sentences':>13} | {'edit cost: sentences changed to alter the attendance minimum':>60}")
G = []
for s, c in ((10, 5), (100, 5), (1000, 5), (10, 20), (100, 20), (1000, 20), (3000, 20)):
    facts, _ = fol_instance(s, c); rules, pf, n = prop_encoding(facts, s, c); form = s * (2 + 4 * c + c * (c - 1) // 2)
    print(f"{s:>6} {c:>3} | {n:>10,} {form:>9,} | {len(FINAL):>10} {len(facts):>9,} | {n + len(pf):>28,} {len(FINAL) + len(facts):>13,} |
prop: {s*c:,} pf-facts recomputed (one per student-course) ;  FOL: 1 fact per course ({c})")
    G.append(dict(s=s, c=c, prop_rules=n, formula=form, axioms=len(FINAL), facts=len(facts), prop_sent=n + len(pf), fol_sent=len(FINAL) +
len(facts)))
out['growth'] = G
print("\nD  INFERENCE COST: 100 eligibility queries Eligible(S_i, C_j) (random pairs, seed 5).  prop = indexed ground backward chaining
with memo;  FOL = backward chaining with unification, indexed facts;  times are totals of the 100 queries")
print(f"{'s':>6} {'c':>3} | {'prop build s':>12} {'prop memory MB':>14} {'prop goals':>10} {'prop query ms':>13} | {'FOL build s':>11}
{'FOL memory MB':>13} {'FOL goals':>9} {'FOL unifications':>16} {'FOL query ms':>12} | {'answers agree':>13} {'break-even #queries':>19}")
H = []
for s, c in ((10, 5), (100, 5), (1000, 5), (10, 20), (100, 20), (1000, 20), (3000, 20)):
    rnd = random.Random(5); qs = [(f'S{rnd.randrange(s)}', f'C{rnd.randrange(c)}') for _ in range(100)]
    facts, _ = fol_instance(s, c)
    tracemalloc.start(); rules, pf, n = prop_encoding(facts, s, c); mem_p = tracemalloc.get_traced_memory()[1] / 1e6; tracemalloc.stop()
    tracemalloc.start(); kb = KB(FINAL, facts); mem_f = tracemalloc.get_traced_memory()[1] / 1e6; tracemalloc.stop()
    tb_p = tb_f = 1e9
    for _ in range(3):                                          # best of 3 builds (the first build of a process includes interpreter
warm-up)
        t0 = time.perf_counter(); rules, pf, n = prop_encoding(facts, s, c); tb_p = min(tb_p, time.perf_counter() - t0)
        t0 = time.perf_counter(); kb = KB(FINAL, facts); tb_f = min(tb_f, time.perf_counter() - t0)
    t0 = time.perf_counter(); ctr = [0]; pa = []
    for st, co in qs: pa.append(prop_ask(rules, pf, f'Eligible_{st}_{co}', {}, ctr))
    tq_p = time.perf_counter() - t0
    t0 = time.perf_counter(); g = u = 0; fa = []
    for st, co in qs:
        p = Prover(kb); fa.append(bool(p.ask(('Eligible', st, co), limit=1))); g += p.stat.goals; u += p.stat.unifications
    tq_f = time.perf_counter() - t0
    # break-even: total = build + q * per-query.  FOL cheaper while  tb_f + q*tf < tb_p + q*tp
    tf, tp = tq_f / 100, tq_p / 100
    be = (tb_p - tb_f) / (tf - tp) if tf > tp else float('inf')
    print(f"{s:>6} {c:>3} | {tb_p:>12.3f} {mem_p:>14.1f} {ctr[0]:>10,} {tq_p*1000:>13.2f} | {tb_f:>11.3f} {mem_f:>13.1f} {g:>9,} {u:>16,}
{tq_f*1000:>12.2f} | {str(pa == fa):>13} {be:>19,.0f}")
    H.append(dict(s=s, c=c, tb_p=tb_p, mem_p=mem_p, goals_p=ctr[0], tq_p=tq_p, tb_f=tb_f, mem_f=mem_f, goals_f=g, unif_f=u, tq_f=tq_f,
agree=pa == fa, breakeven=be))
out['infer'] = H
json.dump(out, open('t3_results.json', 'w'), indent=1, default=str)
```

A.12 Task 4 experiment driver (t4_run.py)

```
"""CSA17 A3 Task 4 - unification, lifted inference and propositionalisation"""
import json, time, itertools
from fol import *
out = {}
print("TASK 4 - Unification and lifted inference\n")
print("A  UNIFICATION TESTS (the five pairs derived by hand in the report; results below come from the program)")
pairs = [
 ("P1  Knows(John, ?x)  vs  Knows(?y, Mother(?y))", ('Knows', 'John', '?x'), ('Knows', '?y', ('Mother', '?y'))),
 ("P2  Likes(?x, ?x)  vs  Likes(Anna, Bob)", ('Likes', '?x', '?x'), ('Likes', 'Anna', 'Bob')),
 ("P3  P(?x, f(?x))  vs  P(?y, ?y)          (occurs check)", ('P', '?x', ('f', '?x')), ('P', '?y', '?y')),
 ("P4  Knows(John, ?x)  vs  Knows(?x, Elizabeth)   (both sentences use the name ?x: standardising apart is essential)", ('Knows', 'John',
'?x'), ('Knows', '?x', 'Elizabeth')),
 ("P5  R(?x, g(?y, ?z), ?z)  vs  R(h(?w), ?v, k(?w))", ('R', '?x', ('g', '?y', '?z'), '?z'), ('R', ('h', '?w'), '?v', ('k', '?w'))),
 ("P6  Q(a, ?x)  vs  Q(b, ?x)   (clash of constants)", ('Q', 'a', '?x'), ('Q', 'b', '?x')),
]
res = []
for name, a, b in pairs:
    r1 = mgu_full(a, b, True); r0 = mgu_full(a, b, False)
    print(f"  {name}\n      with occurs check: {sstr(r1) if r1 is not None else 'FAILURE'}      without: {sstr(r0) if r0 is not None else
'FAILURE'}")
    if r1 is not None and r1 != 'CYCLIC': assert subst(a, r1) == subst(b, r1), 'unifier does not unify'
    res.append(dict(pair=name, with_oc=sstr(r1) if r1 is not None else None, without=sstr(r0) if r0 is not None else None))
print("\n   P4 after standardising apart (rename the second sentence's variable ?x to ?x1): Knows(John, ?x)  vs  Knows(?x1, Elizabeth):")
r = mgu_full(('Knows', 'John', '?x'), ('Knows', '?x1', 'Elizabeth')); print("      ", sstr(r), "  -> common instance:",
tstr(subst(('Knows', 'John', '?x'), r)))
print("   and P3 without the occurs check yields the cyclic binding ?y/?x, ?x/f(?x) -> unsound: it would unify P(x,f(x)) with P(y,y), which
no ground instance satisfies.")
out['unify'] = res
print("\n   Composition of substitutions: s1 = {?x/f(?y), ?z/?w}, s2 = {?y/a, ?w/b}:")
s1 = {'?x': ('f', '?y'), '?z': '?w'}; s2 = {'?y': 'a', '?w': 'b'}; c = compose(s1, s2)
t = ('R', '?x', '?z', '?y')
print("      compose =", sstr(c), "; check apply(apply(t,s1),s2) == apply(t,compose):", subst(subst(t, s1), s2) == subst(t, c))
print("\nB  LIFTED MODUS PONENS  (rule  Parent(?x,?y) & Parent(?y,?z) => Grandparent(?x,?z);  facts Parent(Ann,Bob), Parent(Bob,Cy))")
rule = (('Grandparent', '?x', '?z'), [('Parent', '?x', '?y'), ('Parent', '?y', '?z')]); facts = [('Parent', 'Ann', 'Bob'), ('Parent',
'Bob', 'Cy')]
th = {}; ok = True
for lit, f in zip(rule[1], facts):
    th = unify(lit, f, th); print(f"      unify {tstr(lit)} with {tstr(f)}  ->  {sstr({k: subst(k, th) for k in th})}")
print("      conclusion:", tstr(subst(rule[0], th)))
print("\nC  GROUND-CLAUSE GROWTH WITH THE NUMBER OF CONSTANTS k (rule R1: Edge(?x,?y) & Edge(?y,?z) => Path(?x,?z)  has 3 variables; R2:
Edge(?x,?y) => Path(?x,?y) has 2;  predicate arity r in the table below)")
print(f"{'k':>3} | {'3 variables: k^3':>16} {'counted':>9} | {'2 variables k^2':>15} | arity of the head predicate / number of distinct
variables: r=1 {'k':>5} r=2 {'k^2':>7} r=3 {'k^3':>9} r=4 {'k^4':>11} r=5 {'k^5':>13}")
C = []
r1 = (('Path', '?x', '?z'), [('Edge', '?x', '?y'), ('Edge', '?y', '?z')]); r2 = (('Path', '?x', '?y'), [('Edge', '?x', '?y')])
for k in (3, 5, 10, 20, 30):
    consts = [f'c{i}' for i in range(k)]
    n1 = len(propositionalise([r1], consts)) if k <= 20 else k ** 3; n2 = len(propositionalise([r2], consts))
    ar = [k ** r for r in (1, 2, 3, 4, 5)]
    print(f"{k:>3} | {k**3:>16,} {n1:>9,} | {n2:>15,} |                                                                                {ar[0]:>5,}
{ar[1]:>7,}     {ar[2]:>9,}      {ar[3]:>11,}      {ar[4]:>13,}")
    C.append(dict(k=k, k3=k ** 3, counted3=n1, k2=n2, arity=ar))
```

```
out['ground'] = C
print("   (for k=30 the 27,000 instances were counted by formula; k <= 20 were generated and counted; arity r needs k^r instances for a
rule whose r variables are all different)")
print("\nD  FUNCTION SYMBOL: ground terms of nesting depth <= d  (constants: k = 2;  f unary;  g binary)")
print(f"{'d':>2} | {'only f: 1 unary':>16} {'formula k(d+1)':>15} | {'f and g':>12} {'recurrence T_d = k + T_(d-1) + T_(d-1)^2':>42}")
D = []
T = 2
for d in range(0, 5):
    nf = len(ground_terms(['a', 'b'], {'f': 1}, d)) if d <= 3 else 2 * (d + 1); ng = len(ground_terms(['a', 'b'], {'f': 1, 'g': 2}, d)) if
d <= 3 else None
    if d > 0: T = 2 + T + T * T                          # N_d = k + N_(d-1) [f(t)] + N_(d-1)^2 [g(t,t')]
    print(f"{d:>2} | {nf:>16} {2*(d+1):>15} | {ng if ng is not None else '-':>12} {T:>42}")
    D.append(dict(d=d, f_only=nf, f_g=ng))
out['terms'] = D
print("   the counts of the program equal the recurrence N_d = k + N_(d-1) + N_(d-1)^2 for {f, g} (N = 2, 8, 74, 5,552) and k(d+1) for f
alone")
print("\nE  LIFTED versus PROPOSITIONAL INFERENCE on the same query")
print("   KB: Edge facts on a chain c0->c1->...->c(k-1) rules: R2 and R1' = Path(?x,?z) :- Edge(?x,?y), Path(?y,?z).  Query: Path(c0, ?z)
(all answers) and  Path(c0, c(k-1)) (yes/no)")
print(f"{'k':>4} | {'ground rules':>12} {'prop FC firings':>16} {'prop FC time s':>14} | {'lifted BC goals':>15} {'unifications':>12}
{'time s':>9} | {'lifted FC firings':>17} {'time s':>9} | answers")
E = []
rec = (('Path', '?x', '?z'), [('Edge', '?x', '?y'), ('Path', '?y', '?z')])
for k in (5, 10, 20, 30, 40, 60):
    consts = [f'c{i}' for i in range(k)]; facts = [('Edge', f'c{i}', f'c{i+1}') for i in range(k - 1)]
    rules = [r2, rec]
    t0 = time.perf_counter(); ground = propositionalise(rules, consts)
    gk = KB(); gfacts = set(facts)                                  # propositional forward chaining over ground rules
    known = set(facts); firings = 0; changed = True
    while changed:
        changed = False
        for h, b in ground:
            if h not in known and all(x in known for x in b): known.add(h); changed = True
            firings += 1
    tp = time.perf_counter() - t0
    kb = KB(rules, facts); p = Prover(kb); ans = p.ask(('Path', 'c0', '?z'))
    kb2 = KB(rules, facts); fs = forward_chain(kb2)
    print(f"{k:>4} | {len(ground):>12,} {firings:>16,} {tp:>14.4f} | {p.stat.goals:>15,} {p.stat.unifications:>12,} {p.stat.time:>9.4f} |
{fs.firings:>17,} {fs.time:>9.4f} | {len(ans)} (prop: {sum(1 for f in known if f[0]=='Path' and f[1]=='c0')})")
    E.append(dict(k=k, ground=len(ground), prop_firings=firings, prop_time=tp, bc_goals=p.stat.goals, bc_unif=p.stat.unifications,
bc_time=p.stat.time, fc_firings=fs.firings, fc_time=fs.time, answers=len(ans)))
out['lifted'] = E
print("\n   Query concerning ONE constant in a KB where almost all facts are irrelevant (k=60, query Path(c58, ?z)): lifted backward
chaining touches only the relevant constants")
kb = KB(rules, [('Edge', f'c{i}', f'c{i+1}') for i in range(59)]); p = Prover(kb); a = p.ask(('Path', 'c58', '?z'))
print(f"      answers {len(a)}, goals expanded {p.stat.goals}, unifications {p.stat.unifications};  propositionalisation still creates
{len(propositionalise(rules, [f'c{i}' for i in range(60)])):,} ground rules")
out['one_const'] = dict(answers=len(a), goals=p.stat.goals, unif=p.stat.unifications)
json.dump(out, open('t4_results.json', 'w'), indent=1, default=str)
```

A.13 Task 5 experiment driver (t5_run.py)

```
import json, time
from fol import *
from fol_res import *
from logic import S as PS, Or as POr, Not as PNot, pl_resolution, RS
out = {}
x, d, e, y = '?x', '?d', '?e', '?y'
Emp = lambda t: ('Emp', t); Doc = lambda t: ('Doc', t); Rest = lambda t: ('Restricted', t); Hand = lambda a, b: ('Handled', a, b); Auth =
lambda t: ('Auth', t)
Pd = lambda t: ('ContainsPD', t); Aud = lambda t: ('Audited', t)
A1 = Forall(x, ('imp', ('and', Emp(x), Exists(d, ('and', ('and', Doc(d), Rest(d)), Hand(x, d)))), Auth(x)))
A2 = Forall(d, ('imp', ('and', Doc(d), Pd(d)), Rest(d)))
A3 = Exists(e, ('and', Emp(e), Forall(d, ('imp', ('and', Doc(d), Aud(d)), Hand(e, d)))))
A4 = ('and', ('and', Doc('R1'), Pd('R1')), Aud('R1'))
GOAL = Exists(x, ('and', Emp(x), Auth(x)))
print("TASK 5 - First-order resolution prover\n")
print("A  CONVERSION TO CNF, every step (statements 1-4 and the negated query)\n")
ax = []
for lab, f in (('S1', A1), ('S2', A2), ('S3', A3), ('S4', A4)): ax += to_clauses(f, True, lab); print()
neg = to_clauses(('not', GOAL), True, 'NEGATED QUERY  ~Ex(Emp(x) & Auth(x))')
print(f"\n  Clause set: {len(ax)} axiom clauses + {len(neg)} negated-goal clause")
for c in ax + neg: print('     ', cstr(c))
print("\nB  PROOF THAT AN AUTHORISED EMPLOYEE EXISTS (plain breadth-first resolution, occurs check on)")
g, st, info = prove_resolution(ax, neg, 'plain')
print(f"  result {st.result}: generated {st.generated}, resolution steps {st.steps}, unifications {st.unifications}, proof length
{st.proof_len}, time {st.time*1000:.1f} ms")
for l in proof_text(g, info): print(l)
rows = []
print("\nC  STRATEGY COMPARISON (same clause set)")
print(f"{'strategy':28s} {'result':8s} {'clauses generated':>17} {'resolution steps':>16} {'unifications':>12} {'given clauses':>13}
{'proof length':>12} {'time ms':>8}")
for strat, lab in (('plain', 'plain (FIFO, breadth-first)'), ('unit', 'unit preference'), ('unitfair', 'unit pref. + every 5th oldest'),
('sos', 'set of support')):
    g, st, info = prove_resolution(ax, neg, strat)
    print(f"{lab:28s} {st.result[:8]:8s} {st.generated:>17} {st.steps:>16} {st.unifications:>12} {st.given:>13} {str(st.proof_len):>12}
{st.time*1000:>8.1f}")
    rows.append(dict(strategy=lab, result=st.result[:8], gen=st.generated, steps=st.steps, unif=st.unifications, given=st.given,
proof=st.proof_len, ms=st.time * 1000))
    if strat == 'sos': sos_proof = proof_text(g, info)
out['strategies'] = rows
print("\n   Proof found by the set-of-support strategy:")
for l in sos_proof: print(l)
print("\nD  SECOND QUERY THAT IS NOT ENTAILED:  'some authorised employee handled a document R2'  Ex(Emp(x) & Auth(x) & Handled(x, R2))")
G2 = Exists(x, ('and', ('and', Emp(x), Auth(x)), Hand(x, 'R2'))); neg2 = to_clauses(('not', G2))
print("  KB as given (finite Herbrand universe {Sk1, R1}): ")
for strat in ('plain', 'sos'):
    g, st, info = prove_resolution(ax, neg2, strat, max_steps=20000)
    print(f"    {strat:6s}: {st.result}; generated {st.generated}, steps {st.steps}, time {st.time*1000:.1f} ms")
    out.setdefault('nonent', []).append(dict(kb='finite', strategy=strat, result=st.result[:30], gen=st.generated, steps=st.steps))
print("  KB extended with S5: every employee has an employee manager  Ax(Emp(x) => Ey(Emp(y) & ReportsTo(x,y)))  (Skolem FUNCTION f(x):
infinite Herbrand universe)")
A5 = Forall(x, ('imp', Emp(x), Exists(y, ('and', Emp(y), ('ReportsTo', x, y)))))
ax5 = ax + to_clauses(A5, True, 'S5')
print("   same two queries, step cap 20,000 resolution steps / 20,000 clauses:")
for lab, ng in (('entailed query (authorised employee exists)', neg), ('NOT entailed query (handled R2)', neg2)):
```

```
    for strat in ('plain', 'unit', 'unitfair', 'sos'):
        g, st, info = prove_resolution(ax5, ng, strat, max_steps=20000, max_clauses=20000, time_limit=60)
        print(f"    {lab:46s} {strat:6s}: {st.result[:30]:30s} generated {st.generated:>6}, steps {st.steps:>6}, max clause size
{st.max_clause_size}, time {st.time:.2f}s")
        out.setdefault('nonent', []).append(dict(kb='with Skolem function', query=lab, strategy=strat, result=st.result[:30],
gen=st.generated, steps=st.steps, secs=st.time))
print("\nE  COMPARISON WITH THE PROPOSITIONAL PROVER OF TASK 2 ON AN EQUIVALENT SMALL DOMAIN")
print("   The FOL clauses are instantiated with every constant of a domain (Sk1, R1 and n extra constants) and given to the propositional
resolution prover (logic.pl_resolution).")
def ground(cl, terms):
    res = []
    for c in cl:
        vs = []
        for s, a in c: variables(a, vs)
        for combo in itertools.product(terms, repeat=len(vs)):
            th = dict(zip(vs, combo)); gc = tuple((s, subst(a, th)) for s, a in c)
            if not is_tautology(gc): res.append(gc)
    return res
def to_prop(gc):
    lits = [(PS(tstr(a)) if s else PNot(PS(tstr(a)))) for s, a in gc]
    return POr(*lits) if len(lits) > 1 else lits[0]
print(f"{'domain size':>11} {'ground clauses':>14} | {'propositional: generated':>24} {'attempted':>9} {'time ms':>8} | {'FOL (set of
support): generated':>31} {'steps':>6} {'time ms':>8}")
E = []
for extra in (0, 2, 6, 12):
    terms = ['Sk1', 'R1'] + [f'k{i}' for i in range(extra)]
    gc = ground(ax + neg, terms); rs = RS(); t = time.perf_counter()
    r = pl_resolution([to_prop(c) for c in gc], PS('FALSE__'), rs, time_limit=60); tm = (time.perf_counter() - t) * 1000
    g, st, info = prove_resolution(ax, neg, 'sos')
    print(f"{len(terms):>11} {len(gc):>14} | {rs.generated:>24,} {rs.attempted:>9,} {tm:>8.1f} | {st.generated:>31} {st.steps:>6}
{st.time*1000:>8.1f}   (propositional result: {r})")
    E.append(dict(domain=len(terms), ground=len(gc), p_gen=rs.generated, p_att=rs.attempted, p_ms=tm, f_gen=st.generated, f_steps=st.steps,
f_ms=st.time * 1000, p_res=r))
out['prop'] = E
json.dump(out, open('t5_results.json', 'w'), indent=1, default=str)
```

A.14 Task 6 experiment driver (t6_run.py)

```
import json, time, sys
from org_kb import *
sys.setrecursionlimit(20000)
out = {}
print("TASK 6 - Forward and backward chaining on an organisation hierarchy\n")
print("A  THE KNOWLEDGE BASE (10 definite clauses)")
for i, (h, b) in enumerate(RULES, 1): print(f"   r{i:<2} {tstr(h)} :- {', '.join(tstr(x) for x in b)}")
SIZES = (100, 1000, 10000)
def closure_relevant(kb, atoms):
    """derived atoms that can contribute to any of the given ground atoms, using the fixed-point closure kb (all supporting derivations,
memoised)"""
    seen = set(); stack = list(atoms); base = BASE
    while stack:
        a = stack.pop()
        if a in seen or a in base: continue
        seen.add(a)
        for rule in kb.rules_by_pred.get(a[0], []):
            h, body = rename_rule(rule); th = unify(a, h, {}, False)
            if th is None: continue
            st = FCStat()
            for th2 in match_body(kb, body, th, st):
                for b in body:
                    if b[0] != 'not': stack.append(subst(b, th2))
    return seen
def fc_row(facts, semi):
    kb = KB(RULES, facts, index2=True); st = forward_chain(kb, seminaive=semi); return kb, st
print("\nB  FORWARD CHAINING (Student A):  naive = every rule is re-matched against all facts in every iteration;  semi-naive = a rule
instance must use a fact derived in the previous iteration")
print(f"{'base facts':>10} {'employees':>9} | {'mode':>10} {'iterations':>10} {'facts inferred':>14} {'rule firings':>13} {'redundant':>11}
{'redundant %':>11} {'unifications':>13} {'time s':>8}")
B = []; META = {}
for T in SIZES:
    facts, n = make_facts(T); BASE = set(facts); META[T] = (facts, n)
    for semi in (False, True):
        kb, st = fc_row(facts, semi); inf = len(kb.facts) - len(facts)
        print(f"{len(facts):>10,} {n:>9,} | {'semi-naive' if semi else 'naive':>10} {st.iterations:>10} {inf:>14,} {st.firings:>13,}
{st.redundant:>11,} {100*st.redundant/st.firings:>10.1f}% {st.unifications:>13,} {st.time:>8.2f}")
        B.append(dict(T=T, facts=len(facts), n=n, semi=semi, iters=st.iterations, inferred=inf, firings=st.firings, redundant=st.redundant,
unif=st.unifications, time=st.time))
        if semi: KBFC = kb
    META[T] = (facts, n, KBFC)
out['fc'] = B
print("\n   Derived facts by predicate (semi-naive result) and relevance to three queries")
C = []
for T in SIZES:
    facts, n, kb = META[T]; BASE = set(facts); L = n - 1; dl = depth_of(L)
    derived = [f for f in kb.facts if f not in BASE]; byp = {}
    for f in derived: byp[f[0]] = byp.get(f[0], 0) + 1
    print(f"   {len(facts):>6,} base facts, {n} employees, leaf e{L} at depth {dl}:  " + ', '.join(f'{k} {c:,}' for k, c in
sorted(byp.items())) + f"   total derived {len(derived):,}")
    qs = {'Q1 CanApprove(e0, leaf)': [('CanApprove', 'e0', emp(L))], 'Q4 CanApprove(?x, leaf)': [f for f in kb.facts_by_pred['CanApprove']
if f[2] == emp(L)], 'Q3 Above(e0, ?y)': [f for f in kb.facts_by_pred['Above'] if f[1] == 'e0']}
    for name, ans in qs.items():
        rel = closure_relevant(kb, ans); pct = 100 * (1 - len(rel) / len(derived))
        print(f"        {name:28s} answers {len(ans):>5,}  relevant derived facts {len(rel):>6,}  irrelevant {pct:6.2f}% of
{len(derived):,}")
        C.append(dict(T=T, q=name, answers=len(ans), relevant=len(rel), derived=len(derived), irrelevant_pct=pct))
out['relevance'] = C


print("\nC  BACKWARD CHAINING (Student B): SLD resolution with substitutions, indexed facts (predicate + first or second argument), plain
mode")
print("   Q1 CanApprove(e0, leaf) [ground, true]   Q2 CanApprove(leaf, e0) [ground, false]   Q3 Above(e0, ?y) [all subordinates of the
root]   Q4 CanApprove(?x, leaf) [all who may approve the leaf]   Q5 CanApprove(e1, ?y) [subordinates of a mid-level manager]")
print(f"{'base facts':>10} | {'query':>26} {'solutions':>9} {'goals':>9} {'unifications':>12} {'max depth':>9} {'repeated':>9} {'time
ms':>9} {'= FC answer':>11}")
D = []
for T in SIZES:
```

```
    facts, n, kbfc = META[T]; L = n - 1; kb = KB(RULES, facts, index2=True)
    Q = [('Q1 CanApprove(e0,leaf)', ('CanApprove', 'e0', emp(L))), ('Q2 CanApprove(leaf,e0)', ('CanApprove', emp(L), 'e0')), ('Q3
Above(e0,?y)', ('Above', 'e0', '?y')), ('Q4 CanApprove(?x,leaf)', ('CanApprove', '?x', emp(L))), ('Q5 CanApprove(e1,?y)', ('CanApprove',
'e1', '?y'))]
    for name, g in Q:
        p = Prover(kb); ans = p.ask(g); got = {a for a, _ in ans}
        fcset = {f for f in kbfc.facts_by_pred.get(g[0], []) if all(is_var(g[i]) or g[i] == f[i] for i in range(1, len(g)))}
        ok = got == fcset if not (name[1] in '12') else bool(got) == bool(fcset)
        s = p.stat; print(f"{len(facts):>10,} | {name:>26} {s.solutions:>9,} {s.goals:>9,} {s.unifications:>12,} {s.max_depth:>9}
{s.repeated:>9,} {s.time*1000:>9.2f} {str(ok):>11}")
        D.append(dict(T=T, q=name, sol=s.solutions, goals=s.goals, unif=s.unifications, depth=s.max_depth, repeated=s.repeated,
time=s.time, ok=ok))
out['bc'] = D
print("\nD  DIRECTION OF INFERENCE: work for ONE query (ground query Q1) versus the whole forward-chaining closure")
print(f"{'base facts':>10} | {'FC firings':>11} {'FC unifications':>15} {'FC time s':>9} | {'BC goals':>8} {'BC unif.':>9} {'BC time
ms':>10} | {'unification ratio FC/BC':>24} {'queries before FC pays off (time)':>34}")
E = []
for T in SIZES:
    fs = [b for b in B if b['T'] == T and b['semi']][0]; q1 = [d for d in D if d['T'] == T and d['q'].startswith('Q1')][0]
    r = fs['unif'] / q1['unif']; be = fs['time'] / q1['time']
    print(f"{fs['facts']:>10,} | {fs['firings']:>11,} {fs['unif']:>15,} {fs['time']:>9.2f} | {q1['goals']:>8,} {q1['unif']:>9,}
{q1['time']*1000:>10.3f} | {r:>24,.0f} {be:>34,.0f}")
    E.append(dict(T=T, ratio=r, breakeven=be))
out['direction'] = E


# ------------------------------------------------------------------ left recursion
print("\nE  LEFT-RECURSIVE RULE:  AboveL(x,y) :- AboveL(x,z), ReportsTo(y,z)   (+ base AboveL(x,y) :- ReportsTo(y,x));  query AboveL(e0,
?y) on the 1,000-fact KB")
facts, n, _ = META[1000]; kbL = KB(LEFT, facts, index2=True); true_all = {('AboveL', 'e0', emp(i)) for i in range(1, n)}
p = Prover(kbL, 'plain', budget=200000);
try: a = p.ask(('AboveL', 'e0', '?y')); res = f"returned {len(a)} answers"
except RecursionError: res = "RecursionError (Python stack exhausted by the infinite recursion)"
print(f"   plain backward chaining: {res}; goals expanded before the 200,000-goal budget stopped it: {p.stat.goals:,}; maximum depth
reached: {p.stat.max_depth:,}; budget exhausted: {p.stat.exhausted}; answers produced: {p.stat.solutions}")
print(f"   (answers do appear, because the base clause is tried first, but the search never finishes: after the base clause it descends
AboveL(e0,?z1), AboveL(e0,?z2), ... and every goal is a VARIANT of its parent, so no branch ever fails, and the program cannot know that it
has found all the answers)")
Lres = [dict(method='plain', goals=p.stat.goals, exhausted=p.stat.exhausted, answers=p.stat.solutions)]
print(f"   correct answer set has {len(true_all)} answers (e0 is above all {n-1} other employees)")
print("\n   Remedy 1 - loop check (cut a goal that is a variant of one of its ancestors):")
p = Prover(kbL, 'loopcheck'); a = p.ask(('AboveL', 'e0', '?y')); got = {x for x, _ in a}
print(f"      terminates; answers {len(got)} of {len(true_all)} correct -> {'COMPLETE' if got == true_all else 'INCOMPLETE'} (it cuts the
recursive branch, so only the base clause answers: direct reports);  goals {p.stat.goals}, unifications {p.stat.unifications}, cut branches
{p.stat.cut}")
Lres.append(dict(method='loopcheck', goals=p.stat.goals, answers=len(got), complete=got == true_all))
print("\n   Remedy 2 - depth bound (limit L), re-run for increasing L (iterative deepening); a bound gives termination, and completeness
only when L is large enough; there is no way to know L in advance")
print(f"      {'L':>3} {'answers':>8} {'goals':>10} {'unifications':>13} {'cut':>8} {'complete':>9}")
IDS = []; tot_goals = 0; tot_unif = 0
for Lm in range(1, 12):
    p = Prover(kbL, 'depth', depth_limit=Lm, budget=5_000_000); a = p.ask(('AboveL', 'e0', '?y')); got = {x for x, _ in a}; tot_goals +=
p.stat.goals; tot_unif += p.stat.unifications
    print(f"      {Lm:>3} {len(got):>8} {p.stat.goals:>10,} {p.stat.unifications:>13,} {p.stat.cut:>8,} {str(got == true_all):>9}")
    IDS.append(dict(L=Lm, answers=len(got), goals=p.stat.goals, unif=p.stat.unifications, complete=got == true_all))
    if got == true_all and Lm >= 8: break
print(f"      cumulative cost of iterative deepening up to the last L: goals {tot_goals:,}, unifications {tot_unif:,}")
Lres.append(dict(method='depth', runs=IDS, cum_goals=tot_goals, cum_unif=tot_unif))


class Tabled:
    """memoisation (tabling) for definite clauses: one table per call VARIANT; evaluation is repeated until no table gains an answer"""
    def __init__(s, kb): s.kb = kb; s.tab = {}; s.goals = 0; s.unif = 0; s.passes = 0; s.hits = 0
    def call(s, goal):
        key = variant_key(goal, {}); s.goals += 1
        if key in s.vis: s.hits += 1; return list(s.tab[key])
        s.vis.add(key); ans = s.tab.setdefault(key, set())
        for f in s.kb.candidates(goal):
            s.unif += 1; th = unify(goal, f, {}, False)
            if th is not None and subst(goal, th) not in ans: ans.add(subst(goal, th)); s.changed = True
        for rule in s.kb.rules_by_pred.get(goal[0], []):
            h, body = rename_rule(rule); s.unif += 1; th = unify(goal, h, {}, False)
            if th is None: continue
            for th2 in s.body(body, th):
                a = subst(goal, th2)
                if a not in ans: ans.add(a); s.changed = True
        return list(ans)
    def body(s, body, th):
        if not body: yield th; return
        g = subst(body[0], th)
        for a in s.call(g):
            s.unif += 1; th1 = unify(g, a, th, False)
            if th1 is not None: yield from s.body(body[1:], th1)
    def ask(s, goal):
        while True:
            s.vis = set(); s.changed = False; s.passes += 1; ans = s.call(goal)
            if not s.changed: return ans
print("\n   Remedy 3 - memoisation / tabling (one table per call variant, evaluate until no table changes):")
print(f"      {'base facts':>10} {'answers':>8} {'complete':>9} {'passes':>7} {'tables':>7} {'goals':>9} {'unifications':>13} {'table
hits':>10} {'time ms':>9}")
TB = []
for T in SIZES:
    facts, n, _ = META[T]; k = KB(LEFT, facts, index2=True); tb = Tabled(k); t0 = time.perf_counter(); a = tb.ask(('AboveL', 'e0', '?y'));
dt = time.perf_counter() - t0
    tru = {('AboveL', 'e0', emp(i)) for i in range(1, n)}
    print(f"      {len(facts):>10,} {len(a):>8,} {str(set(a) == tru):>9} {tb.passes:>7} {len(tb.tab):>7,} {tb.goals:>9,} {tb.unif:>13,}
{tb.hits:>10,} {dt*1000:>9.1f}")
    TB.append(dict(T=T, answers=len(a), complete=set(a) == tru, passes=tb.passes, tables=len(tb.tab), goals=tb.goals, unif=tb.unif,
time=dt))
Lres.append(dict(method='tabling', rows=TB)); out['left'] = Lres


print("\nF  REPEATED SUBGOALS AND CYCLES (right-recursive Above, r1-r2)")
print("   F1 cyclic data: ReportsTo(a,b), ReportsTo(b,a), ReportsTo(c,b);  query Above(z, c) with z not in the data (fails):")
cyc = [('ReportsTo', 'a', 'b'), ('ReportsTo', 'b', 'a'), ('ReportsTo', 'c', 'b')]
kc = KB(RULES[:2], cyc, index2=True); p = Prover(kc, 'plain', budget=3000)
try: a = p.ask(('Above', 'z', 'c')); note = ''
except RecursionError: note = ' (Python RecursionError first)'
print(f"      plain BC: goals {p.stat.goals:,}, max depth {p.stat.max_depth:,}, exhausted budget: {p.stat.exhausted}{note}  (does not
terminate)")
```

```
p2 = Prover(kc, 'loopcheck'); a2 = p2.ask(('Above', 'a', 'c')); p3 = Prover(kc, 'loopcheck'); a3_ = p3.ask(('Above', 'z', 'c')); print(f"
loop check on the failing query Above(z, c): terminates, answers {len(a3_)}, goals {p3.stat.goals}, cut {p3.stat.cut}"); print(f"      loop
check on Above(a, c) [true]: answers {len(a2)}, goals {p2.stat.goals}, cut {p2.stat.cut}")
kf = KB(RULES[:2], cyc, index2=True); sf = forward_chain(kf); print(f"      forward chaining on the same cyclic data: terminates,
{sf.new_facts} Above facts, {sf.iterations} iterations (the fixed point is finite because facts are never repeated)")
print("   F2 matrix organisation (each employee has a second, dotted-line manager) -> the reports-to graph is a DAG; query Above(e_k, leaf)
that FAILS, so every path is explored:")
print(f"      {'facts':>6} {'mode':>10} {'goals':>10} {'unifications':>13} {'repeated':>10} {'repeated %':>10}")
M = []
for T in (100, 1000, 10000):
    facts, n = make_facts(T, dag=True); k = KB(RULES[:2], facts, index2=True); L = n - 1
    for mode in ('plain', 'tabled'):
        if mode == 'plain': p = Prover(k, 'plain', budget=3_000_000); p.ask(('Above', 'e2', emp(L))); g, u, r = p.stat.goals,
p.stat.unifications, p.stat.repeated; ex = p.stat.exhausted
        else: tb = Tabled(k); tb.ask(('Above', 'e2', emp(L))); g, u, r = tb.goals, tb.unif, tb.hits; ex = False
        print(f"      {len(facts):>6,} {mode:>10} {g:>10,} {u:>13,} {r:>10,} {100*r/max(g,1):>9.1f}%" + ('  (budget hit)' if ex else ''))
        M.append(dict(T=T, mode=mode, goals=g, unif=u, repeated=r))
out['dag'] = M
json.dump(out, open('t6_results.json', 'w'), indent=1, default=str)
```

A.15 Task 7 experiment driver (37 tests, scaling, baselines) (t7_run.py)

```
import json, time, random, tracemalloc, statistics, copy
from med_kb import *
out = {}
def pat(age, rx, allergies=(), takes=(), conds=()): return dict(age=age, rx=rx, allergies=list(allergies), takes=list(takes),
conds=list(conds))
def facts_of(pid, p):
    f = [('Age', pid, p['age']), ('Rx', pid, p['rx'])] + [('Allergy', pid, a) for a in p['allergies']] + [('Takes', pid, t) for t in
p['takes']] + [('Condition', pid, c) for c in p['conds']]
    return f
# expected verdicts written from the clinical specification BEFORE running the program
TESTS = [
 ('T01 warfarin + aspirin (dangerous combination)', pat(60, 'warfarin', takes=['aspirin']), {'interaction:warfarin+aspirin'}),
 ('T02 aspirin prescribed, patient on warfarin (reverse order)', pat(60, 'aspirin', takes=['warfarin']), {'interaction:aspirin+warfarin'}),
 ('T03 ibuprofen prescribed, patient on warfarin', pat(55, 'ibuprofen', takes=['warfarin']), {'interaction:ibuprofen+warfarin'}),
 ('T04 sertraline + tramadol (serotonin syndrome)', pat(40, 'sertraline', takes=['tramadol']), {'interaction:sertraline+tramadol'}),
 ('T05 tramadol prescribed, patient on sertraline', pat(40, 'tramadol', takes=['sertraline']), {'interaction:tramadol+sertraline'}),
 ('T06 sertraline + phenelzine (MAOI)', pat(35, 'sertraline', takes=['phenelzine']), {'interaction:sertraline+phenelzine'}),
 ('T07 simvastatin + clarithromycin', pat(65, 'simvastatin', takes=['clarithromycin']), {'interaction:simvastatin+clarithromycin'}),
 ('T08 sildenafil + nitroglycerin', pat(58, 'sildenafil', takes=['nitroglycerin']), {'interaction:sildenafil+nitroglycerin'}),
 ('T09 nitroglycerin prescribed, patient on sildenafil', pat(58, 'nitroglycerin', takes=['sildenafil']),
{'interaction:nitroglycerin+sildenafil'}),
 ('T10 lisinopril + spironolactone (hyperkalaemia)', pat(70, 'lisinopril', takes=['spironolactone']),
{'interaction:lisinopril+spironolactone'}),
 ('T11 spironolactone prescribed, patient on lisinopril', pat(70, 'spironolactone', takes=['lisinopril']),
{'interaction:spironolactone+lisinopril'}),
 ('T12 methotrexate + trimethoprim', pat(50, 'methotrexate', takes=['trimethoprim']), {'interaction:methotrexate+trimethoprim'}),
 ('T13 trimethoprim prescribed, patient on methotrexate', pat(50, 'trimethoprim', takes=['methotrexate']),
{'interaction:trimethoprim+methotrexate'}),
 ('T14 amoxicillin, penicillin allergy', pat(30, 'amoxicillin', allergies=['penicillin']), {'allergy:penicillin:amoxicillin'}),
 ('T15 cefalexin, penicillin allergy (cross-reactivity)', pat(30, 'cefalexin', allergies=['penicillin']),
{'allergy:penicillin:cefalexin'}),
 ('T16 sulfamethoxazole, sulfa allergy', pat(45, 'sulfamethoxazole', allergies=['sulfa']), {'allergy:sulfa:sulfamethoxazole'}),
 ('T17 ibuprofen, NSAID allergy', pat(45, 'ibuprofen', allergies=['nsaid']), {'allergy:nsaid:ibuprofen'}),
 ('T18 diclofenac, NSAID allergy', pat(45, 'diclofenac', allergies=['nsaid']), {'allergy:nsaid:diclofenac'}),
 ('T19 ibuprofen, kidney disease', pat(62, 'ibuprofen', conds=['kidney_disease']), {'contra:ibuprofen:kidney_disease'}),
 ('T20 metformin, kidney disease', pat(62, 'metformin', conds=['kidney_disease']), {'contra:metformin:kidney_disease'}),
 ('T21 lisinopril, pregnancy', pat(28, 'lisinopril', conds=['pregnancy']), {'contra:lisinopril:pregnancy'}),
 ('T22 warfarin, pregnancy', pat(28, 'warfarin', conds=['pregnancy']), {'contra:warfarin:pregnancy'}),
 ('T23 aspirin, child aged 8', pat(8, 'aspirin'), {'age:child:aspirin'}),
 ('T24 aspirin, boundary age 12 (not a child)', pat(12, 'aspirin'), set()),
 ('T25 tramadol, boundary age 11 (child)', pat(11, 'tramadol'), {'age:child:tramadol'}),
 ('T26 diazepam, age 80', pat(80, 'diazepam'), {'age:elderly:diazepam'}),
 ('T27 diazepam, boundary age 74 (not elderly)', pat(74, 'diazepam'), set()),
 ('T28 diazepam, boundary age 75 (elderly)', pat(75, 'diazepam'), {'age:elderly:diazepam'}),
 ('T29 warfarin + aspirin + pregnancy (two reasons)', pat(30, 'warfarin', takes=['aspirin'], conds=['pregnancy']),
{'interaction:warfarin+aspirin', 'contra:warfarin:pregnancy'}),
 ('T30 aspirin, child 9, NSAID allergy, on warfarin, ulcer (four reasons)', pat(9, 'aspirin', allergies=['nsaid'], takes=['warfarin'],
conds=['ulcer']), {'age:child:aspirin', 'allergy:nsaid:aspirin', 'interaction:aspirin+warfarin', 'contra:aspirin:ulcer'}),
 ('T31 ibuprofen, patient on naproxen (duplicate NSAID)', pat(40, 'ibuprofen', takes=['naproxen']), {'duplicate:nsaid'}),
 ('T32 ibuprofen, patient already on ibuprofen', pat(40, 'ibuprofen', takes=['ibuprofen']), {'duplicate:nsaid'}),
 ('T33 simvastatin, liver disease, on clarithromycin (two reasons)', pat(66, 'simvastatin', takes=['clarithromycin'],
conds=['liver_disease']), {'interaction:simvastatin+clarithromycin', 'contra:simvastatin:liver_disease'}),
 ('T34 paracetamol, full risky history (safe: no rule applies)', pat(50, 'paracetamol', allergies=['penicillin'], takes=['warfarin'],
conds=['ulcer']), set()),
 ('T35 amoxicillin, no allergy (safe)', pat(40, 'amoxicillin'), set()),
 ('T36 amoxicillin, sulfa allergy only (wrong allergen, safe)', pat(40, 'amoxicillin', allergies=['sulfa']), set()),
 ('T37 metformin, patient on ibuprofen, no condition (safe)', pat(50, 'metformin', takes=['ibuprofen']), set()),
]
print("TASK 7 (Student B) - EXPERIMENTAL VALIDATION OF THE MEDICATION-SAFETY ADVISOR\n")
def run_tests(keyed):
    adv = Advisor(keyed, [f for i, (_, p, _) in enumerate(TESTS) for f in facts_of(f'P{i+1}', p)]); rows = []
    for i, (lab, p, exp) in enumerate(TESTS):
        t0 = time.perf_counter(); got, proofs, st = adv.check(f'P{i+1}', p['rx']); dt = time.perf_counter() - t0
        rows.append((lab, exp, got, got == exp, st.goals, st.unifications, sum(proof_size(x) for x in proofs), dt, proofs))
    return rows
print("A  FIRST DRAFT KB (39 rules): 37 test cases")
d = run_tests(base_rules(draft=True)); nd = sum(r[3] for r in d)
for r in d:
    if not r[3]: print(f"   FAIL {r[0]}: expected {sorted(r[1])}, got {sorted(r[2])}")
print(f"   -> {nd}/{len(d)} pass")
print("   DEFECTS FOUND: (1) T11 and T13 - the two one-directional interaction rules (lisinopril->spironolactone, methotrexate-
>trimethoprim) missed the reverse order; fix: add the two mirrored rules.   (2) T18 - NSAID allergy list had no diclofenac although
diclofenac is an NSAID fact; fix: add allergy rule (nsaid, diclofenac).")
print("\nB  FINAL KB: 37 test cases with per-case evidence")
FK = base_rules(); f = run_tests(FK); nf = sum(r[3] for r in f)
print(f"   rules in the final KB: {len(HELPERS) + len(GENERAL) + len(FK)} ({len(FK)} drug-triggered + {len(GENERAL)} general +
{len(HELPERS)} helper/summary)")
print(f"   {'case':70s} {'ok':>4} {'goals':>6} {'unif':>5} {'expl. nodes':>11} {'us':>7}")
for r in f: print(f"   {r[0][:70]:70s} {'PASS' if r[3] else 'FAIL':>4} {r[4]:>6} {r[5]:>5} {r[6]:>11} {r[7]*1e6:>7.0f}")
print(f"   -> {nf}/{len(f)} pass; dangerous cases {sum(1 for r in f if r[1])}, safe cases {sum(1 for r in f if not r[1])}")
print("\n   Example explanation (T30, four independent reasons):")
for pr in f[29][8][:2]:
    for l in proof_lines(pr): print('      ' + l)
```

```
out['tests'] = dict(draft_pass=nd, final_pass=nf, n=len(f), rows=[dict(case=r[0], ok=r[3], goals=r[4], unif=r[5], expl=r[6], us=r[7] * 1e6)
for r in f])
# ---------------------------------------------------------------- synthetic patients
def make_patients(n, keyed, drugs, allerg, conds, seed=11, hit=0.4):
    rnd = random.Random(seed); pool_drugs = [d for d in drugs] + ['warfarin', 'aspirin', 'ibuprofen', 'paracetamol', 'metformin']; res = []
    for i in range(n):
        p = pat(rnd.randint(1, 95), rnd.choice(pool_drugs), [rnd.choice(allerg) for _ in range(rnd.randint(0, 2))], [rnd.choice(pool_drugs)
for _ in range(rnd.randint(0, 5))], [rnd.choice(conds) for _ in range(rnd.randint(0, 2))])
        if rnd.random() < hit:
            d, r = rnd.choice(keyed); p['rx'] = d; lit = r[1][1]
            if lit[0] == 'Allergy': p['allergies'].append(lit[2])
            elif lit[0] == 'Takes': p['takes'].append(lit[2])
            elif lit[0] == 'Condition': p['conds'].append(lit[2])
            elif lit[0] == 'Child': p['age'] = 8
            elif lit[0] == 'Elderly': p['age'] = 80
        res.append(p)
    return res
def pct(xs, q): xs = sorted(xs); return xs[min(len(xs) - 1, int(q * len(xs)))]
print("\nC  KB-SIZE SCALING: 2,000 patients, one prescription each; indexed (rules by trigger drug) versus un-indexed (every Danger rule is
tried)")
print(f"{'rules':>6} {'mode':>9} | {'goals/q':>8} {'unif/q':>8} {'rule tries/q':>12} {'mean us':>8} {'p95 us':>8} {'explain nodes
(dangerous)':>25} {'dangerous %':>11} | {'KB build s':>10} {'KB mem MB':>9} {'query peak KB':>13} | {'agree':>6}")
S = []
for total in (42, 400, 2000):
    if total == 42: keyed = base_rules(); drugs = ['X0', 'X1']; allerg = ['penicillin', 'sulfa', 'nsaid']; conds = ['ulcer',
'kidney_disease', 'pregnancy']
    else: keyed, drugs, allerg, conds = grown_rules(total); keyed = list(keyed)
    pats = make_patients(2000, keyed, drugs, allerg, conds); facts = [f for i, p in enumerate(pats) for f in facts_of(f'P{i}', p)]
    res = {}
    for mode in (True, False):
        tracemalloc.start(); t0 = time.perf_counter(); adv = Advisor(keyed, facts, indexed=mode)
        if mode:                                                    # build every per-drug index eagerly (the cost belongs to the
build)
            for dname in list(adv.by_drug): adv._kb_for(dname)
        tb = time.perf_counter() - t0; mem = tracemalloc.get_traced_memory()[1] / 1e6; tracemalloc.stop()
        G = []; U = []; T = []; Ex = []; RT = []; dang = 0; answers = []; peaks = []
        for i, p in enumerate(pats):
            t0 = time.perf_counter(); got, proofs, st = adv.check(f'P{i}', p['rx']); T.append((time.perf_counter() - t0) * 1e6)
            G.append(st.goals); U.append(st.unifications); RT.append(st.rule_tries); answers.append(got)
            if got: dang += 1; Ex.append(sum(proof_size(x) for x in proofs))
        tracemalloc.start()
        for i in range(0, 2000, 50): adv.check(f'P{i}', pats[i]['rx'])
        pk = tracemalloc.get_traced_memory()[1] / 1024 / 20 if False else tracemalloc.get_traced_memory()[1] / 1024; tracemalloc.stop()
        res[mode] = answers
        agree = '' if mode else str(res[True] == res[False])
        print(f"{total:>6} {'indexed' if mode else 'unindexed':>9} | {statistics.mean(G):>8.1f} {statistics.mean(U):>8.1f}
{statistics.mean(RT):>12.1f} {statistics.mean(T):>8.0f} {pct(T, .95):>8.0f} {statistics.mean(Ex) if Ex else 0:>25.1f}
{100*dang/2000:>10.1f}% | {tb:>10.2f} {mem:>9.1f} {pk:>13.0f} | {agree:>6}")
        S.append(dict(rules=total, indexed=mode, goals=statistics.mean(G), unif=statistics.mean(U), rt=statistics.mean(RT),
mean_us=statistics.mean(T), p95_us=pct(T, .95), expl=statistics.mean(Ex) if Ex else 0, dangerous=dang, build=tb, mem=mem, peak_kb=pk))
    # forward-chaining baseline (bulk)
    t0 = time.perf_counter(); kbf = KB(HELPERS + GENERAL + [r for _, r in keyed], BASE_FACTS + facts, index2=True); st = forward_chain(kbf,
seminaive=True); tf = time.perf_counter() - t0
    fc_d = {}
    for f_ in kbf.facts_by_pred.get('Danger', []): fc_d.setdefault(f_[1], set()).add(f_[2])
    ok = all(fc_d.get(f'P{i}', set()) == res[True][i] for i in range(2000))
    print(f"{total:>6} {'FC (all)':>9} | firings {st.firings:,}  unifications {st.unifications:,}  facts inferred {st.new_facts:,}
iterations {st.iterations}  total time {tf:.2f}s = {tf/2000*1e6:,.0f} us per patient (amortised over all 2,000 patients, answers agree with
BC: {ok})")
    S.append(dict(rules=total, mode='fc', firings=st.firings, unif=st.unifications, time=tf, agree=ok))
out['scaling'] = S
# ---------------------------------------------------------------- propositional footprint and 50,000 patients
print("\nD  SCALE TEST: 2,000-rule KB and 50,000 patients (about 6 facts each)")
keyed, drugs, allerg, conds = grown_rules(2000); keyed = list(keyed)
tracemalloc.start(); t0 = time.perf_counter(); pats = make_patients(50000, keyed, drugs, allerg, conds, seed=21)
facts = [f for i, p in enumerate(pats) for f in facts_of(f'P{i}', p)]; adv = Advisor(keyed, facts, indexed=True)
for dname in list(adv.by_drug): adv._kb_for(dname)
tb = time.perf_counter() - t0; mem = tracemalloc.get_traced_memory()[1] / 1e6; tracemalloc.stop()
rnd = random.Random(3); sample = rnd.sample(range(50000), 3000); T = []; G = []; dang = 0
for i in sample:
    t0 = time.perf_counter(); got, proofs, st = adv.check(f'P{i}', pats[i]['rx']); T.append((time.perf_counter() - t0) * 1e6);
G.append(st.goals + st.unifications); dang += bool(got)
print(f"   facts {len(facts):,}; build {tb:.2f}s; memory {mem:.0f} MB;  3,000 random queries: mean {statistics.mean(T):.0f} us, p95
{pct(T,.95):.0f} us, max {max(T):.0f} us, mean steps (goals+unifications) {statistics.mean(G):.1f}, dangerous {100*dang/3000:.1f}%")
t0 = time.perf_counter(); kbf = KB(HELPERS + GENERAL + [r for _, r in keyed], BASE_FACTS + facts, index2=True); st = forward_chain(kbf,
seminaive=True); tf = time.perf_counter() - t0
print(f"   forward chaining over all 50,000 patients (baseline): {tf:.1f}s, firings {st.firings:,}, unifications {st.unifications:,}, facts
inferred {st.new_facts:,};  one new patient admitted -> BC {statistics.mean(T):.0f} us versus FC re-run unless made incremental")
print(f"   propositional encoding (one copy of every rule per patient) would need 50,000 x 2,000 = {50000*2000:,} ground rules; the first-
order rules stay at 2,000")
out['scale'] = dict(facts=len(facts), build=tb, mem=mem, mean_us=statistics.mean(T), p95=pct(T, .95), max=max(T), steps=statistics.mean(G),
fc_time=tf, fc_firings=st.firings, fc_unif=st.unifications)
json.dump(out, open('t7_results.json', 'w'), indent=1, default=str)
```

Appendix B – Complete Console Output of Each Task

B.1 Task 1 (t1_output.txt)

```
TASK 1 - Wumpus world, propositional KB and model checking


A  SYMBOL AND SENTENCE COUNTS (derived formulas: symbols = 4n^2;  sentences = 2 + 2n^2 + 1 + n^2(n^2-1)/2  (+ percept sentences))
  n  symbols m  formula  sentences  formula |                 models 2^m
  2         16       16         17       17 |                     65,536
  3         36       36         57       57 |             68,719,476,736
  4         64       64        155      155 | 18,446,744,073,709,551,616
  5        100      100        353      353 | 1,267,650,600,228,229,401,496,703,205,376
  6        144      144        705      705 | 22,300,745,198,530,623,141,535,718,272,648,361,505,980,416


B  MODEL CHECKING, FULL SYMBOL SET vs RELEVANT-SYMBOL RESTRICTION
 n query                                    method     symbols  models enumerated sentence evals    time s |  KB|=q  KB|=~q verdict
 2 entailed: [1,2] has no pit               full            16             65,634        131,688    0.0636 |   True   False ENTAILED
 2 entailed: [1,2] has no pit               relevant         8                257            512    0.0002 |   True   False ENTAILED
 2 negation entailed: pit at [2,1]          full            16             65,634        131,688    0.0622 |  False    True NEGATION ENTAILED
 2 negation entailed: pit at [2,1]          relevant         8                257            512    0.0002 |  False    True NEGATION ENTAILED
 2 neither: pit at [2,2]                    full            16             25,028         47,591    0.0222 |  False   False NEITHER
 2 neither: pit at [2,2]                    relevant         9                 18             77    0.0000 |  False   False NEITHER
 3 entailed: pit at [3,1]                   full            36 2^36 = 68,719,476,736              -  ~6.1e+04 | infeasible (rate extrapolation)
 3 entailed: pit at [3,1]                   relevant        18            295,561        586,960    0.2814 |   True   False ENTAILED
 3 negation entailed: pit at [2,2]          full            36 2^36 = 68,719,476,736              -  ~6.1e+04 | infeasible (rate extrapolation)
 3 negation entailed: pit at [2,2]          relevant        18            295,561        586,960    0.2821 |  False    True NEGATION ENTAILED
 3 neither: pit at [3,3]                    full            36 2^36 = 68,719,476,736              -  ~6.1e+04 | infeasible (rate extrapolation)
 3 neither: pit at [3,3]                    relevant        19            133,906        254,201    0.1251 |  False   False NEITHER
 3 entailed: [2,2] is safe (no pit, no wumpus)  full        36 2^36 = 68,719,476,736              -  ~6.1e+04 | infeasible (rate extrapolation)
 3 entailed: [2,2] is safe (no pit, no wumpus)  relevant    18            295,561        586,960    0.2786 |   True   False ENTAILED
 4 entailed: pit at [3,1]                   full            64 2^64 = 18,446,744,073,709,551,616              -  ~1.6e+13 | infeasible (rate
extrapolation)
 4 entailed: pit at [3,1]                   relevant        18            295,561        586,960    0.3292 |   True   False ENTAILED
 4 negation entailed: pit at [2,2]          full            64 2^64 = 18,446,744,073,709,551,616              -  ~1.6e+13 | infeasible (rate
extrapolation)
 4 negation entailed: pit at [2,2]          relevant        18            295,561        586,960    0.2976 |  False    True NEGATION ENTAILED
 4 neither: pit at [3,3]                    full            64 2^64 = 18,446,744,073,709,551,616              -  ~1.6e+13 | infeasible (rate
extrapolation)
 4 neither: pit at [3,3]                    relevant        19            133,906        254,201    0.1277 |  False   False NEITHER
 4 entailed: [2,2] is safe (no pit, no wumpus)  full        64 2^64 = 18,446,744,073,709,551,616              -  ~1.6e+13 | infeasible (rate
extrapolation)
 4 entailed: [2,2] is safe (no pit, no wumpus)  relevant    18            295,561        586,960    0.2944 |   True   False ENTAILED


C  INDEPENDENT CROSS-CHECK on the FULL KB with a DPLL satisfiability solver (KB & ~q unsatisfiable <=> KB |= q); n=3, 4 have 36 / 64 symbols
  n=3  entailed: pit at [3,1]                       full-KB verdict ENTAILED           relevant-symbol verdict ENTAILED           agree: True
(0.00s)
  n=3  negation entailed: pit at [2,2]              full-KB verdict NEGATION ENTAILED  relevant-symbol verdict NEGATION ENTAILED  agree: True
(0.00s)
  n=3  neither: pit at [3,3]                        full-KB verdict NEITHER            relevant-symbol verdict NEITHER            agree: True
(0.00s)
  n=3  entailed: [2,2] is safe (no pit, no wumpus)  full-KB verdict ENTAILED           relevant-symbol verdict ENTAILED           agree: True
(0.00s)
  n=4  entailed: pit at [3,1]                       full-KB verdict ENTAILED           relevant-symbol verdict ENTAILED           agree: True
(0.00s)
  n=4  negation entailed: pit at [2,2]              full-KB verdict NEGATION ENTAILED  relevant-symbol verdict NEGATION ENTAILED  agree: True
(0.00s)
  n=4  neither: pit at [3,3]                        full-KB verdict NEITHER            relevant-symbol verdict NEITHER            agree: True
(0.00s)
  n=4  entailed: [2,2] is safe (no pit, no wumpus)  full-KB verdict ENTAILED           relevant-symbol verdict ENTAILED           agree: True
(0.00s)


D  PART B - INFERENCE-RULE DERIVATIONS (n=4, same KB and percepts)


  Goal: [2,2] is safe  ->  derived: True;  inference steps needed = 8
     1. Biconditional-Elimination  (B12 <=> ((P22 | P11) | P13))  gives  ((B12 => ((P22 | P11) | P13)) & (((P22 | P11) | P13) => B12))
     2. And-Elimination            ((B12 => ((P22 | P11) | P13)) & (((P22 | P11) | P13) => B12))  gives  (((P22 | P11) | P13) => B12)
     3. And-Elimination            split (((P22 | P11) | P13) => B12) (as clauses) gives {-P22, B12}
     4. Biconditional-Elimination  (S21 <=> ((W11 | W31) | W22))  gives  ((S21 => ((W11 | W31) | W22)) & (((W11 | W31) | W22) => S21))
     5. And-Elimination            ((S21 => ((W11 | W31) | W22)) & (((W11 | W31) | W22) => S21))  gives  (((W11 | W31) | W22) => S21)
     6. And-Elimination            split (((W11 | W31) | W22) => S21) (as clauses) gives {-W22, S21}
     7. Unit Resolution            {-P22, B12} with -B12  gives  {-P22}
     8. Unit Resolution            {-W22, S21} with -S21  gives  {-W22}
    -> 8 inference steps   versus   model checking (18 relevant symbols): 262,144 models, 523,661 sentence evaluations   (ratio 65,458x)


  Goal: [1,2] and [2,1] have no pit  ->  derived: True;  inference steps needed = 6
     1. Biconditional-Elimination  (B11 <=> (P21 | P12))  gives  ((B11 => (P21 | P12)) & ((P21 | P12) => B11))
     2. And-Elimination            ((B11 => (P21 | P12)) & ((P21 | P12) => B11))  gives  ((P21 | P12) => B11)
     3. And-Elimination            split ((P21 | P12) => B11) (as clauses) gives {-P21, B11}
     4. And-Elimination            split ((P21 | P12) => B11) (as clauses) gives {-P12, B11}
     5. Unit Resolution            {-P21, B11} with -B11  gives  {-P21}
     6. Unit Resolution            {-P12, B11} with -B11  gives  {-P12}
    -> 6 inference steps   versus   model checking (18 relevant symbols): 262,144 models, 523,661 sentence evaluations   (ratio 87,277x)


  Goal: pit at [3,1]  ->  derived: True;  inference steps needed = 9
     1. Biconditional-Elimination  (B12 <=> ((P22 | P11) | P13))  gives  ((B12 => ((P22 | P11) | P13)) & (((P22 | P11) | P13) => B12))
     2. And-Elimination            ((B12 => ((P22 | P11) | P13)) & (((P22 | P11) | P13) => B12))  gives  (((P22 | P11) | P13) => B12)
     3. And-Elimination            split (((P22 | P11) | P13) => B12) (as clauses) gives {-P22, B12}
     4. Biconditional-Elimination  (B21 <=> ((P11 | P31) | P22))  gives  ((B21 => ((P11 | P31) | P22)) & (((P11 | P31) | P22) => B21))
     5. And-Elimination            ((B21 => ((P11 | P31) | P22)) & (((P11 | P31) | P22) => B21))  gives  (B21 => ((P11 | P31) | P22))
     6. Unit Resolution            {-P22, B12} with -B12  gives  {-P22}
     7. Modus Ponens               {-B21, P11, P22, P31} with B21  gives  {P11, P22, P31}
     8. Unit Resolution            {P11, P22, P31} with -P11  gives  {P22, P31}
     9. Unit Resolution            {P22, P31} with -P22  gives  {P31}
    -> 9 inference steps   versus   model checking (18 relevant symbols): 262,144 models, 523,661 sentence evaluations   (ratio 58,185x)
```

B.2 Task 2 (t2_output.txt)

```
TASK 2 - Model checking versus resolution


A  MANUAL CNF CONVERSION, stage by stage (the hand derivations in the report are checked against these outputs)
  R1 : (BadgeValid <=> ~BadgeExpired)
```

```
      1 eliminate <=>:  ((BadgeValid => ~BadgeExpired) & (~BadgeExpired => BadgeValid))
      2 eliminate =>:   ((~BadgeValid | ~BadgeExpired) & (~~BadgeExpired | BadgeValid))
      3 push ~ inward:  ((~BadgeValid | ~BadgeExpired) & (BadgeExpired | BadgeValid))
      4 distribute | over &:  {-BadgeExpired, -BadgeValid}  &  {BadgeExpired, BadgeValid}
  R5 : (((Employee & BadgeValid) & WorkHours) => MayEnter)
      1 eliminate <=>:  (((Employee & BadgeValid) & WorkHours) => MayEnter)
      2 eliminate =>:   (~((Employee & BadgeValid) & WorkHours) | MayEnter)
      3 push ~ inward:  (((~Employee | ~BadgeValid) | ~WorkHours) | MayEnter)
      4 distribute | over &:  {-BadgeValid, -Employee, -WorkHours, MayEnter}
  R7 : (((AfterHours & ~Emergency) & ~Security) => ~MayEnter)
      1 eliminate <=>:  (((AfterHours & ~Emergency) & ~Security) => ~MayEnter)
      2 eliminate =>:   (~((AfterHours & ~Emergency) & ~Security) | ~MayEnter)
      3 push ~ inward:  (((~AfterHours | Emergency) | Security) | ~MayEnter)
      4 distribute | over &:  {-AfterHours, -MayEnter, Emergency, Security}


B  THE 12-RULE POLICY: six designed cases, resolution and model checking
case                                                          |   models res: gen attempted  taut  subs | verdict MC / RES
C1 employee, valid badge, work hours  |=  DoorUnlocked          |   16,384       94       534    56   196 | True / True
C2 visitor, unescorted, work hours    |=  MayEnter ?           |    1,028       20       188    30    70 | False / False   (also ~query not
entailed: the KB leaves it OPEN)
C3 after hours, no emergency, not security |= ~DoorUnlocked      |   16,384       77       476   129   156 | True / True
C4 emergency |= DoorUnlocked & Alarm                           |   16,384       64       704   215   207 | True / True
C5 server-room request by a visitor |= false (inconsistent)     |   16,384       26       129    44    36 | True / True   (KB+facts inconsistent:
entails everything)
C6 contractor, unescorted, work hours |= ~MayEnter              |   16,384       41       301    61   106 | True / True


   Resolution proof of case C1 (clause, parents):
      1. {BadgeExpired, BadgeValid}                          given
      2. {-BadgeExpired}                                     given
      3. {BadgeValid}                                        resolve 1 and 2 on BadgeExpired
      4. {-BadgeValid, -Employee, -WorkHours, MayEnter}      given
      5. {Employee}                                          given
      6. {-BadgeValid, -WorkHours, MayEnter}                 resolve 4 and 5 on -Employee
      7. {-WorkHours, MayEnter}                              resolve 3 and 6 on BadgeValid
      8. {-MayEnter, DoorUnlocked}                           given
      9. {-DoorUnlocked}                                     given
     10. {-MayEnter}                                         resolve 8 and 9 on DoorUnlocked
     11. {-WorkHours}                                        resolve 7 and 10 on MayEnter
     12. {WorkHours}                                         given
     13. {}  (empty clause)                                  resolve 11 and 12 on -WorkHours


C  RESOLUTION AS THE KB GROWS: implication chains A0, A0=>A1, ..., A(n-1)=>An ; query An (entailed); NOT-entailed test: the same chain WITHOUT the fact
A0, query An
   n clauses | RES entailed: generated attempted  taut subsumed rounds   time s | RES not entailed: generated attempted terminates   time s |    MC
models MC time s
  10      11 |                      57       149     0       32      4    0.000 |                        55       205       True    0.000 |
2,048      0.00
  20      21 |                     214       952     0      118      5    0.002 |                       210     1,359       True    0.002 |
2,097,152      2.12
  40      41 |                     814     6,780     0      450      6    0.009 |                       820     9,919       True    0.013 | 2^41 =
2.2e+12 infeasible
  80      81 |                   3,166    51,204     0     1754      7    0.059 |                     3,240    75,887       True    0.082 | 2^81 =
2.4e+24 infeasible


   Same chains with and without subsumption (n = 40):
     subsumption True : generated    814, attempted   6,780, subsumed   450, time 0.008s, result True
     subsumption False: generated    816, attempted   8,537, subsumed     0, time 0.008s, result True


D  KB FAMILIES THAT FAVOUR EACH METHOD
   D1. Model checking time against the number of symbols m (chain KB, query entailed; every model enumerated: 2^m)
     m= 9: MC       512 models    0.000s     resolution    43 clauses   0.0002s
     m=13: MC     8,192 models    0.006s     resolution    76 clauses   0.0003s
     m=17: MC   131,072 models    0.129s     resolution   151 clauses   0.0009s
     m=19: MC   524,288 models    0.490s     resolution   181 clauses   0.0011s


   D2. Pigeonhole PHP(k+1, k): dense, unsatisfiable, few symbols.  KB entails every query; query = contradiction 'Z & ~Z'.
  k symbols m  clauses |  MC models MC time s |  RES generated  attempted  subsumed   time s   result
  2         7        9 |        128     0.000 |             64        540        48     0.00   proved
  3        13       22 |      8,192     0.014 |          5,738  7,479,819   111,978    13.24   proved
  4        21       45 |  2,097,152     5.525 |         62,578    955,397   148,819    60.03    LIMIT
  5        31       81 |       2^31 infeasible |        139,067    651,742   178,952    60.00    LIMIT


   D3. Horn chains with many symbols: resolution is polynomial, model checking is not even runnable beyond m = 25
     chain n=80 (m=81):  resolution 3,166 clauses in 0.059s;  model checking would need 2^81 = 2.42e+24 models; at the measured 0.93 us per model =
7.2e+10 years
```

B.3 Task 3 (t3_output.txt)

```
TASK 3 - University exam-eligibility advisor: knowledge engineering


A  THE FOL AXIOMS (14; * = executed as a definite clause with negation as failure)
   1*  Ax,c,p,m  Enrolled(s,c) & Attendance(s,c,p) & MinAtt(c,m) & Ge(p,m)  =>  AttendanceOK(s,c)
   2*  Ax,c  Enrolled(s,c) & ApprovedLeave(s,c)  =>  AttendanceOK(s,c)
   3*  Ax,c,g  Grade(s,c,g) & Ge(g,40)  =>  Passed(s,c)
   4*  Ac,p  Requires(c,p)  =>  Needs(c,p)
   5*  Ac,p,q  Requires(c,q) & Needs(q,p)  =>  Needs(c,p)
   6*  As,c  Enrolled(s,c) & Ep (Needs(c,p) & ~Passed(s,p))  =>  MissingPrereq(s,c)
   7*  As,c  Enrolled(s,c) & ~MissingPrereq(s,c)  =>  PrereqsCleared(s,c)
   8*  As  Student(s) & ~FeeDue(s)  =>  FeesCleared(s)
   9*  As  Ek (DisciplinaryCase(s,k) & Active(k))  =>  Barred(s)
   10  As  Barred(s)  =>  Ek (DisciplinaryCase(s,k) & Active(k))                    [existential: the 'only if' half of rule 9]
   11* As,c  AttendanceOK(s,c) & PrereqsCleared(s,c) & FeesCleared(s) & ~Barred(s)  =>  Eligible(s,c)
   12  As,c  Eligible(s,c)  =>  Enrolled(s,c)                                      [typing / sanity axiom]
   13  As,c  Passed(s,c)  =>  Ee Grade(s,c,e)                                      [existential: a pass has a recorded grade]
   14  Ac  Course(c)  =>  Ef Coordinator(c,f)                                      [existential: every course has a coordinator]


B  QUERY SUITE ON DRAFT 1 (the first version of the KB; expected answers were written from the regulations before running)
   PASS  S1 Eligible(alice, CS201)                        expected True                                                    got True
goals   21
   PASS  S2 Eligible(carol, CS201)  [approved leave]      expected True                                                    got True
goals   21
   PASS  S3 Eligible(hari, CS101)                         expected True                                                    got True
goals   18
   FAIL  S4 Eligible(gina, MA101)  [closed case]          expected True                                                    got False
goals   18
```

```
   PASS  S5 Passed(alice, CS101)                          expected True                                                          got True
goals    3
   PASS  F1 Eligible(bob, CS201)  [attendance]            expected False                                                         got False
goals    8
   FAIL  F2 Eligible(dave, CS301)  [indirect prerequisite]  expected False                                                       got True
goals   21
   PASS  F3 Eligible(erin, MA101)  [fees due]             expected False                                                         got False
goals   15
   PASS  F4 Eligible(frank, MA101)  [active case]         expected False                                                         got False
goals   18
   PASS  F5 Eligible(ida, CS101)  [not enrolled]          expected False                                                         got False
goals    4
   PASS  B1 Eligible(?s, CS201)                           expected ['alice', 'carol']                                            got ['alice',
'carol']                                      goals   42
   FAIL  B2 Needs(CS301, ?p)                              expected ['CS101', 'CS201']                                            got []
goals    1
   FAIL  B3 Barred(?s)                                    expected ['frank']                                                     got ['frank', 'gina']
goals    2
   FAIL  B4 FeesCleared(?s)                               expected ['alice', 'bob', 'carol', 'dave', 'frank', 'gina', 'hari', ' got ERROR
goals    1   [floundering: non-ground negative literal ('FeeDue', '?s_171')]
   FAIL  B5 Eligible(?s, MA101)                           expected ['gina']                                                      got []
goals   43
   -> 9/15 pass; 6 fail


   DEFECT LOG (each defect found by a failing query, diagnosed and fixed):
   D1  S4 Eligible(gina, MA101) fails and B3 Barred(?s) returns gina as well as frank: rule 'Barred(s) :- DisciplinaryCase(s,k)' ignored whether the
case is still ACTIVE (closed case wrongly bars).  FIX: add Active(k) to the body.
   D2  F2 Eligible(dave, CS301) succeeds although dave failed CS101, and B2 Needs(CS301, ?p) has no answer: the draft only checked DIRECT prerequisites
(no transitive closure).  FIX: add predicate Needs with a base rule and a recursive rule; MissingPrereq uses Needs.
   D3  B4 FeesCleared(?s) raises an error (floundering): the body 'not FeeDue(s), Student(s)' evaluates the negative literal while s is unbound.  FIX:
put Student(s) first so that s is bound before the negation is evaluated.


   QUERY SUITE ON THE FINAL KB
   PASS  S1 Eligible(alice, CS201)                        expected True                                                          got True
goals   26
   PASS  S2 Eligible(carol, CS201)  [approved leave]      expected True                                                          got True
goals   26
   PASS  S3 Eligible(hari, CS101)                         expected True                                                          got True
goals   20
   PASS  S4 Eligible(gina, MA101)  [closed case]          expected True                                                          got True
goals   21
   PASS  S5 Passed(alice, CS101)                          expected True                                                          got True
goals    3
   PASS  F1 Eligible(bob, CS201)  [attendance]            expected False                                                         got False
goals    8
   PASS  F2 Eligible(dave, CS301)  [indirect prerequisite]  expected False                                                       got False
goals   23
   PASS  F3 Eligible(erin, MA101)  [fees due]             expected False                                                         got False
goals   18
   PASS  F4 Eligible(frank, MA101)  [active case]         expected False                                                         got False
goals   21
   PASS  F5 Eligible(ida, CS101)  [not enrolled]          expected False                                                         got False
goals    4
   PASS  B1 Eligible(?s, CS201)                           expected ['alice', 'carol']                                            got ['alice',
'carol']                                      goals   52
   PASS  B2 Needs(CS301, ?p)                              expected ['CS101', 'CS201']                                            got ['CS101',
'CS201']                                      goals    9
   PASS  B3 Barred(?s)                                    expected ['frank']                                                     got ['frank']
goals    4
   PASS  B4 FeesCleared(?s)                               expected ['alice', 'bob', 'carol', 'dave', 'frank', 'gina', 'hari', ' got ['alice', 'bob',
'carol', 'dave', 'frank', 'gina', 'hari', ' goals   11
   PASS  B5 Eligible(?s, MA101)                           expected ['gina']                                                      got ['gina']
goals   52
   -> 15/15 pass


   Proof (explanation) for Eligible(alice, CS201):
     rule   Eligible(?s, ?c) :- AttendanceOK(?s, ?c), PrereqsCleared(?s, ?c), FeesCleared(?s), not Barred(?s)
       rule   AttendanceOK(?s, ?c) :- Enrolled(?s, ?c), Attendance(?s, ?c, ?p), MinAtt(?c, ?m), Ge(?p, ?m)
         fact   Enrolled(alice, CS201)
         fact   Attendance(alice, CS201, 82)
         fact   MinAtt(CS201, 75)
         fact   Ge(82, 75)
       rule   PrereqsCleared(?s, ?c) :- Enrolled(?s, ?c), not MissingPrereq(?s, ?c)
         fact   Enrolled(alice, CS201)
         not     MissingPrereq(alice, CS201)  (no proof found: negation as failure)
       rule   FeesCleared(?s) :- Student(?s), not FeeDue(?s)
         fact   Student(alice)
         not     FeeDue(alice)  (no proof found: negation as failure)
       not     Barred(alice)  (no proof found: negation as failure)
   Proof size 13 nodes


C  GROWTH OF THE KB WITH s STUDENTS AND c COURSES
   Propositional encoding, rules per student s and course c (counted by the program):  per (s,c): 2 attendance rules + (c_index) missing-prerequisite
rules + 1 prerequisite rule + 1 eligibility rule;  per student: 2 rules (fees, barred)
     N_rules(s, c) = s * [ 2 + sum_{j=0}^{c-1} (4 + j) ]  =  s * [ 2 + 4c + c(c-1)/2 ]       (the dependence is linear in s and quadratic in c because
of the prerequisite chain)
   First-order encoding: the number of AXIOMS is the constant 10 executable rules (14 axioms in the full theory), whatever s and c are; only the FACTS
(data) grow with s and c.


     s   c | prop rules   formula | FOL axioms FOL facts | prop sentences (rules+facts) FOL sentences | edit cost: sentences changed to alter the
attendance minimum
    10   5 |        320       320 |         10       108 |                          400           118 | prop: 50 pf-facts recomputed (one per student-
course) ;  FOL: 1 fact per course (5)
   100   5 |      3,200     3,200 |         10       941 |                        3,996           951 | prop: 500 pf-facts recomputed (one per student-
course) ;  FOL: 1 fact per course (5)
  1000   5 |     32,000    32,000 |         10     9,662 |                       40,056         9,672 | prop: 5,000 pf-facts recomputed (one per
student-course) ;  FOL: 1 fact per course (5)
    10  20 |      2,720     2,720 |         10       430 |                        3,027           440 | prop: 200 pf-facts recomputed (one per student-
course) ;  FOL: 1 fact per course (20)
   100  20 |     27,200    27,200 |         10     3,802 |                       30,282         3,812 | prop: 2,000 pf-facts recomputed (one per
student-course) ;  FOL: 1 fact per course (20)
  1000  20 |    272,000   272,000 |         10    37,131 |                      302,387        37,141 | prop: 20,000 pf-facts recomputed (one per
student-course) ;  FOL: 1 fact per course (20)
  3000  20 |    816,000   816,000 |         10   111,967 |                      907,450       111,977 | prop: 60,000 pf-facts recomputed (one per
student-course) ;  FOL: 1 fact per course (20)


D  INFERENCE COST: 100 eligibility queries Eligible(S_i, C_j) (random pairs, seed 5).  prop = indexed ground backward chaining with memo;  FOL =
backward chaining with unification, indexed facts;  times are totals of the 100 queries
     s   c | prop build s prop memory MB prop goals prop query ms | FOL build s FOL memory MB FOL goals FOL unifications FOL query ms | answers agree
break-even #queries
    10   5 |        0.000            0.1        579          0.48 |       0.000           0.0       962            1,341        27.17 |          True
1
```

```
   100   5 |       0.002          1.1       679         1.40 |       0.000          0.1     1,174          1,609       24.49 |         True
6
  1000   5 |       0.066         11.6       692         1.30 |       0.008          1.3     1,147          1,604       40.12 |         True
151
    10  20 |       0.003          0.8       587         0.81 |       0.000          0.1     1,225          4,586       46.75 |         True
5
   100  20 |       0.041          7.3       620         1.18 |       0.001          0.3     1,191          4,666       47.16 |         True
87
  1000  20 |       0.496         74.6       697         1.06 |       0.032          3.5     1,149          4,184       24.19 |         True
2,005
  3000  20 |       1.685        224.6       578         1.41 |       0.091          9.3       963          3,835       48.64 |         True
3,376
```

B.4 Task 4 (t4_output.txt)

```
TASK 4 - Unification and lifted inference


A  UNIFICATION TESTS (the five pairs derived by hand in the report; results below come from the program)
  P1  Knows(John, ?x)  vs  Knows(?y, Mother(?y))
      with occurs check: {?x/Mother(John), ?y/John}      without: {?x/Mother(John), ?y/John}
  P2  Likes(?x, ?x)  vs  Likes(Anna, Bob)
      with occurs check: FAILURE      without: FAILURE
  P3  P(?x, f(?x))  vs  P(?y, ?y)          (occurs check)
      with occurs check: FAILURE      without: CYCLIC binding (infinite term, e.g. ?x = f(f(f(...))))
  P4  Knows(John, ?x)  vs  Knows(?x, Elizabeth)   (both sentences use the name ?x: standardising apart is essential)
      with occurs check: FAILURE      without: FAILURE
  P5  R(?x, g(?y, ?z), ?z)  vs  R(h(?w), ?v, k(?w))
      with occurs check: {?v/g(?y, k(?w)), ?x/h(?w), ?z/k(?w)}      without: {?v/g(?y, k(?w)), ?x/h(?w), ?z/k(?w)}
  P6  Q(a, ?x)  vs  Q(b, ?x)   (clash of constants)
      with occurs check: FAILURE      without: FAILURE


   P4 after standardising apart (rename the second sentence's variable ?x to ?x1): Knows(John, ?x)  vs  Knows(?x1, Elizabeth):
       {?x/Elizabeth, ?x1/John}   -> common instance: Knows(John, Elizabeth)
   and P3 without the occurs check yields the cyclic binding ?y/?x, ?x/f(?x) -> unsound: it would unify P(x,f(x)) with P(y,y), which no ground instance
satisfies.


   Composition of substitutions: s1 = {?x/f(?y), ?z/?w}, s2 = {?y/a, ?w/b}:
      compose = {?w/b, ?x/f(a), ?y/a, ?z/b} ; check apply(apply(t,s1),s2) == apply(t,compose): True


B  LIFTED MODUS PONENS  (rule  Parent(?x,?y) & Parent(?y,?z) => Grandparent(?x,?z);  facts Parent(Ann,Bob), Parent(Bob,Cy))
      unify Parent(?x, ?y) with Parent(Ann, Bob)  ->  {?x/Ann, ?y/Bob}
      unify Parent(?y, ?z) with Parent(Bob, Cy)  ->  {?x/Ann, ?y/Bob, ?z/Cy}
      conclusion: Grandparent(Ann, Cy)


C  GROUND-CLAUSE GROWTH WITH THE NUMBER OF CONSTANTS k (rule R1: Edge(?x,?y) & Edge(?y,?z) => Path(?x,?z)  has 3 variables; R2: Edge(?x,?y) =>
Path(?x,?y) has 2;  predicate arity r in the table below)
  k | 3 variables: k^3   counted | 2 variables k^2 | arity of the head predicate / number of distinct variables: r=1     k r=2     k^2 r=3       k^3
r=4         k^4 r=5           k^5
  3 |              27        27 |               9 |                                                                    3           9            27
81                243
  5 |             125       125 |              25 |                                                                    5          25           125
625             3,125
 10 |           1,000     1,000 |             100 |                                                                   10         100         1,000
10,000           100,000
 20 |           8,000     8,000 |             400 |                                                                   20         400         8,000
160,000         3,200,000
 30 |          27,000    27,000 |             900 |                                                                   30         900        27,000
810,000        24,300,000
   (for k=30 the 27,000 instances were counted by formula; k <= 20 were generated and counted; arity r needs k^r instances for a rule whose r variables
are all different)


D  FUNCTION SYMBOL: ground terms of nesting depth <= d  (constants: k = 2;  f unary;  g binary)
 d |  only f: 1 unary  formula k(d+1) |      f and g   recurrence T_d = k + T_(d-1) + T_(d-1)^2
 0 |                2              2 |           2                                          2
 1 |                4              4 |           8                                          8
 2 |                6              6 |          74                                         74
 3 |                8              8 |        5552                                       5552
 4 |               10             10 |           -                                   30830258
   the counts of the program equal the recurrence N_d = k + N_(d-1) + N_(d-1)^2 for {f, g} (N = 2, 8, 74, 5,552) and k(d+1) for f alone


E  LIFTED versus PROPOSITIONAL INFERENCE on the same query
   KB: Edge facts on a chain c0->c1->...->c(k-1) rules: R2 and R1' = Path(?x,?z) :- Edge(?x,?y), Path(?y,?z).  Query: Path(c0, ?z)  (all answers) and
Path(c0, c(k-1)) (yes/no)
   k | ground rules  prop FC firings prop FC time s | lifted BC goals unifications    time s | lifted FC firings    time s | answers
   5 |         150             600         0.0005 |             15          18    0.0002 |                10    0.0001 | 4 (prop: 4)
  10 |       1,100           9,900         0.0052 |             30          38    0.0003 |                45    0.0008 | 9 (prop: 9)
  20 |       8,400         159,600         0.0539 |             60          78    0.0006 |               190    0.0045 | 19 (prop: 19)
  30 |      27,900         809,100         0.2380 |             90         118    0.0010 |               435    0.0136 | 29 (prop: 29)
  40 |      65,600       2,558,400         0.6797 |            120         158    0.0015 |               780    0.0306 | 39 (prop: 39)
  60 |     219,600      12,956,400         3.3903 |            180         238    0.0026 |             1,770    0.0925 | 59 (prop: 59)


   Query concerning ONE constant in a KB where almost all facts are irrelevant (k=60, query Path(c58, ?z)): lifted backward chaining touches only the
relevant constants
      answers 1, goals expanded 6, unifications 6;  propositionalisation still creates 219,600 ground rules
```

B.5 Task 5 (t5_output.txt)

```
TASK 5 - First-order resolution prover


A  CONVERSION TO CNF, every step (statements 1-4 and the negated query)


  S1: Ax ((Emp(?x) & Ed ((Doc(?d) & Restricted(?d)) & Handled(?x, ?d))) => Auth(?x))
      1. eliminate => and <=>                    Ax (~(Emp(?x) & Ed ((Doc(?d) & Restricted(?d)) & Handled(?x, ?d))) | Auth(?x))
      2. move ~ inwards (quantifier duality)     Ax ((~Emp(?x) | Ad ((~Doc(?d) | ~Restricted(?d)) | ~Handled(?x, ?d))) | Auth(?x))
      3. standardise variables apart             Ax1 ((~Emp(?x1) | Ad2 ((~Doc(?d2) | ~Restricted(?d2)) | ~Handled(?x1, ?d2))) | Auth(?x1))
      4. Skolemise (remove existential quantifiers) Ax1 ((~Emp(?x1) | Ad2 ((~Doc(?d2) | ~Restricted(?d2)) | ~Handled(?x1, ?d2))) | Auth(?x1))
      5. drop universal quantifiers              ((~Emp(?x1) | ((~Doc(?d2) | ~Restricted(?d2)) | ~Handled(?x1, ?d2))) | Auth(?x1))
      6. distribute | over & : clauses           [~Emp(?x1) | ~Doc(?d2) | ~Restricted(?d2) | ~Handled(?x1, ?d2) | Auth(?x1)]
```

```
  S2: Ad ((Doc(?d) & ContainsPD(?d)) => Restricted(?d))
      1. eliminate => and <=>                  Ad (~(Doc(?d) & ContainsPD(?d)) | Restricted(?d))
      2. move ~ inwards (quantifier duality)   Ad ((~Doc(?d) | ~ContainsPD(?d)) | Restricted(?d))
      3. standardise variables apart           Ad3 ((~Doc(?d3) | ~ContainsPD(?d3)) | Restricted(?d3))
      4. Skolemise (remove existential quantifiers) Ad3 ((~Doc(?d3) | ~ContainsPD(?d3)) | Restricted(?d3))
      5. drop universal quantifiers            ((~Doc(?d3) | ~ContainsPD(?d3)) | Restricted(?d3))
      6. distribute | over & : clauses         [~Doc(?d3) | ~ContainsPD(?d3) | Restricted(?d3)]


  S3: Ee (Emp(?e) & Ad ((Doc(?d) & Audited(?d)) => Handled(?e, ?d)))
      1. eliminate => and <=>                  Ee (Emp(?e) & Ad (~(Doc(?d) & Audited(?d)) | Handled(?e, ?d)))
      2. move ~ inwards (quantifier duality)   Ee (Emp(?e) & Ad ((~Doc(?d) | ~Audited(?d)) | Handled(?e, ?d)))
      3. standardise variables apart           Ee4 (Emp(?e4) & Ad5 ((~Doc(?d5) | ~Audited(?d5)) | Handled(?e4, ?d5)))
      4. Skolemise (remove existential quantifiers) (Emp(Sk1) & Ad5 ((~Doc(?d5) | ~Audited(?d5)) | Handled(Sk1, ?d5)))      Skolem: ?e4 -> Sk1   (a
constant: the existential is not inside any universal)
      5. drop universal quantifiers            (Emp(Sk1) & ((~Doc(?d5) | ~Audited(?d5)) | Handled(Sk1, ?d5)))
      6. distribute | over & : clauses         [Emp(Sk1)]   [~Doc(?d5) | ~Audited(?d5) | Handled(Sk1, ?d5)]


  S4: ((Doc(R1) & ContainsPD(R1)) & Audited(R1))
      1. eliminate => and <=>                  ((Doc(R1) & ContainsPD(R1)) & Audited(R1))
      2. move ~ inwards (quantifier duality)   ((Doc(R1) & ContainsPD(R1)) & Audited(R1))
      3. standardise variables apart           ((Doc(R1) & ContainsPD(R1)) & Audited(R1))
      4. Skolemise (remove existential quantifiers) ((Doc(R1) & ContainsPD(R1)) & Audited(R1))
      5. drop universal quantifiers            ((Doc(R1) & ContainsPD(R1)) & Audited(R1))
      6. distribute | over & : clauses         [Doc(R1)]   [ContainsPD(R1)]   [Audited(R1)]


  NEGATED QUERY  ~Ex(Emp(x) & Auth(x)): ~Ex (Emp(?x) & Auth(?x))
      1. eliminate => and <=>                  ~Ex (Emp(?x) & Auth(?x))
      2. move ~ inwards (quantifier duality)   Ax (~Emp(?x) | ~Auth(?x))
      3. standardise variables apart           Ax6 (~Emp(?x6) | ~Auth(?x6))
      4. Skolemise (remove existential quantifiers) Ax6 (~Emp(?x6) | ~Auth(?x6))
      5. drop universal quantifiers            (~Emp(?x6) | ~Auth(?x6))
      6. distribute | over & : clauses         [~Emp(?x6) | ~Auth(?x6)]


  Clause set: 7 axiom clauses + 1 negated-goal clause
      [~Emp(?x1) | ~Doc(?d2) | ~Restricted(?d2) | ~Handled(?x1, ?d2) | Auth(?x1)]
      [~Doc(?d3) | ~ContainsPD(?d3) | Restricted(?d3)]
      [Emp(Sk1)]
      [~Doc(?d5) | ~Audited(?d5) | Handled(Sk1, ?d5)]
      [Doc(R1)]
      [ContainsPD(R1)]
      [Audited(R1)]
      [~Emp(?x6) | ~Auth(?x6)]


B  PROOF THAT AN AUTHORISED EMPLOYEE EXISTS (plain breadth-first resolution, occurs check on)
  result PROVED: generated 97, resolution steps 507, unifications 12544, proof length 18, time 13.8 ms
    1. [Emp(Sk1)]                                        given
    2. [~Emp(?v7) | ~Doc(?v8) | ~Restricted(?v8) | ~Handled(?v7, ?v8) | Auth(?v7)]given
    3. [~Doc(?v36) | ~Restricted(?v36) | ~Handled(Sk1, ?v36) | Auth(Sk1)]resolve 1, 2
    4. [Doc(R1)]                                         given
    5. [~Restricted(R1) | ~Handled(Sk1, R1) | Auth(Sk1)]   resolve 3, 4
    6. [~Emp(?v19) | ~Auth(?v19)]                        given
    7. [~Auth(Sk1)]                                      resolve 6, 1
    8. [~Restricted(R1) | ~Handled(Sk1, R1)]             resolve 5, 7
    9. [~Doc(?v13) | ~ContainsPD(?v13) | Restricted(?v13)] given
   10. [~ContainsPD(R1) | Restricted(R1)]                resolve 4, 9
   11. [ContainsPD(R1)]                                  given
   12. [Restricted(R1)]                                  resolve 10, 11
   13. [~Handled(Sk1, R1)]                               resolve 8, 12
   14. [~Doc(?v16) | ~Audited(?v16) | Handled(Sk1, ?v16)]  given
   15. [~Audited(R1) | Handled(Sk1, R1)]                 resolve 4, 14
   16. [Audited(R1)]                                     given
   17. [Handled(Sk1, R1)]                                resolve 15, 16
   18. []                                                resolve 13, 17


C  STRATEGY COMPARISON (same clause set)
strategy                    result   clauses generated resolution steps unifications given clauses proof length  time ms
plain (FIFO, breadth-first)  PROVED                 97              507        12544            96           18     15.3
unit preference             PROVED                  42               53          358            19           18      1.0
unit pref. + every 5th oldest PROVED                 42               53          359            19           18      1.1
set of support              PROVED                  50              128          896            47           15      2.6


   Proof found by the set-of-support strategy:
    1. [~Emp(?v839) | ~Auth(?v839)]                      given
    2. [~Emp(?v827) | ~Doc(?v828) | ~Restricted(?v828) | ~Handled(?v827, ?v828) | Auth(?v827)]given
    3. [~Emp(?v841) | ~Doc(?v842) | ~Restricted(?v842) | ~Handled(?v841, ?v842)]resolve 1, 2
    4. [~Doc(?v833) | ~ContainsPD(?v833) | Restricted(?v833)]given
    5. [~Emp(?v848) | ~Doc(?v849) | ~Handled(?v848, ?v849) | ~ContainsPD(?v849)]resolve 3, 4
    6. [Emp(Sk1)]                                        given
    7. [~Doc(?v869) | ~Handled(Sk1, ?v869) | ~ContainsPD(?v869)]resolve 5, 6
    8. [~Doc(?v836) | ~Audited(?v836) | Handled(Sk1, ?v836)]given
    9. [~Doc(?v918) | ~ContainsPD(?v918) | ~Audited(?v918)]resolve 7, 8
   10. [Doc(R1)]                                         given
   11. [~ContainsPD(R1) | ~Audited(R1)]                  resolve 9, 10
   12. [ContainsPD(R1)]                                  given
   13. [~Audited(R1)]                                    resolve 11, 12
   14. [Audited(R1)]                                     given
   15. []                                                resolve 13, 14


D  SECOND QUERY THAT IS NOT ENTAILED:  'some authorised employee handled a document R2'  Ex(Emp(x) & Auth(x) & Handled(x, R2))
  KB as given (finite Herbrand universe {Sk1, R1}):
    plain : SATURATED (no proof exists: the clause set is satisfiable); generated 205, steps 1195, time 52.8 ms
    sos   : SATURATED (no proof exists: the clause set is satisfiable); generated 136, steps 413, time 17.7 ms
  KB extended with S5: every employee has an employee manager  Ax(Emp(x) => Ey(Emp(y) & ReportsTo(x,y)))  (Skolem FUNCTION f(x): infinite Herbrand
universe)
  S5: Ax (Emp(?x) => Ey (Emp(?y) & ReportsTo(?x, ?y)))
      1. eliminate => and <=>                  Ax (~Emp(?x) | Ey (Emp(?y) & ReportsTo(?x, ?y)))
      2. move ~ inwards (quantifier duality)   Ax (~Emp(?x) | Ey (Emp(?y) & ReportsTo(?x, ?y)))
      3. standardise variables apart           Ax1997 (~Emp(?x1997) | Ey1998 (Emp(?y1998) & ReportsTo(?x1997, ?y1998)))
      4. Skolemise (remove existential quantifiers) Ax1997 (~Emp(?x1997) | (Emp(Sk2(?x1997)) & ReportsTo(?x1997, Sk2(?x1997))))      Skolem: ?y1998 ->
Sk2(?x1997)   (a function of the universal variables ?x1997 in whose scope it occurs)
      5. drop universal quantifiers            (~Emp(?x1997) | (Emp(Sk2(?x1997)) & ReportsTo(?x1997, Sk2(?x1997))))
      6. distribute | over & : clauses         [~Emp(?x1997) | Emp(Sk2(?x1997))]   [~Emp(?x1997) | ReportsTo(?x1997, Sk2(?x1997))]
   same two queries, step cap 20,000 resolution steps / 20,000 clauses:
    entailed query (authorised employee exists)    plain : PROVED                        generated   1374, steps   5895, max clause size 5, time 0.28s
    entailed query (authorised employee exists)    unit  : DIVERGED (term nesting exceede generated      0, steps      0, max clause size 0, time 0.00s
    entailed query (authorised employee exists)    unitfair: CAP                           generated  11304, steps  20025, max clause size 5, time
42.43s
    entailed query (authorised employee exists)    sos   : PROVED                        generated    186, steps    413, max clause size 5, time 0.02s
    NOT entailed query (handled R2)                plain : CAP                           generated   4289, steps  20028, max clause size 6, time 1.67s
    NOT entailed query (handled R2)                unit  : DIVERGED (term nesting exceede generated      0, steps      0, max clause size 0, time 0.00s
```

```
    NOT entailed query (handled R2)              unitfair: CAP                          generated   9780, steps  20030, max clause size 6, time
57.34s
    NOT entailed query (handled R2)              sos   : CAP                          generated   6058, steps  20002, max clause size 6, time
29.32s


E  COMPARISON WITH THE PROPOSITIONAL PROVER OF TASK 2 ON AN EQUIVALENT SMALL DOMAIN
   The FOL clauses are instantiated with every constant of a domain (Sk1, R1 and n extra constants) and given to the propositional resolution prover
(logic.pl_resolution).
domain size ground clauses | propositional: generated attempted  time ms | FOL (set of support): generated  steps  time ms
          2             14 |                      104       309      1.4 |                             50    128      4.1   (propositional result:
True)
          4             32 |                      180       495      2.0 |                             50    128      3.5   (propositional result:
True)
          8             92 |                      404     1,011      5.3 |                             50    128      2.8   (propositional result:
True)
         14            242 |                      920     2,145     14.6 |                             50    128      2.6   (propositional result:
True)
```

B.6 Task 6 (t6_output.txt)

```
TASK 6 - Forward and backward chaining on an organisation hierarchy


A  THE KNOWLEDGE BASE (10 definite clauses)
   r1  Above(?x, ?y) :- ReportsTo(?y, ?x)
   r2  Above(?x, ?y) :- ReportsTo(?y, ?z), Above(?x, ?z)
   r3  Peer(?x, ?y) :- ReportsTo(?x, ?m), ReportsTo(?y, ?m)
   r4  SameDept(?x, ?y) :- DeptMember(?d, ?x), DeptMember(?d, ?y)
   r5  CanApprove(?x, ?y) :- Above(?x, ?y), Senior(?x)
   r6  Escalation(?x, ?m) :- ReportsTo(?x, ?m)
   r7  SkipLevel(?x, ?g) :- Escalation(?x, ?m), Escalation(?m, ?g)
   r8  TeamMember(?x, ?m) :- Above(?m, ?x), Lead(?m)
   r9  Collab(?x, ?y) :- SameDept(?x, ?y), Peer(?x, ?y)
   r10 Reviewer(?x, ?y) :- Peer(?x, ?y), Senior(?x)


B  FORWARD CHAINING (Student A):  naive = every rule is re-matched against all facts in every iteration;  semi-naive = a rule instance must use a fact
derived in the previous iteration
base facts employees |       mode iterations facts inferred  rule firings   redundant redundant %  unifications   time s
         112        47 |      naive          5          1,195         5,313       4,118       77.5%        15,720     0.05
         112        47 | semi-naive          5          1,195         1,195           0        0.0%        15,720     0.04
       1,117       467 |      naive          7         13,841        84,417      70,576       83.6%       254,300     1.28
       1,117       467 | semi-naive          7         13,841        13,841           0        0.0%       254,300     1.07
      11,167     4,667 |      naive          8        154,671     1,030,838     876,167       85.0%     3,070,851    58.70
      11,167     4,667 | semi-naive          8        154,671       154,671           0        0.0%     3,070,851    56.54


   Derived facts by predicate (semi-naive result) and relevance to three queries
      112 base facts, 47 employees, leaf e46 at depth 3:  Above 114, CanApprove 114, Collab 156, Escalation 46, Peer 180, Reviewer 44, SameDept 449,
SkipLevel 42, TeamMember 50   total derived 1,195
        Q1 CanApprove(e0, leaf)      answers     1  relevant derived facts      4  irrelevant  99.67% of 1,195
        Q4 CanApprove(?x, leaf)      answers     3  relevant derived facts      9  irrelevant  99.25% of 1,195
        Q3 Above(e0, ?y)             answers    46  relevant derived facts     46  irrelevant  96.15% of 1,195
    1,117 base facts, 467 employees, leaf e466 at depth 5:  Above 1,882, CanApprove 1,882, Collab 1,584, Escalation 466, Peer 1,860, Reviewer 464,
SameDept 4,649, SkipLevel 462, TeamMember 592   total derived 13,841
        Q1 CanApprove(e0, leaf)      answers     1  relevant derived facts      6  irrelevant  99.96% of 13,841
        Q4 CanApprove(?x, leaf)      answers     5  relevant derived facts     20  irrelevant  99.86% of 13,841
        Q3 Above(e0, ?y)             answers   466  relevant derived facts    466  irrelevant  96.63% of 13,841
   11,167 base facts, 4667 employees, leaf e4666 at depth 6:  Above 26,184, CanApprove 26,184, Collab 15,864, Escalation 4,666, Peer 18,660, Reviewer
4,664, SameDept 46,649, SkipLevel 4,662, TeamMember 7,138   total derived 154,671
        Q1 CanApprove(e0, leaf)      answers     1  relevant derived facts      7  irrelevant 100.00% of 154,671
        Q4 CanApprove(?x, leaf)      answers     6  relevant derived facts     27  irrelevant  99.98% of 154,671
        Q3 Above(e0, ?y)             answers 4,666  relevant derived facts  4,666  irrelevant  96.98% of 154,671


C  BACKWARD CHAINING (Student B): SLD resolution with substitutions, indexed facts (predicate + first or second argument), plain mode
   Q1 CanApprove(e0, leaf) [ground, true]   Q2 CanApprove(leaf, e0) [ground, false]   Q3 Above(e0, ?y) [all subordinates of the root]   Q4
CanApprove(?x, leaf) [all who may approve the leaf]   Q5 CanApprove(e1, ?y) [subordinates of a mid-level manager]
base facts |                     query solutions     goals unifications max depth  repeated   time ms = FC answer
       112 |     Q1 CanApprove(e0,leaf)         1        14           16         5         0      0.20        True
       112 |     Q2 CanApprove(leaf,e0)         0         4            3         2         0      0.05        True
       112 |           Q3 Above(e0,?y)        46       345          416         4       306      5.66        True
       112 |     Q4 CanApprove(?x,leaf)         3        16           18         5         4      0.27        True
       112 |       Q5 CanApprove(e1,?y)        20       366          437         5       325      4.15        True
     1,117 |     Q1 CanApprove(e0,leaf)         1        20           24         7         0      0.22        True
     1,117 |     Q2 CanApprove(leaf,e0)         0         4            3         2         0      0.06        True
     1,117 |           Q3 Above(e0,?y)       466     5,649        7,068         6     5,295     71.24        True
     1,117 |     Q4 CanApprove(?x,leaf)         5        24           28         7         6      0.29        True
     1,117 |       Q5 CanApprove(e1,?y)       210     5,860        7,279         7     5,504    103.03        True
    11,167 |     Q1 CanApprove(e0,leaf)         1        23           28         8         0      0.33        True
    11,167 |     Q2 CanApprove(leaf,e0)         0         4            3         2         0      0.07        True
    11,167 |           Q3 Above(e0,?y)     4,666    78,555      100,076         7    75,051   1269.21        True
    11,167 |     Q4 CanApprove(?x,leaf)         6        28           33         8         7      0.39        True
    11,167 |       Q5 CanApprove(e1,?y)     1,364    79,920      101,441         8    76,414   1115.19        True


D  DIRECTION OF INFERENCE: work for ONE query (ground query Q1) versus the whole forward-chaining closure
base facts |  FC firings FC unifications FC time s | BC goals  BC unif. BC time ms |  unification ratio FC/BC  queries before FC pays off (time)
       112 |       1,195          15,720      0.04 |       14        16      0.197 |                      982                              223
     1,117 |      13,841         254,300      1.07 |       20        24      0.219 |                   10,596                            4,888
    11,167 |     154,671       3,070,851     56.54 |       23        28      0.327 |                  109,673                          172,783


E  LEFT-RECURSIVE RULE:  AboveL(x,y) :- AboveL(x,z), ReportsTo(y,z)   (+ base AboveL(x,y) :- ReportsTo(y,x));  query AboveL(e0, ?y) on the 1,000-fact
KB
   plain backward chaining: returned 466 answers; goals expanded before the 200,000-goal budget stopped it: 200,000; maximum depth reached: 432; budget
exhausted: True; answers produced: 466
   (answers do appear, because the base clause is tried first, but the search never finishes: after the base clause it descends AboveL(e0,?z1),
AboveL(e0,?z2), ... and every goal is a VARIANT of its parent, so no branch ever fails, and the program cannot know that it has found all the answers)
   correct answer set has 466 answers (e0 is above all 466 other employees)


   Remedy 1 - loop check (cut a goal that is a variant of one of its ancestors):
      terminates; answers 4 of 466 correct -> INCOMPLETE (it cuts the recursive branch, so only the base clause answers: direct reports);  goals 3,
unifications 6, cut branches 1


   Remedy 2 - depth bound (limit L), re-run for increasing L (iterative deepening); a bound gives termination, and completeness only when L is large
enough; there is no way to know L in advance
        L  answers      goals  unifications      cut  complete
        1        0          3            2        2     False
        2        4          5            8        2     False
        3       20         11           30        2     False
        4       84         33          116        2     False
```

```
        5      340        119           458        2     False
        6      466        461           926        2      True
        7      466        929         1,394        2      True
        8      466      1,397         1,862        2      True
      cumulative cost of iterative deepening up to the last L: goals 2,958, unifications 4,796


   Remedy 3 - memoisation / tabling (one table per call variant, evaluate until no table changes):
      base facts  answers  complete  passes  tables     goals  unifications table hits   time ms
             112       46      True       3      48        79           300          3      1.6
           1,117      466      True       5     468       929         3,676          5     22.2
          11,167    4,666      True       6   4,668     6,496        28,770          6    168.8


F  REPEATED SUBGOALS AND CYCLES (right-recursive Above, r1-r2)
   F1 cyclic data: ReportsTo(a,b), ReportsTo(b,a), ReportsTo(c,b);  query Above(z, c) with z not in the data (fails):
      plain BC: goals 3,000, max depth 1,000, exhausted budget: True  (does not terminate)
      loop check on the failing query Above(z, c): terminates, answers 0, goals 10, cut 1
      loop check on Above(a, c) [true]: answers 1, goals 10, cut 1
      forward chaining on the same cyclic data: terminates, 6 Above facts, 3 iterations (the fixed point is finite because facts are never repeated)
   F2 matrix organisation (each employee has a second, dotted-line manager) -> the reports-to graph is a DAG; query Above(e_k, leaf) that FAILS, so
every path is explored:
       facts       mode      goals  unifications   repeated repeated %
         110      plain         21            26          6      28.6%
         110     tabled         34            56          4      11.8%
       1,080      plain         60            78         33      55.0%
       1,080     tabled         64           128         10      15.6%
      10,797      plain        171           226        132      77.2%
      10,797     tabled         98           210         20      20.4%
```

B.7 Task 7 (t7_output.txt)

```
TASK 7 (Student B) - EXPERIMENTAL VALIDATION OF THE MEDICATION-SAFETY ADVISOR


A  FIRST DRAFT KB (39 rules): 37 test cases
   FAIL T11 spironolactone prescribed, patient on lisinopril: expected ['interaction:spironolactone+lisinopril'], got []
   FAIL T13 trimethoprim prescribed, patient on methotrexate: expected ['interaction:trimethoprim+methotrexate'], got []
   FAIL T18 diclofenac, NSAID allergy: expected ['allergy:nsaid:diclofenac'], got []
   -> 34/37 pass
   DEFECTS FOUND: (1) T11 and T13 - the two one-directional interaction rules (lisinopril->spironolactone, methotrexate->trimethoprim) missed the
reverse order; fix: add the two mirrored rules.   (2) T18 - NSAID allergy list had no diclofenac although diclofenac is an NSAID fact; fix: add allergy
rule (nsaid, diclofenac).


B  FINAL KB: 37 test cases with per-case evidence
   rules in the final KB: 42 (38 drug-triggered + 1 general + 3 helper/summary)
   case                                                                ok  goals  unif expl. nodes      us
   T01 warfarin + aspirin (dangerous combination)                    PASS      9    10           3     129
   T02 aspirin prescribed, patient on warfarin (reverse order)       PASS     15    15           3     234
   T03 ibuprofen prescribed, patient on warfarin                     PASS     13    13           3     103
   T04 sertraline + tramadol (serotonin syndrome)                    PASS      7     8           3      64
   T05 tramadol prescribed, patient on sertraline                    PASS      9     9           3      85
   T06 sertraline + phenelzine (MAOI)                                PASS      7     8           3      62
   T07 simvastatin + clarithromycin                                  PASS      7     7           3      65
   T08 sildenafil + nitroglycerin                                    PASS      5     5           3      48
   T09 nitroglycerin prescribed, patient on sildenafil               PASS      5     5           3      49
   T10 lisinopril + spironolactone (hyperkalaemia)                   PASS      7     7           3      65
   T11 spironolactone prescribed, patient on lisinopril              PASS      5     5           3      47
   T12 methotrexate + trimethoprim                                   PASS      5     5           3      47
   T13 trimethoprim prescribed, patient on methotrexate              PASS      5     5           3      73
   T14 amoxicillin, penicillin allergy                               PASS      5     5           3      53
   T15 cefalexin, penicillin allergy (cross-reactivity)              PASS      5     5           3      54
   T16 sulfamethoxazole, sulfa allergy                               PASS      5     5           3      47
   T17 ibuprofen, NSAID allergy                                      PASS     12    12           3      95
   T18 diclofenac, NSAID allergy                                     PASS      6     6           3      54
   T19 ibuprofen, kidney disease                                     PASS     12    13           3      94
   T20 metformin, kidney disease                                     PASS      5     5           3      50
   T21 lisinopril, pregnancy                                         PASS      7     7           3      63
   T22 warfarin, pregnancy                                           PASS      9     9           3      73
   T23 aspirin, child aged 8                                         PASS     14    13           5     110
   T24 aspirin, boundary age 12 (not a child)                        PASS     14    13           0     100
   T25 tramadol, boundary age 11 (child)                             PASS      9     8           5     207
   T26 diazepam, age 80                                              PASS      7     6           5      96
   T27 diazepam, boundary age 74 (not elderly)                       PASS      7     6           0     197
   T28 diazepam, boundary age 75 (elderly)                           PASS      7     6           5     104
   T29 warfarin + aspirin + pregnancy (two reasons)                  PASS      9    11           6     136
   T30 aspirin, child 9, NSAID allergy, on warfarin, ulcer (four reasons) PASS     15    17          14     164
   T31 ibuprofen, patient on naproxen (duplicate NSAID)              PASS     13    14           5     167
   T32 ibuprofen, patient already on ibuprofen                       PASS     13    14           5     171
   T33 simvastatin, liver disease, on clarithromycin (two reasons)   PASS      7     8           6      81
   T34 paracetamol, full risky history (safe: no rule applies)       PASS      3     2           0      44
   T35 amoxicillin, no allergy (safe)                                PASS      5     4           0     103
   T36 amoxicillin, sulfa allergy only (wrong allergen, safe)        PASS      5     5           0      59
   T37 metformin, patient on ibuprofen, no condition (safe)          PASS      5     4           0      48
   -> 37/37 pass; dangerous cases 31, safe cases 6


   Example explanation (T30, four independent reasons):
      rule   Danger(?p, allergy:nsaid:aspirin) :- Rx(?p, aspirin), Allergy(?p, nsaid)
        fact   Rx(P30, aspirin)
        fact   Allergy(P30, nsaid)
      rule   Danger(?p, interaction:aspirin+warfarin) :- Rx(?p, aspirin), Takes(?p, warfarin)
        fact   Rx(P30, aspirin)
        fact   Takes(P30, warfarin)


C  KB-SIZE SCALING: 2,000 patients, one prescription each; indexed (rules by trigger drug) versus un-indexed (every Danger rule is tried)
 rules      mode |  goals/q   unif/q rule tries/q  mean us   p95 us explain nodes (dangerous) dangerous % | KB build s KB mem MB query peak KB |  agree
    42   indexed |      8.0      9.9          3.0       80      167                       5.2       62.0% |       0.05       2.3           135 |
    42 unindexed |     44.1     82.1         39.2      523      677                       5.2       62.0% |       0.04       2.3           183 |   True
    42  FC (all) | firings 4,511  unifications 75,100  facts inferred 3,798  iterations 4  total time 0.34s = 169 us per patient (amortised over all
2,000 patients, answers agree with BC: True)
   400   indexed |     12.3     17.2          5.6      123      233                       4.1       50.2% |       0.04       2.4           172 |
   400 unindexed |    404.4    801.5        397.8     5652     8227                       4.1       50.2% |       0.04       2.4           187 |   True
   400  FC (all) | firings 3,140  unifications 106,840  facts inferred 2,923  iterations 4  total time 0.43s = 216 us per patient (amortised over all
2,000 patients, answers agree with BC: True)
  2000   indexed |     13.4     19.0          6.2      178      353                       4.0       47.8% |       0.09       2.8           180 |
  2000 unindexed |   2005.1   4002.3       1997.9    29625    43463                       4.0       47.8% |       0.06       2.5           211 |   True
  2000  FC (all) | firings 2,892  unifications 115,260  facts inferred 2,725  iterations 4  total time 0.37s = 185 us per patient (amortised over all
2,000 patients, answers agree with BC: True)


D  SCALE TEST: 2,000-rule KB and 50,000 patients (about 6 facts each)
```

```
   facts 341,062; build 5.95s; memory 120 MB;  3,000 random queries: mean 158 us, p95 341 us, max 1400 us, mean steps (goals+unifications) 32.0,
dangerous 48.0%
   forward chaining over all 50,000 patients (baseline): 30.9s, firings 74,660, unifications 2,887,520, facts inferred 69,875;  one new patient
admitted -> BC 158 us versus FC re-run unless made incremental
   propositional encoding (one copy of every rule per patient) would need 50,000 x 2,000 = 100,000,000 ground rules; the first-order rules stay at
2,000
```